Інтелектуальний Аналіз Даних

Навальний посібник

Косович І. Т. Матвій О. В. Мельник В. С. Мельник Г. В.

Чернівецький Національний Університет, 2026

1. Збір даних (Data Collection)

Першим етапом практично будь-якого проєкту з інтелектуального аналізу даних, машинного навчання або аналітики є збір даних (Data Collection). Якість отриманих результатів значною мірою залежить від якості та повноти вихідних даних. Навіть найскладніші алгоритми машинного навчання не здатні компенсувати відсутність важливої інформації або низьку якість даних, тому процес їх отримання є одним із найважливіших етапів підготовки аналітичного проєкту.

Сучасні дані можуть надходити з великої кількості різноманітних джерел. Це можуть бути реляційні бази даних, електронні таблиці, CSV-файли, журнали подій (logs), сенсори Інтернету речей (IoT), соціальні мережі, веб-сайти, а також численні програмні інтерфейси (API), що надають доступ до інформаційних сервісів. У деяких випадках дані вже доступні у вигляді готових файлів і можуть бути безпосередньо завантажені для подальшого аналізу. В інших ситуаціях необхідно виконувати запити до баз даних або зовнішніх інформаційних систем для отримання актуальної інформації.

Автоматизований збір даних є значно швидшим, дозволяє обробляти великі обсяги інформації, зменшує кількість людських помилок і забезпечує регулярне оновлення даних без постійного втручання користувача. Основними труднощами під час збору даних є різні формати даних, неоднакова структура, дублікати, пропущені значення, обмеження доступу, необхідність автентифікації та різна якість отриманої інформації.

В цій темі ми поговоримо про збір даних з двох основних джерел:

- запит до API (більшість з яких сьогодні є веб-орієнтованими); та
- вилучення даних з веб-сторінки за допомогою технологій веб-скрапінгу (Web Scraping).

Збір даних з веб-джерел (Collecting data from web-based sources)

Веб-джерело даних — це будь-який ресурс у мережі Інтернет, з якого можна отримати інформацію, наприклад веб-сторінка, REST API або онлайн-база даних. Для збору даних можуть використовуватися веб-сайти, REST API, XML- та JSON-сервіси, відкриті набори даних, хмарні сервіси та веб-додатки.

Сьогодні переважна більшість автоматизованих систем обміну даними використовує протокол HTTP (HyperText Transfer Protocol). Хоча спочатку HTTP був створений для передачі веб-сторінок між браузерами та веб-серверами, з часом він став універсальним механізмом взаємодії між програмними системами. Саме на основі HTTP функціонує більшість сучасних веб-сервісів, REST API та хмарних платформ.

Під час збору даних клієнтська програма надсилає HTTP-запит до віддаленого сервера, а сервер повертає відповідь, яка може містити HTML-документ, JSON-структуру, XML-файл, зображення або інші типи даних. Розуміння принципів роботи HTTP є необхідною передумовою для ефективного

використання веб-джерел даних, тому в наступних розділах буде розглянуто основні типи HTTP-запитів, структуру відповідей сервера та способи обробки отриманих даних у Python.

HTTP-відповідь — це повідомлення, яке сервер надсилає клієнту після обробки HTTP-запиту. Вона містить код стану, заголовки та тіло відповіді, у якому можуть передаватися HTML, JSON, XML або інші дані.

Веб-скрапінг — це автоматизований процес отримання та вилучення інформації з HTML-сторінок веб-сайтів для її подальшого аналізу або обробки. Процес веб-скрапінгу включає отримання HTML-сторінки, аналіз її структури, пошук потрібних елементів, вилучення необхідних даних та їх подальше збереження або обробку.

```
import requests
response = requests.get("https://fmi.chnu.edu.ua/")

print("Status Code:", response.status_code)
print("Headers:", response.headers)

print(response.text)
```

```
Status Code: 200
Headers: {'Date': 'Fri, 12 Sep 2025 05:44:43 GMT', 'Content-Type': 'text/html;
charset=utf-8' ...
```

Коди стану HTTP повідомляють клієнту про результат обробки запиту сервером. Код 200 (OK) означає, що запит успішно виконано. Код 404 (Not Found) вказує на те, що запитуваний ресурс не знайдено. Код 500 (Internal Server Error) свідчить про виникнення внутрішньої помилки на сервері під час обробки запиту.

HTTP GET є найпоширенішим методом, але існують також методи PUT, POST, DELETE, які змінюють деякі параметри на сервері.

Параметри URL можна вказати за допомогою бібліотеки запитів, як показано нижче:

```
params = {"sa":"t", "rct":"j", "q":"", "esrc":"s",
"source":"web", "cd":"9", "cad":"rja", "uact":"8"}
response = requests.get("http://www.google.com/url", params=params)

print("Final URL:", response.url)
print(response.text)
```

```
Final URL: http://www.google.com/url?
sa=t&rct=j&q=&esrc=s&source=web&cd=9&cad=rja&uact=8
```

```
<html lang="uk"><head><meta http-equiv="Content-Type" content="text/html;
charset=ISO-8859-1"><title>&#1055 ...
```

RESTful API

Якщо ви перейдете від простого запиту веб-сторінок до веб-API, ви, швидше за все, зіткнетеся з REST API (Representational State Transfer). REST — це скоріше архітектура дизайну, але ось кілька ключових моментів:

- Використовує стандартний інтерфейс і методи HTTP (GET, PUT, POST, DELETE)
- Stateless – сервер не запам'ятовує, що ви робили
- Практичне правило: якщо ви надсилаєте ключ свого облікового запису разом із кожним викликом API, ви, ймовірно, використовуєте REST API

REST API забезпечує стандартизований, швидкий і надійний доступ до актуальних даних, дозволяє автоматизувати їх отримання та легко інтегрується з різними програмними системами. Ви запитуєте REST API подібно до стандартних HTTP-запитів, але майже завжди потрібно включати параметри. Параметри можуть передаватися як частина URL-адреси, через рядок запиту (Query Parameters), у заголовках HTTP або в тілі запиту залежно від використовуваного HTTP-методу.

Автентифікація API — це процес підтвердження особи користувача або програми перед наданням доступу до ресурсів сервера. Найпоширенішими способами є API Key, Bearer Token (JWT), Basic Authentication та OAuth 2.0.

Отримайте власний токен доступу на https://github.com/settings/tokens/new GitHub API використовує GET/PUT/DELETE, щоб ви могли автоматично запитувати або оновлювати елементи у вашому обліковому записі GitHub Приклад REST: сервер не запам'ятовує ваші останні запити, наприклад, ви завжди повинні включати свій токен доступу, якщо використовуєте його таким чином

```
import os
from dotenv import load_dotenv

load_dotenv()
token = os.getenv("GITHUB_TOKEN")

headers = {'Authorization': 'Bearer '+token}
response = requests.get("https://api.github.com/user", headers=headers)
print(response.content)
```

```
b'{"login":"maxvonlancaster","id":38358502,"node_id":"MDQ6VXNlcjM4MzU4NTAy","avata
r_url": ...
```

REST-архітектура базується на взаємодії клієнта і сервера, використанні стандартних HTTP-методів, відсутності збереження стану між запитами (stateless), єдиному інтерфейсі доступу до ресурсів та представленні ресурсів за допомогою URI. Дані найчастіше передаються у форматах JSON або XML.

HTTP-заголовки використовуються для передачі службової інформації між клієнтом і сервером. Вони можуть містити тип передаваних даних, параметри автентифікації, інформацію про кешування, мову, кодування, тип клієнта та інші параметри, необхідні для коректної обробки запиту або відповіді.

CSV-файли

CSV (Comma-Separated Values): Відноситься до будь-якого текстового файлу з роздільниками (не завжди розділеного комами).

Якщо самі значення містять коми, ви можете взяти їх у лапки (наш реєстратор, очевидно, завжди так робить, щоб бути впевненим).

Імпортуємо дані із csv-файлу, що містить дані про історичну ціну біткоїна, та зобразимо ці дані, за допомогою наступного коду.

```
import pandas as pd

dataframe = pd.read_csv("../resources/bitcoin.csv", delimiter=',', quotechar='"')
dataframe.head(10)
```

```
Date    Open    High    Low Close   Volume  Currency
0   2010-07-18  0.0 0.1 0.1 0.1 75      USD
1   2010-07-19  0.1 0.1 0.1 0.1 574     USD
2   2010-07-20  0.1 0.1 0.1 0.1 262     USD
3   2010-07-21  0.1 0.1 0.1 0.1 575     USD
4   2010-07-22  0.1 0.1 0.1 0.1 2160    USD
5   2010-07-23  0.1 0.1 0.1 0.1 2403    USD
6   2010-07-24  0.1 0.1 0.1 0.1 496     USD
7   2010-07-25  0.1 0.1 0.1 0.1 1551    USD
8   2010-07-26  0.1 0.1 0.1 0.1 877     USD
9   2010-07-27  0.1 0.1 0.1 0.1 3374    USD
```

Формат CSV є простим, компактним і зрозумілим, підтримується більшістю програм для роботи з даними та легко обробляється різними мовами програмування. Він забезпечує швидке зчитування й запис табличних даних, що робить його одним із найпоширеніших форматів для обміну інформацією.

Формат CSV не підтримує вкладені структури даних, типи даних, метадані та форматування. Крім того, можуть виникати проблеми через використання різних роздільників, кодувань символів або символів нового рядка, що іноді ускладнює обмін даними між різними програмами.

Парсинг JSON в Python

JSON (JavaScript Object Notation) — це легкий текстовий формат обміну даними, який використовується для передачі структурованої інформації між програмами. Незважаючи на назву, JSON сьогодні є незалежним від JavaScript і підтримується практично всіма сучасними мовами програмування.

JSON широко використовується у REST API, веб-сервісах, мобільних застосунках та хмарних системах. Основними перевагами JSON є компактність, простота читання людиною та легкість автоматичної

обробки.

Вбудована бібліотека для читання/запису об'єктів Python з/у файли JSON, імпортується за допомогою команди `import json`. Нижче наведено приклад роботи із json-даними в python. REST-запит вертає json дані, які ми потім відображаємо.

```
import json
# завантаження JSON-даних за допомогою виклику REST API

headers = {'Authorization': 'token ' + token}
response = requests.get("https://api.github.com/user", headers=headers)
data = json.loads(response.content)

#json.load(file) # завантажуємо json з файлу
json.dumps(data) # повертаємо json строку
#json.dump(obj, file) # записуємо json у файл
```

```
'{"login": "maxvonlancaster", "id": 38358502, "node_id": "MDQ6VXNlcjM4MzU4NTAy",
"avatar_url": ...
```

JSON підтримує рядки, числа, логічні значення, масиви, об'єкти, а також спеціальне значення `null`. JSON-файл можна зчитати за допомогою стандартного модуля `json`, використовуючи функцію `json.load()`, або за допомогою бібліотеки `pandas`, якщо дані мають табличну структуру.

Серіалізація JSON — це процес перетворення об'єкта Python у JSON-рядок або JSON-файл для його збереження чи передачі. Десеріалізація JSON — це процес перетворення JSON-рядка або JSON-файлу у відповідні об'єкти Python.

Після десеріалізації json в python значення вкладених полів отримують шляхом послідовного звернення до вкладених словників або списків за допомогою ключів та індексів.

Нижче наведено приклад роботи з вкладеними структурами в JSON:

```
import json

json_string = '''
{
    "employee":
    {
        "name":"John",
        "department":
        {
            "name":"IT",
            "floor":4
        }
    }
}
```

```
'''

obj = json.loads(json_string)

print(obj["employee"]["department"]["name"])
```

```
IT
```

Парсинг XML/HTML в Python

XML (eXtensible Markup Language) — це мова розмітки, призначена для зберігання, структурування та обміну даними між різними інформаційними системами. XML-документ складається з декларації, кореневого елемента, вкладених тегів, атрибутів та текстового вмісту. XML використовує теги для опису структури даних і є більш громіздким, тоді як JSON має компактніший синтаксис, легше читається людиною та швидше обробляється програмами.

Існує ряд XML/HTML-парсерів для Python, але для науки про дані добре підходить бібліотека BeautifulSoup (спеціально призначена для вилучення даних з XML/HTML-файлів). Нижче наведено приклад завантаження html-розмітки із сайту, та код, що знаходить всі anchor-посилання всередені елементів `<div>`.

```
# отримати всі посилання на веб-сторінці
from bs4 import BeautifulSoup
import requests

response = requests.get("https://mathmod.chnu.edu.ua/")

root = BeautifulSoup(response.content)
root.find("div").findAll("a")


# пошук елемента за класом
item = root.find(class_="flex-item")
print(item.text)


# пошук за CSS селектором
titles = root.select("div.flex-item h2")

for t in titles:
    print(t.text)


# отримання атрибутів HTML-тега
link = root.find("a")

print(link["href"])
print(link.text)
```

```
# отримання всіх рядків таблиці
table = root.find("table")

rows = table.find_all("tr")

for row in rows:
    cells = row.find_all("td")
    print([cell.text.strip() for cell in cells])
```

```
[<a href="/cdn-cgi/l/email-protection#77141b105a1a16031f37141f190259121302590216">
<span><span class="__cf_email__" data-
cfemail="a0c3ccc78dcdc1d4c8e0c3c8ced58ec5c4d58ed5c1">[email protected]</span>
</span></a>,
 <a href="https://www.facebook.com/fmi.org.ua/" rel="noopener" target="_blank"
title="Факультет математики та інформатики ЧНУ ім. Ю. Федьковича"><span
class="iconify-icon footer-social-link-icon icon--fa-brands icon--fa-brands--
facebook-f"></span></a>,
 <a href="https://www.instagram.com/m._i_.f/" rel="noopener" target="_blank"
title="Профбюро студентів ФМІ"><span class="iconify-icon footer-social-link-icon
icon--fa-brands icon--fa-brands--instagram"></span></a>,
 <a href="https://youtube.com/@MathTube_" rel="noopener" target="_blank"
title="Youtube"><span class="iconify-icon footer-social-link-icon icon--fa-brands
icon--fa-brands--youtube"></span></a>]
```

HTML-документ — це текстовий файл, який містить структуру веб-сторінки, описану за допомогою HTML-тегів. HTML є основним джерелом інформації під час веб-скрапінгу, оскільки саме в ньому містяться текст, таблиці, посилання, зображення та інші елементи веб-сторінки.

HTML-елемент за тегом можна знайти за допомогою методів `find()` або `find_all()`, вказавши назву потрібного тегу. Елементи знаходять, вказуючи значення атрибутів, наприклад `class`, `id` або інших, використовуючи параметри методів `find()` чи `find_all()`.

DOM (Document Object Model) — це деревоподібне представлення HTML-документа, у якому кожен HTML-елемент є окремим вузлом, доступним для аналізу та модифікації.

Регулярні вирази (Regular expressions)

Після завантаження даних (або якщо вам потрібно створити парсер для завантаження даних іншого формату) часто доводиться шукати певні елементи в даних.

Регулярні вирази (Regular Expressions, скорочено RegEx або Regex) — це спеціальна мова шаблонів, призначена для пошуку, перевірки, виділення та заміни текстових фрагментів. Регулярні вирази широко застосовуються під час збору, очищення та попередньої обробки даних, оскільки дозволяють швидко знаходити рядки, що відповідають заданому шаблону. У задачах інтелектуального аналізу даних регулярні вирази використовуються для пошуку даних (електронна пошта, номери телефонів, дати/час),

перевірки правильності введених даних, очищення тексту від службових символів, та попередньої підготовки текстів перед їх аналізом.

Наприклад, знайти перше входження рядка «data science»

```
import re
text = "This course will introduce the basics of data science"
match = re.search(r"data science", text)
print(match.start())
print(re.match(r"This", text))

# Регулярні вирази в Python

match = re.match(r"data science", text) # перевірити, чи збігається початок тексту
match = re.search(r"data science", text) # знайти перший збіг або None

all_matches = re.findall(r"a", text) # повернути всі збіги
print(all_matches)
```

```
41
<re.Match object; span=(0, 4), match='This'>
['a', 'a', 'a']
```

Під час збору та підготовки даних регулярні вирази використовуються для автоматичного вилучення структурованої інформації з текстів. Типовими прикладами є пошук електронних адрес, телефонних номерів, URL-адрес, IP-адрес, поштових індексів, дат, часу, грошових сум та інших шаблонних даних.

Крім пошуку інформації, регулярні вирази широко застосовуються для очищення текстів перед аналізом: видалення HTML-тегів, спеціальних символів, зайвих пробілів, службових позначок та інших небажаних елементів. Вони також використовуються для перевірки правильності введених даних (валідації) та підготовки текстових корпусів до подальшої обробки методами обробки природної мови (Natural Language Processing, NLP).

Завдяки простоті використання та високій швидкодії регулярні вирази є одним із базових інструментів інженера даних і спеціаліста з інтелектуального аналізу даних.

Відповідність декількох потенційних символів (Matching multiple potential characters)

Справжня сила регулярних виразів полягає в можливості збігу декількох можливих послідовностей символів. Спеціальні символи в регулярних виразах: `.^$*+?{}\[]|()` (якщо ви хочете, щоб ці символи збігалися точно, вам потрібно їх екранувати: `\$`)

Відповідність наборів символів:

- Відповідність символу «a»: `a`
- Відповідність символів «a», «b» або «c»: `[abc]`
- Будь-який символ, крім «a», «b» або «c»: `[^abc]`
- Відповідність будь-якої цифри: `\d` (те саме, що `[0-9]`)

- Відповідність будь-якого алфавітно-цифрового символу: `\w` (те саме, що `[a-zA-z0-9_]`)
- Відповідність пробілу: `\s` (те саме, що `[ \t\n\r\f\v]`)
- Відповідність будь-якого символу: `.` (включно з новою лінією з re.DOTALL)

Може збігатися з одним або декількома випадками символу (або набору символів)

Деякі поширені модифікатори:

- Відповідність символу «a» рівно один раз: `a`
- Відповідність символу «a» нуль або один раз: `a?`
- Відповідність символу «a» нуль або більше разів: `a*`
- Відповідність символу «a» один або більше разів: `a+`
- Відповідність символу «a» рівно n разів: `a{n}`

Можна поєднувати їх із збігом декількох символів:

- Відповідність усіх випадків «`<something>` science», де `<something>` — це алфавітно-цифровий рядок, що містить принаймні один символ: `\w+\s+science`

Групування (Grouping)

Часто нам потрібно отримати більше інформації, ніж просто те, чи знайшли ми збіг чи ні (наприклад, ми можемо хотіти знати, який текст збігся).

Групування: оберніть частини регулярного виразу в дужки, щоб «запам'ятати» ці частини збігу `(\w+)\s([Ss]cience)`.

```
text = "This course will introduce the basics of data science"
match = re.search(r"(\w+)\s([Ss]cience)", text)
print(match.start(), match.groups())
# Чому перед рядком стоїть «r»? Це дозволяє уникнути необхідності подвійного
екранування рядків
```

```
41 ('data', 'science')
```

Заміни (Substitutions)

Регулярні вирази надають механізм для заміни одного тексту іншим текстом.

```
better_text = re.sub(r"data science", r"schmada science", text)

# Щоб включити текст, який було збережено під час пошуку за допомогою груп,
використовуйте екрановані послідовності
# \1, \2, … у тексті заміни
better_text = re.sub(r"(\w+)\s([Ss])cience", r"\1 \2hmience", text)
print(better_text)
```

```
This course will introduce the basics of data shmience
```

Порядок і жадібне зіставлення (Ordering and greedy matching)

У регулярних виразах існує порядок операцій. `abc|def` відповідає рядкам «abc» або «def», а не «ab(c або d)ef». Ви можете обійти це, використовуючи дужки, наприклад `a(bc|de)f`. Це також створює групу, використовуйте `a(?:bc|de)f`, якщо ви не хочете її захоплювати.

За замовчуванням регулярні вирази намагаються захопити якомога більше тексту (жадібне співставлення). `<(.*)>`, застосоване до `<a>text</a>`, буде відповідати всьому виразу. Якщо ви хочете захопити якомога менше тексту, використовуйте `<(.*?)>`, це буде відповідати тільки терміну `<a>`.

Питання для самоконтролю.

1. Що таке збір даних (Data Collection) та яке його значення в процесі інтелектуального аналізу даних?
2. Які основні джерела даних використовуються в Data Mining?
3. Які переваги має автоматизований збір даних порівняно з ручним?
4. Які труднощі можуть виникати під час збору даних з різних джерел?
5. Що таке веб-джерело даних?
6. Які типи веб-джерел можуть використовуватися для збору даних?
7. Що таке веб-скрапінг (Web Scraping)?
8. Які етапи процесу веб-скрапінгу?
9. Які бібліотеки Python найчастіше використовуються для отримання веб-сторінок?
10. Для чого використовується бібліотека `requests`?
11. Що таке HTTP-запит?
12. Які основні HTTP-методи використовуються при роботі з веб-ресурсами?
13. Що таке HTTP-відповідь?
14. Яке призначення кодів стану HTTP?
15. Що означають коди стану 200, 404 та 500?
16. Що таке RESTful API?
17. Які принципи лежать в основі REST-архітектури?
18. Чим REST API відрізняється від звичайної веб-сторінки?
19. Які переваги використання REST API для збору даних?
20. Як передаються параметри в REST API-запитах?
21. Що таке JSON-відповідь REST API?
22. Для чого використовуються HTTP-заголовки (headers)?
23. Що таке автентифікація API?
24. Які способи автентифікації API ви знаєте?
25. Що таке CSV-файл?
26. Які особливості структури CSV-файлів?
27. Які переваги має формат CSV?
28. Які недоліки має формат CSV?
29. Як зчитати CSV-файл у Python?
30. Для чого використовується бібліотека `pandas` при роботі з CSV?
31. Що таке JSON?

32. Які типи даних підтримує формат JSON?
33. Як зчитати JSON-файл у Python?
34. Що таке серіалізація JSON?
35. Що таке десеріалізація JSON?
36. Як отримати значення вкладених полів у JSON-структурі?
37. Що таке XML?
38. Які основні елементи XML-документа?
39. Чим XML відрізняється від JSON?
40. Як виконати парсинг XML у Python?
41. Що таке HTML-документ?
42. Яку роль HTML відіграє у веб-скрапінгу?
43. Для чого використовується бібліотека BeautifulSoup?
44. Як знайти HTML-елемент за тегом?
45. Як знайти HTML-елемент за атрибутом або класом?
46. Що таке DOM-структура веб-сторінки?
47. Що таке регулярний вираз (Regular Expression)?
48. Для яких задач використовуються регулярні вирази під час збору даних?
49. Які основні метасимволи регулярних виразів ви знаєте?
50. Як за допомогою регулярних виразів знаходити та витягувати потрібні фрагменти тексту?

2. Обробка даних (Data Processing)

Обробка даних (Data Processing) — це сукупність методів і процедур, спрямованих на підготовку даних до подальшого аналізу та побудови моделей машинного навчання. Основними етапами обробки є очищення даних, усунення пропущених значень і дублікатів, виявлення викидів, перетворення типів даних, а також масштабування та нормалізація ознак. Якість виконання цих етапів безпосередньо впливає на точність, швидкість і надійність моделей машинного навчання, оскільки алгоритми навчаються саме на підготовлених даних. «Сирі» дані часто містять пропущені значення, дублікати, помилки введення, викиди, некоректні формати та неоднорідні масштаби ознак, що може призводити до погіршення якості аналізу та зниження точності прогнозування.

Pandas

Існує ряд бібліотек Python, які обробляють реляційні дані, зазвичай написані як інтерфейси до декількох різних систем управління реляційними базами даних (таких як PostgreSQL, MySQL та інші). [Примітка: слід зазначити, що програмне забезпечення, таке як PostreSQL і MySQL, правильніше називати системами управління реляційними базами даних (RDBMS), а не базами даних. База даних — це фактичні таблиці та записи, що визначають фактичну сукупність даних.

Однак у цьому курсі ми будемо працювати з реляційними даними переважно за допомогою двох бібліотек: Pandas і SQLite. Це особливо прості бібліотеки, якщо говорити про реальні бази даних: Pandas, безумовно, не є реальною системою реляційних баз даних (хоча вона надає функції, що віддзеркалюють деякі їхні можливості), тоді як SQLite є «реальною» RDBMS, але надзвичайно простою, без стандартної архітектури клієнт/сервер, яка є практично у всіх реальних виробничих базах даних. Проте для багатьох задач у галузі науки про дані вони будуть достатніми, тому ми зосередимося на них тут.

Ми вже коротко розглянули Panda, коли обговорювали збір даних, і вона виявилася однією з найкорисніших бібліотек Python для науки про дані. Як ми вже згадували вище (але будемо

повторювати цей факт багато разів), Pandas — це не бібліотека реляційних баз даних, а бібліотека «даних-рам». Ви можете уявити собі фрейм даних як 2D-масив, за винятком того, що записи у фреймі даних можуть бути будь-яким типом об'єкта Python (і мати змішані типи в масиві), а рядки/стовпці можуть мати «мітки» замість цілочисельних індексів, як у стандартному масиві.

Давайте подивимося, як спочатку створити фрейм даних у Pandas, який відображає нашу таблицю Person вище (ми залишимо стовпець «Role ID» поза увагою, щоб спростити завдання).

```
import pandas as pd

# pandas dataframe example from csv:
# dataframe = pd.read_csv("../resources/bitcoin.csv", delimiter=',',
quotechar='"')

df = pd.DataFrame([(1, 'Kolter', 'Zico'),
                   (2, 'Xi', 'Edgar'),
                   (3, 'Lee', 'Mark'),
                   (4, 'Mani', 'Shouvik'),
                   (5, 'Gates', 'Bill'),
                   (6, 'Musk', 'Elon')],
                  columns=["id", "last_name", "first_name"])
df
```

```
    id  last_name   first_name
0   1   Kolter  Zico
1   2   Xi      Edgar
2   3   Lee     Mark
3   4   Mani    Shouvik
4   5   Gates   Bill
5   6   Musk    Elon
```

«Індекс» для Pandas насправді означає щось більше, ніж «первинний ключ» у таблиці бази даних (хоча з тим винятком, що можливі дублікати записів).

Тобто індекс (якщо він зроблений правильно, без дублікатів) є ідентифікатором для кожного рядка в базі даних. Ми можемо встановити індекс для одного з існуючих стовпців за допомогою виклику `.set_index()`.

Але тут потрібно бути дуже обережним з одним моментом. За замовчуванням більшість операцій Pandas, таких як `.set_index()` та багато інших, не виконуються на місці. Тобто, хоча виклик df.set_index(«id») вище повертає копію фрейму даних df з індексом, встановленим для стовпця id (пам'ятайте, що Jupyter notebook відображає значення, яке повертається в останньому рядку комірки), оригінальний об'єкт df насправді тут не змінюється.

Якщо ми хочемо фактично змінити сам об'єкт df, потрібно використовувати прапор `inplace=True` для цих функцій (або присвоїти оригінальний об'єкт результату функції, але це не так чітко).

```
df.set_index("id", inplace=True)
df
```

```
    last_name   first_name
id
1   Kolter  Zico
2   Xi  Edgar
3   Lee Mark
4   Mani    Shouvik
5   Gates   Bill
6   Musk    Elon
```

Подивимось на іще деякі методи для роботи із фреймами в Pandas:

```
# Доступ до окремих елементів можна отримати за допомогою нотації `.loc[row, column]`, де row позначає індекс, який
# ви шукаєте, а column — назву стовпця.
df.loc[1, "last_name"]

# Якщо ми хочемо отримати доступ до всіх прізвищ (або до всіх елементів у певному рядку), ми використовуємо символ-замінник :. Наприклад
df.loc[:, "last_name"]

# Ми можемо передати список потрібних стовпців, щоб отримати об’єкт типу DataFrame:
df.loc[:, ["last_name"]]

# Те саме можна зробити з індексами рядків.
df.loc[[1,2],:]

# Крім того, ми можемо використовувати `.loc` для зміни вмісту існуючих записів:
df.loc[1,"last_name"] = "Kilter"
df

# Ми можемо навіть додати додаткові рядки/стовпці
df.loc[7,:] = ('Moore', 'Andrew')
df

# Наостанок пам’ятайте, що `.loc` завжди індексує на основі «індексу» (тобто, фактично, первинного ключа) фрейму даних
# разом з назвою стовпця. Якщо ви хочете отримати доступ на основі позиційного індексу (тобто, використовуючи лічильники з нульовим індексом
# як для рядків, так і для стовпців), ви можете скористатися властивістю `.iloc`
df.iloc[4,1]
```

Методи очищення даних та обробка відсутніх значень.

У реальному світі необроблені дані рідко бувають ідеальними. Вони часто містять помилки, невідповідності та відсутні значення. Очищення даних — це процес виявлення та виправлення цих проблем для поліпшення якості даних.

- *Дублікати*: це повторювані записи в наборі даних. Дублікати рядків можуть спотворити ваш аналіз або модель, особливо якщо вони представляють одну й ту саму подію або об'єкт.
- *Неправильні типи даних*: дані, які введені неправильно (наприклад, числа, збережені як рядки), можуть призвести до помилок в аналізі та навчанні моделі.
- *Випадкові значення*: екстремальні значення, які значно відрізняються від решти даних. Вони можуть спотворити результати статистичного аналізу або моделей машинного навчання.

Відсутність даних може відбуватися з різних причин, таких як помилки введення даних, несправності обладнання або проблеми з конфіденційністю. Важливо обережно поводитися з відсутніми значеннями, оскільки неправильне поводження може призвести до упереджених результатів.

Типи відсутніх даних:

- *MCAR* (Missing Completely at Random, повністю випадкова відсутність): ймовірність відсутності даних не залежить від самих даних.
- *MAR* (відсутність випадкова): ймовірність відсутності даних пов'язана з іншими спостережуваними даними, але не з самими відсутніми даними.
- *MNAR* (відсутність невипадкова): відсутність пов'язана з самими неспостережуваними даними.

Видалення відсутніх даних:

Видалення рядків або стовпців з відсутніми значеннями є найпростішим підходом, але може призвести до втрати цінної інформації.

```
import pandas as pd

df = pd.read_csv('../resources/data.csv')
print(df, '\n')
df_cleaned = df.dropna()  # Видалити всі рядки, у яких є хоча б одне відсутнє значення
# Видалити рядки, у яких є хоча б одне відсутнє значення в числових стовпцях
df_feature_cleaned = df.dropna(subset=['feature1', 'feature2'])
print(df_cleaned, '\n')
print(df_feature_cleaned, '\n')
```

```
   feature1  feature2 categorical_column  target
0       1.2       3.2                 A      10
1       2.3       NaN                 B      15
2       3.1       1.1                 C       7
3       4.7       4.8                 A      18
4       NaN       2.7                 B      11
5       5.1       4.2                 C      19
6       2.8       NaN                 A      13
7       4.1       3.9                 B      16
8       5.3       4.1               NaN      14
```

```
9       3.3       2.1                  C      12

   feature1  feature2 categorical_column  target
0       1.2       3.2                  A      10
2       3.1       1.1                  C       7
3       4.7       4.8                  A      18
5       5.1       4.2                  C      19
7       4.1       3.9                  B      16
9       3.3       2.1                  C      12

   feature1  feature2 categorical_column  target
0       1.2       3.2                  A      10
2       3.1       1.1                  C       7
3       4.7       4.8                  A      18
5       5.1       4.2                  C      19
7       4.1       3.9                  B      16
8       5.3       4.1                NaN      14
9       3.3       2.1                  C      12
```

Mean Imputation (заповнення середнім): Замінити відсутні значення середнім значенням стовпця.

```
df['feature1'].fillna(df['feature1'].mean(), inplace=True)
df['feature2'].fillna(df['feature2'].mean(), inplace=True)
df
```

```
feature1    feature2    categorical_column  target
0   1.200000    3.2000  A   10
1   2.300000    3.2625  B   15
2   3.100000    1.1000  C   7
3   4.700000    4.8000  A   18
4   3.544444    2.7000  B   11
5   5.100000    4.2000  C   19
6   2.800000    3.2625  A   13
7   4.100000    3.9000  B   16
8   5.300000    4.1000  NaN 14
9   3.300000    2.1000  C   12
```

Median Imputation: Заміщення медіанним значенням. Корисно, коли дані є викривленими.

```
df = pd.read_csv('../resources/data.csv')
df['feature1'].fillna(df['feature1'].median(), inplace=True)
df
```

```
    feature1    feature2    categorical_column  target
0   1.2 3.2 A   10
1   2.3 NaN B   15
2   3.1 1.1 C   7
3   4.7 4.8 A   18
4   3.3 2.7 B   11
5   5.1 4.2 C   19
6   2.8 NaN A   13
7   4.1 3.9 B   16
8   5.3 4.1 NaN 14
9   3.3 2.1 C   12
```

Mode Imputation: Заміщення модою. Часто використовується для категоріальних даних.

```
df['categorical_column'].fillna(df['categorical_column'].mode()[0], inplace=True)
df
```

```
feature1    feature2    categorical_column  target
0   1.2 3.2 A   10
1   2.3 NaN B   15
2   3.1 1.1 C   7
3   4.7 4.8 A   18
4   3.3 2.7 B   11
5   5.1 4.2 C   19
6   2.8 NaN A   13
7   4.1 3.9 B   16
8   5.3 4.1 A   14
9   3.3 2.1 C   12
```

Розширене заповнення даних:

Імпутування K-найближчих сусідів (KNN): оцінює відсутні значення на основі значень найближчих сусідів.

Багатовимірне імпутування (підстановка): враховує взаємозв'язки між ознаками для імпутування відсутніх значень.

Створення індикаторних змінних: інший підхід полягає у створенні бінарної індикаторної змінної, яка позначає наявність відсутніх даних.

Масштабування та нормалізація даних (Data Scaling and Normalization)

Масштабування та нормалізація даних є важливими етапами попередньої обробки, які можуть суттєво вплинути на ефективність ваших моделей.

Масштабування даних гарантує, що всі ознаки однаково впливають на процес навчання моделі. Коли ознаки мають дуже різні діапазони, ті, що мають більші діапазони, можуть домінувати в обчисленнях

відстані або кроках градієнта, що призводить до неоптимальних моделей.

Алгоритми на основі відстані: В алгоритмах, таких як K-найближчі сусіди (KNN) або машини опорних векторів (SVM), відстань між точками даних має вирішальне значення. Якщо одна ознака має набагато більший діапазон, ніж інші, вона непропорційно впливатиме на метрику відстані, що може спотворити результати.

Градієнтний спуск: Алгоритми, такі як лінійна регресія та нейронні мережі, використовують градієнтний спуск для мінімізації функції вартості. Якщо ознаки не масштабуються, алгоритм градієнтного спуску може потребувати більше часу для збіжності або може збігатися до неоптимального рішення через нерівномірні кроки, зроблені в просторі параметрів.

Мінімально-максимальне масштабування

Перемасштабовує дані до фіксованого діапазону, зазвичай [0, 1]. Ця техніка особливо корисна, коли ви знаєте, що ваші дані відповідають розподілу з чіткими верхньою та нижньою межами.

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- Використовуйте мінімально-максимальне масштабування, коли ви хочете, щоб усі ваші ознаки мали однаковий масштаб (наприклад, в алгоритмах, таких як KNN, SVM).
- Особливо корисно, коли дані розподілені по відомому і фіксованому діапазону.

```
from sklearn.preprocessing import MinMaxScaler
import pandas as pd

data = {'feature': [10, 50, 100, 150, 200]}
df = pd.DataFrame(data)

scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(df)

print(scaled_data)
```

```
[[0.        ]
 [0.21052632]
 [0.47368421]
 [0.73684211]
 [1.        ]]
```

```
df = pd.read_csv('../resources/data.csv')

# Вибираємо числові стовпці для масштабування
numerical_cols = df[['feature1', 'feature2']]

# Розділяємо стовпці категорій та цільові стовпці
```

```
categorical_cols = df[['categorical_column', 'target']]

from sklearn.preprocessing import MinMaxScaler

# IInціалізуємо масштабування
scaler = MinMaxScaler()

# Підготуємо та перетворюємо числові стовпці
numerical_scaled = scaler.fit_transform(numerical_cols)

# Перетворити нормалізовані дані назад у DataFrame
numerical_scaled_df = pd.DataFrame(numerical_scaled,
columns=numerical_cols.columns)

# Об’єднати нормалізовані числові дані з категоріальними стовпцями
df_scaled = pd.concat([numerical_scaled_df,
categorical_cols.reset_index(drop=True)], axis=1)

print(df_scaled)
```

```
   feature1  feature2 categorical_column  target
0  0.000000  0.567568                 A      10
1  0.268293       NaN                 B      15
2  0.463415  0.000000                 C       7
3  0.853659  1.000000                 A      18
4       NaN  0.432432                 B      11
5  0.951220  0.837838                 C      19
6  0.390244       NaN                 A      13
7  0.707317  0.756757                 B      16
8  1.000000  0.810811               NaN      14
9  0.512195  0.270270                 C      12
```

Нормалізація (стандартизація) за Z-балом (Z-Score Normalization or standardization)

Перемасштабовує дані так, щоб середнє значення дорівнювало 0, а стандартне відхилення — 1. Ця техніка корисна, коли дані мають гаусівський (нормальний) розподіл, але не обов'язково обмежені.

$$X' = \frac{X - \nu}{\sigma}$$

- Використовуйте нормалізацію за Z-балом, коли ваші дані мають нормальний розподіл.
- Це стандартний вибір для багатьох моделей машинного навчання, особливо тих, що передбачають нормальний розподіл даних.

```
from sklearn.preprocessing import StandardScaler
import pandas as pd

data = {'feature': [10, 50, 100, 150, 200]}
df = pd.DataFrame(data)
```

```
scaler = StandardScaler()
standardized_data = scaler.fit_transform(df)

print(standardized_data)
```

```
[[-1.35411306]
 [-0.76536825]
 [-0.02943724]
 [ 0.70649377]
 [ 1.44242478]]
```

Надійне масштабування (Robust Scaling)

Використовує медіану та міжквартильний розмах (IQR) для масштабування, що робить його надійним щодо винятків. На відміну від мінімально-максимального масштабування та нормалізації Z-балу, надійне масштабування менше піддається впливу екстремальних значень у даних.

$$X' = \frac{X - \text{медіана}(X)}{\text{IQR}(X)}$$

Де $\text{IQR}(X)$ — діапазон між 25-м процентилем ($Q1$) і 75-м процентилем ($Q3$).

- Використовуйте надійне масштабування, якщо ваші дані містять винятки або мають негаусівський розподіл.
- Ідеально підходить для ситуацій, коли дані є асиметричними або містять екстремальні значення, які не повинні сильно впливати на масштабування.

```
from sklearn.preprocessing import RobustScaler
import pandas as pd

data = {'feature': [10, 50, 90, 100, 110, 150, 160, 200]}
df = pd.DataFrame(data)

scaler = RobustScaler()
robust_scaled_data = scaler.fit_transform(df)

print(robust_scaled_data)
```

```
[[-1.42857143]
 [-0.85714286]
 [-0.28571429]
 [-0.14285714]
 [ 0.        ]
 [ 0.57142857]
 [ 0.71428571]
```

```
 [ 1.28571429]
 [34.14285714]]
```

Мінімально-максимальне масштабування: використовується, коли відомі межі даних або коли алгоритм вимагає даних у певному діапазоні.

Нормалізація за Z-балом: підходить для нормально розподілених даних або коли потрібно стандартизувати ознаки, щоб вони мали однакову важливість.

Надійне (Robust) масштабування: найкраще підходить для роботи з даними, що містять винятки.

Вступ до конструювання ознак (Feature Engineering)

Інженерія ознак є важливим етапом у процесі попередньої обробки даних. Вона передбачає перетворення необроблених даних на ознаки, які краще відображають основну проблему для моделей прогнозування. Якісна інженерія ознак може значно покращити ефективність моделей машинного навчання, надаючи їм найбільш релевантні та інформативні вхідні дані.

Інженерія ознак — це процес відбору, модифікації або створення нових ознак з необроблених даних для покращення ефективності моделей машинного навчання. Метою є створення ознак, які ефективніше відображають основні закономірності в даних, ніж вихідні необроблені дані. Правильна інженерія ознак може призвести до спрощення моделей, підвищення точності та скорочення часу навчання.

Створення нових ознак

Включає виведення нових змінних з існуючих, які краще відображають основну структуру даних. Це може включати математичні перетворення, агрегації або операції, специфічні для певної області.

```
import pandas as pd

data = {
    'height': [1.60, 1.75, 1.82, 1.90],
    'weight': [55, 80, 72, 90]
}
df = pd.DataFrame(data)

df
# Створюємо нову ознаку: BMI (body mass index, індекс маси тіла)
df['BMI'] = df['weight'] / (df['height'] ** 2)

df
```

```
    height  weight  BMI
0   1.60    55  21.484375
1   1.75    80  26.122449
2   1.82    72  21.736505
3   1.90    90  24.930748
```

Кодування категоріальних змінних

Алгоритми машинного навчання зазвичай вимагають числових даних. Тому категоріальні змінні необхідно перетворити в числовий формат. Існує кілька методів кодування категоріальних змінних:

Кодування *One-hot* перетворює кожну категорію в новий бінарний стовпець. Цей метод підходить для випадків, коли кількість категорій обмежена.

```
data = {'color': ['red', 'blue', 'green']}
df = pd.DataFrame(data)

# One-Hot Encoding
df_encoded = pd.get_dummies(df, columns=['color'])

print(df_encoded)
```

```
   color_blue  color_green  color_red
0           0            0          1
1           1            0          0
2           0            1          0
```

Кодування міток присвоює кожної категорії унікальне ціле число. Це простий метод, але він передбачає порядкові відносини між категоріями, що може бути недоречним у деяких випадках.

```
from sklearn.preprocessing import LabelEncoder

data = {'color': ['red', 'blue', 'green']}
df = pd.DataFrame(data)

# Label Encoding
encoder = LabelEncoder()
df['color_encoded'] = encoder.fit_transform(df['color'])

df
```

```
    color   color_encoded
0   red 2
1   blue    0
2   green   1
```

Цільове кодування передбачає заміну кожної категорії середнім значенням цільової змінної для цієї категорії. Ця техніка є корисною, коли категоріальна змінна має велику кількість рівнів.

```
data = {
    'color': ['red', 'blue', 'green', 'red', 'blue', 'green'],
    'target': [1, 0, 0, 1, 1, 0]
}
df = pd.DataFrame(data)

# Target Encoding
df['color_encoded'] = df.groupby('color')['target'].transform('mean')

print(df)
```

```
   color  target  color_encoded
0    red       1            1.0
1   blue       0            0.5
2  green       0            0.0
3    red       1            1.0
4   blue       1            0.5
5  green       0            0.0
```

Функція взаємодії

Функція взаємодії створюються шляхом поєднання двох або більше змінних для відображення взаємодії між ними. Ці функції можуть виявити взаємозв'язки, які не є очевидними при розгляді змінних окремо.

Приклад:

Припустимо, що ви маєте такі дані, як вік і дохід користувачів. Термін взаємодії можна створити шляхом множення цих двох властивостей, щоб відобразити, як дохід змінюється з віком.

```
data = {
    'age': [25, 35, 45, 55],
    'income': [30000, 50000, 70000, 90000]
}
df = pd.DataFrame(data)

# Створюємо функцію взаємодії
df['age_income_interaction'] = df['age'] * df['income']

df
```

```
    age income  age_income_interaction
0   25  30000   750000
1   35  50000   1750000
2   45  70000   3150000
3   55  90000   4950000
```

Поліноміальні ознаки

Створюються шляхом генерації нових ознак, які є поліноміальними комбінаціями існуючих ознак. Це може бути особливо корисно при моделюванні нелінійних взаємозв'язків.

Приклад:

Припустимо, у вас є одна ознака x. Ви можете створити поліноміальні ознаки, такі як x^2, x^3 тощо, які можуть допомогти у виявленні нелінійних взаємозв'язків у даних.

```
from sklearn.preprocessing import PolynomialFeatures

data = {'x': [2, 3, 4]}
df = pd.DataFrame(data)

# Створюємо поліноміальні ознаки (3-го ступеня)
poly = PolynomialFeatures(degree=3, include_bias=False)
poly_features = poly.fit_transform(df)

# Перетворюємо поліноміальні ознаки назад у DataFrame
df_poly = pd.DataFrame(poly_features, columns=poly.get_feature_names_out(['x']))

df_poly
```

```
    x   x^2 x^3
0   2.0 4.0 8.0
1   3.0 9.0 27.0
2   4.0 16.0    64.0
```

Після завершення обробки необхідно переконатися, що дані не містять пропущених значень, дублікатів, некоректних типів даних та аномальних викидів, а всі ознаки мають очікуваний формат і масштаб. Для цього використовують статистичний аналіз, перевірку структури даних, візуалізацію та засоби бібліотеки pandas, такі як `info()`, `describe()`, `isnull()` і `duplicated()`. Якісно підготовлені дані дозволяють моделям машинного навчання точніше виявляти закономірності, швидше навчатися, бути менш чутливими до шуму та краще узагальнювати знання на нових даних, що безпосередньо підвищує точність, стійкість і надійність прогнозування.

Питання для самоконтролю.

1. Що таке обробка даних (Data Processing) та яке її значення в процесі інтелектуального аналізу даних?
2. Які основні етапи включає процес обробки даних?
3. Чому якість обробки даних впливає на результати машинного навчання?
4. Які основні проблеми можуть бути присутні у «сирих» даних?
5. Що таке бібліотека pandas та для чого вона використовується?
6. Які основні структури даних підтримує бібліотека pandas?
7. Що таке DataFrame?

8. Чим DataFrame відрізняється від Series?
9. Як створити DataFrame у Python?
10. Як зчитати дані у DataFrame з CSV-файлу?
11. Як переглянути перші та останні записи DataFrame?
12. Як отримати інформацію про структуру DataFrame?
13. Як отримати статистичний опис числових даних за допомогою pandas?
14. Як вибрати окремий стовпець або кілька стовпців DataFrame?
15. Яка різниця між методами `loc[]` та `iloc[]`?
16. Як відфільтрувати рядки DataFrame за певною умовою?
17. Як додати новий стовпець до DataFrame?
18. Як видалити стовпець або рядок із DataFrame?
19. Як перейменувати стовпці DataFrame?
20. Як відсортувати дані за одним або кількома стовпцями?
21. Що таке очищення даних (Data Cleaning)?
22. Які основні причини виникнення неякісних даних?
23. Що таке пропущені значення (Missing Values)?
24. Які причини появи пропущених значень у наборах даних?
25. Як виявити пропущені значення у DataFrame?
26. Які методи використовуються для видалення пропущених значень?
27. Які методи використовуються для заповнення пропущених значень?
28. У яких випадках доцільно видаляти рядки з пропущеними значеннями?
29. Що таке імпутація даних (Data Imputation)?
30. Які статистичні методи використовуються для заповнення пропущених значень?
31. Що таке дублікати даних?
32. Як знайти дублікати у DataFrame?
33. Як видалити дублікати за допомогою pandas?
34. Що таке викиди (Outliers)?
35. Якими методами можна виявляти викиди?
36. Чому викиди можуть негативно впливати на результати аналізу даних?
37. Що таке масштабування даних (Data Scaling)?
38. Для чого виконується масштабування ознак перед навчанням моделей?
39. У чому полягає різниця між масштабуванням та нормалізацією даних?
40. Що таке Min-Max нормалізація?
41. Як обчислюється Min-Max нормалізація?
42. Що таке стандартизація (Standardization)?
43. Як обчислюється стандартизація за Z-оцінкою (Z-score)?
44. Для яких алгоритмів машинного навчання масштабування є особливо важливим?
45. Які переваги має нормалізація даних?
46. Які недоліки має Min-Max нормалізація?
47. Які переваги має стандартизація порівняно з Min-Max нормалізацією?
48. Які інструменти бібліотеки scikit-learn використовуються для масштабування та нормалізації даних?
49. Як перевірити, що дані були успішно очищені та підготовлені до аналізу?
50. Яким чином правильна обробка даних впливає на точність, стійкість і узагальнювальну здатність моделей машинного навчання?

3. Візуалізація даних (Data visualization)

Важливо підкреслити різницю між двома дуже різними поняттями, які люди описують, використовуючи загальний термін «візуалізація». Перше з них — «візуалізація для дослідження даних»: використання візуалізації для розуміння (тобто для власного розуміння) сукупності даних з метою подальшого аналізу. Коротко кажучи, мета цього типу візуалізації — «знайти істину» в даних. Альтернативою є «візуалізація для презентації»: виокремлення певного аспекту даних у вигляді зрозумілої діаграми або графіку, що передає широкій аудиторії конкретний аспект даних, який ви хочете донести. Цей тип візуалізації має на меті «переконати інших людей у правдивості ваших висновків».

Візуалізація VS статистика: Візуалізація майже завжди надає більш інформативний (хоча і менш кількісний) огляд ваших даних, ніж статистика (підхід, а не галузь науки в цілому).

Типи даних (Types of data)

Чотири основні типи даних:

- *Номінальні дані (Nominal data)*: категоріальні дані без внутрішнього порядку між категоріями. Наприклад, змінна «тип домашньої тварини» може складатися з класів {собака, кіт, кролик}, і між цими двома типами немає відносного порядку, це просто різні дискретні значення.

- *Порядкові дані (Ordinal data)*: категоріальні дані з внутрішньою послідовністю, але «відмінності» між категоріями не мають суто числового значення. Канонічним прикладом тут є відповіді на опитування з такими відповідями: {категорично не згоден, трохи не згоден, нейтральний, трохи згоден, категорично згоден}. Важливою особливістю тут є те, що, хоча між цими типами існує чітка послідовність, немає сенсу вважати, що різниця між «трохи згоден» і «категорично згоден» є «такою ж», як різниця між «нейтральний» і «трохи згоден».

- *Інтервальні дані (Interval data)*: числові дані, тобто дані, які можна відобразити на «числовій прямій»; важливим аспектом на відміну від порядкових даних є не «дискретна проти безперервної диференціації» (наприклад, цілочисельні значення можна вважати інтервальними даними), а той факт, що відносні відмінності в інтервальних даних мають значення. Класичним прикладом є температура (у Фаренгейті або Цельсії, що ми підкреслимо трохи пізніше): тут різниці між температурами мають значення: 10 і 15 градусів розділені такою ж величиною, як 15 і 20 (ця властивість настільки властива числовим даним, що майже дивно її підкреслювати). З іншого боку, інтервальні дані охоплюють випадки, коли нульова точка «не має реального значення»; на практиці це означає, що співвідношення між двома точками даних не має значення. Двадцять градусів за Фаренгейтом не є «вдвічі гарячішими» в будь-якому значущому сенсі, ніж 10 градусів; і, звичайно, не нескінченно гарячішими, ніж нуль градусів.

- *Дані про співвідношення (Ratio data)*: також числові дані, але співвідношення між вимірами має певне значення. Класичним прикладом тут є температура за Кельвіном. Очевидно, що, як і температура за Фаренгейтом або Цельсієм, це описує основне явище температури, але на відміну від попередніх випадків, нуль за Кельвіном має значення з точки зору молекулярної енергії в речовині (тобто її відсутність). Це означає, що співвідношення мають реальне значення: речовина при 20 градусах Кельвіна має вдвічі більше кінетичної енергії на молекулярному рівні, ніж та сама речовина при 10 градусах Кельвіна.

«Допустимі операції», які ми можемо виконувати над двома даними різних типів:

Номінальні дані: $=, \neq$. Все, що ми можемо зробити з номінальними даними, це порівняти два дані та перевірити, чи вони рівні.

Порядкові дані: =,≠,<,>. Окрім перевірки на рівність, ми також можемо порівнювати порядок різних точок даних.

Інтервальні дані: =,≠,<,>,−. Ми маємо всі операції з порядковими даними, але також можемо обчислювати точні числові різниці між двома точками даних. Залежно від контексту, ми можемо сказати, що додавання також дозволено, але іноді додавання фактично передбачає нульову точку і тому застосовується тільки до даних про співвідношення.

Дані співвідношення: =,≠,<,0,−,+,÷. Ми маємо всі операції з інтервальними даними, але додавання тепер практично завжди дозволено, і ми можемо додатково виконувати ділення (для визначення співвідношень) між різними точками даних.

Коротко:

Номінальні: категоріальні дані, без упорядкування Приклад – Домашні тварини: {собака, кіт, кролик, ...} Операції: =, ≠

Порядкові: категоріальні дані, з упорядкуванням Приклад – Рейтинг: {«поганий»,«нейтральний»,“хороший”,«дуже хороший»} Операції: =, ≠, ≥, ≤, >, <

Інтервал: числові дані, нуль не означає нульову «кількість» Приклад – температура за Фаренгейтом, Цельсієм Операції: = , ≠, ≥, ≤, >, <, +, −

Співвідношення: числові дані, нуль має значення, пов'язане з нульовою «кількістю» Приклад – температура за Кельвіном Операції: = , ≠, ≥, ≤, >, <, +, −,÷

Побудова графіків різних типів даних

Щоб бути більш формальними, далі ми припускаємо, що маємо доступ до набору даних, який можна позначити як

$$\{x^{(1)}, x^{(2)}, \ldots, x^{(m)}\}$$

де кожне $x^{(i)}$ є деякою точкою даних. $x^{(i)} = (a_1^i, a_2^i, a_3^i, ...)$

Для побудови всіх цих графіків ми будемо використовувати бібліотеку matplotlib, яка добре інтегрується з Jupyter notebook.

1D дані

Стовпчасті діаграми (Bar charts) — категоріальні дані

Якщо ваші дані є одновимірними категоріальними, єдиним реальним варіантом для їх візуалізації є використання стовпчастої діаграми. Зверніть увагу, що, виходячи з нашого припущення про те, що порядок точок даних у нашому наборі даних не має значення, єдиною значущою інформацією, якщо ми маємо набір даних типу

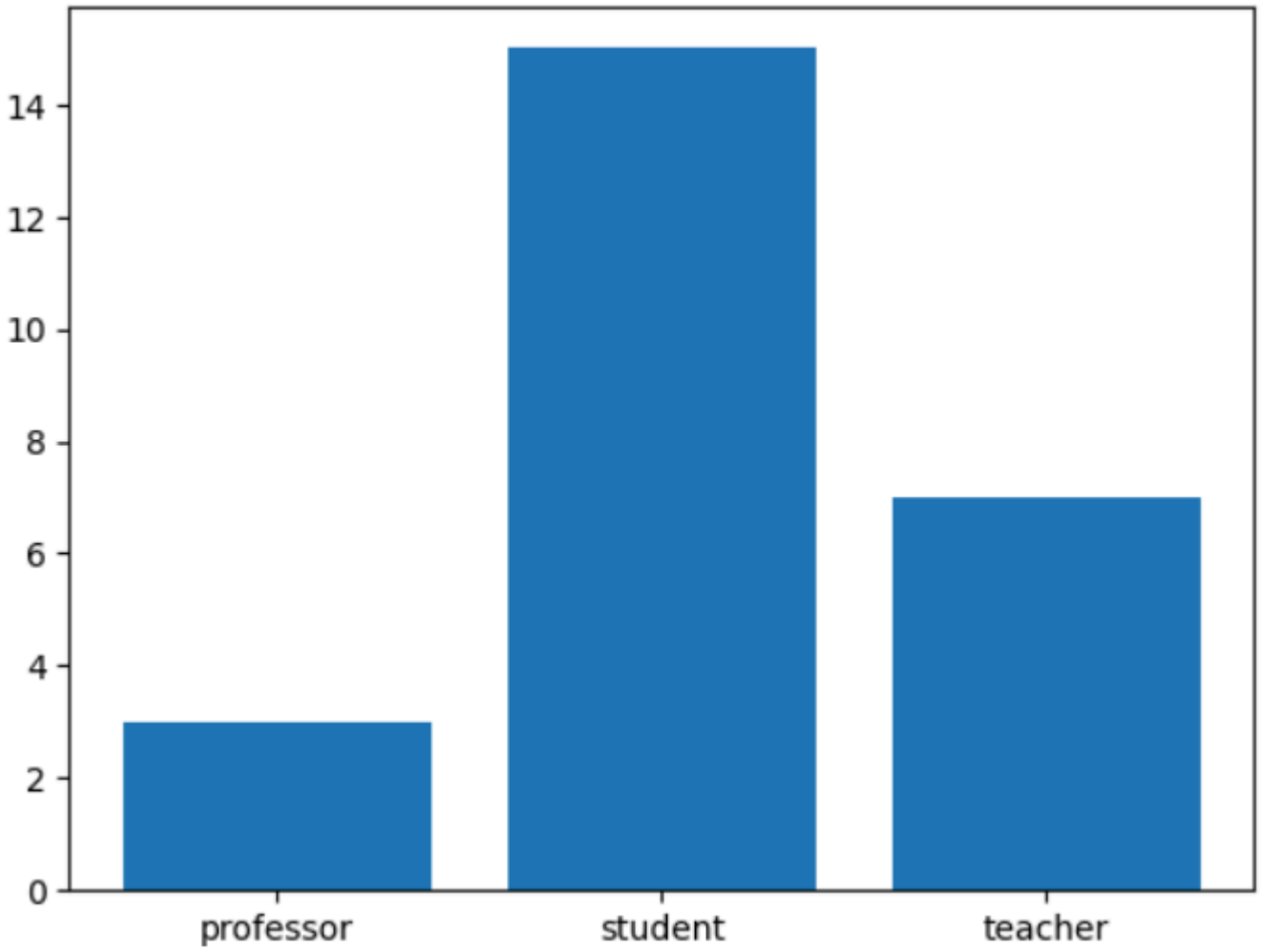
$$\{student, teacher, student, professor, teacher, student, \ldots\}$$

буде кількість повторень кожного елемента. Таким чином, ми можемо ефективно узагальнити дані, включивши кількість повторень кожного значення даних (це легко зробити за допомогою класу `collections.Counter`).

Примітка: незважаючи на те, що ви, можливо, бачили на деяких рисунках, зверніть увагу, що включення ліній між різними стовпчиками однозначно не має сенсу.

```
import collections
import matplotlib.pyplot as plt
import numpy as np

data = np.random.permutation(np.array(["student"]*15 + ["teacher"]*7 +
["professor"]*3))
# print(data)
counts = collections.Counter(data)
plt.bar(range(len(counts)), list(counts.values()), tick_label=list(counts.keys()))
```



Кругові діаграми (Pie charts) — просто скажіть «ні»

Кругові діаграми призначені для представлення даних, а не для їх дослідження!

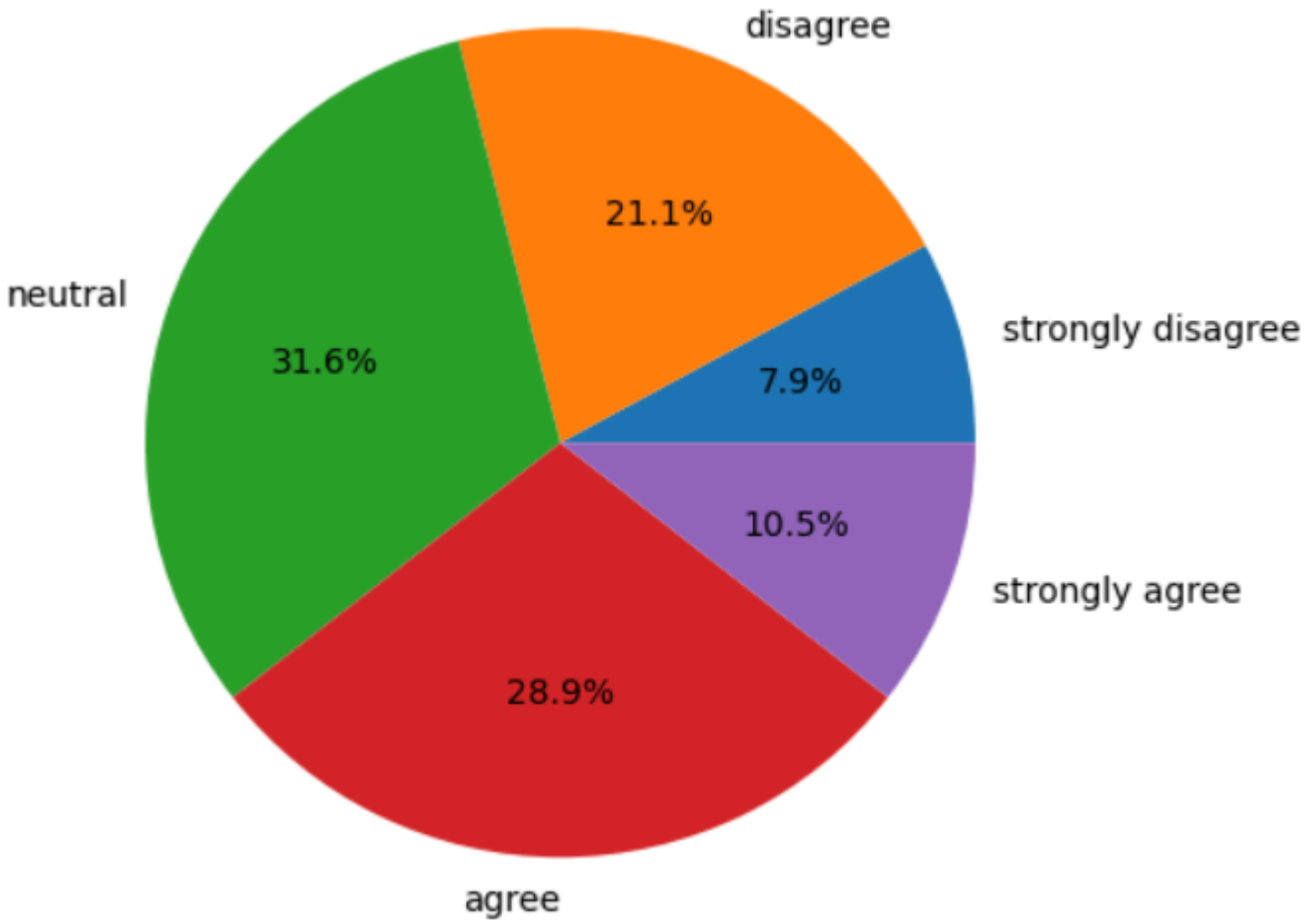
Люди не дуже добре оцінюють площу або кути (наприклад, важко визначити, чи один сегмент становить 23 %, а інший — 27 %). Стовпчасті діаграми набагато простіші, оскільки люди точніше порівнюють довжини.

Порівняти дві різні кругові діаграми (наприклад, два періоди часу) майже неможливо. Неможливо легко додати смуги похибки, опорні лінії або точні порівняння.

Ось приклад того, як побудувати кругову діаграму:

```
data = {"strongly disagree": 3,
        "disagree": 8,
        "neutral": 12,
        "agree": 11,
        "strongly agree": 4}

plt.pie(data.values(), labels=data.keys(), autopct='%1.1f%%')
plt.axis('equal')
```

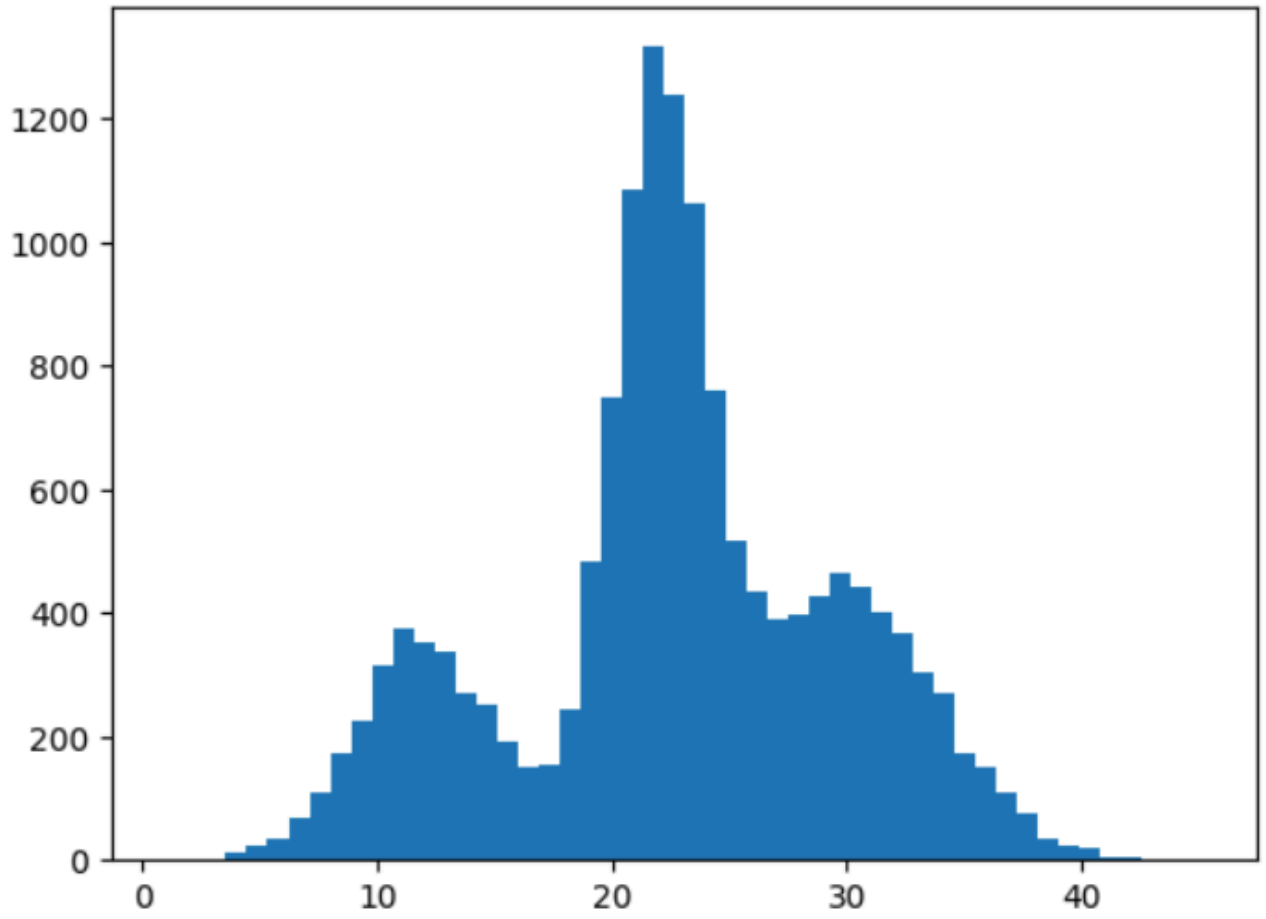


Гістограми (Histograms) — числові дані

Гістограма — це графік, який відображає розподіл числових даних, показуючи кількість спостережень у кожному інтервалі (біні).

```
np.random.seed(0)
data = np.concatenate([30 + 4*np.random.randn(5000),
                       22 + 2*np.random.randn(7000),
                       12 + 3*np.random.randn(3000)])
plt.hist(data, bins=50)
```

```
# data = np.concatenate([10 + 5*np.random.randn(10000)])
# plt.hist(data, bins=50);
```



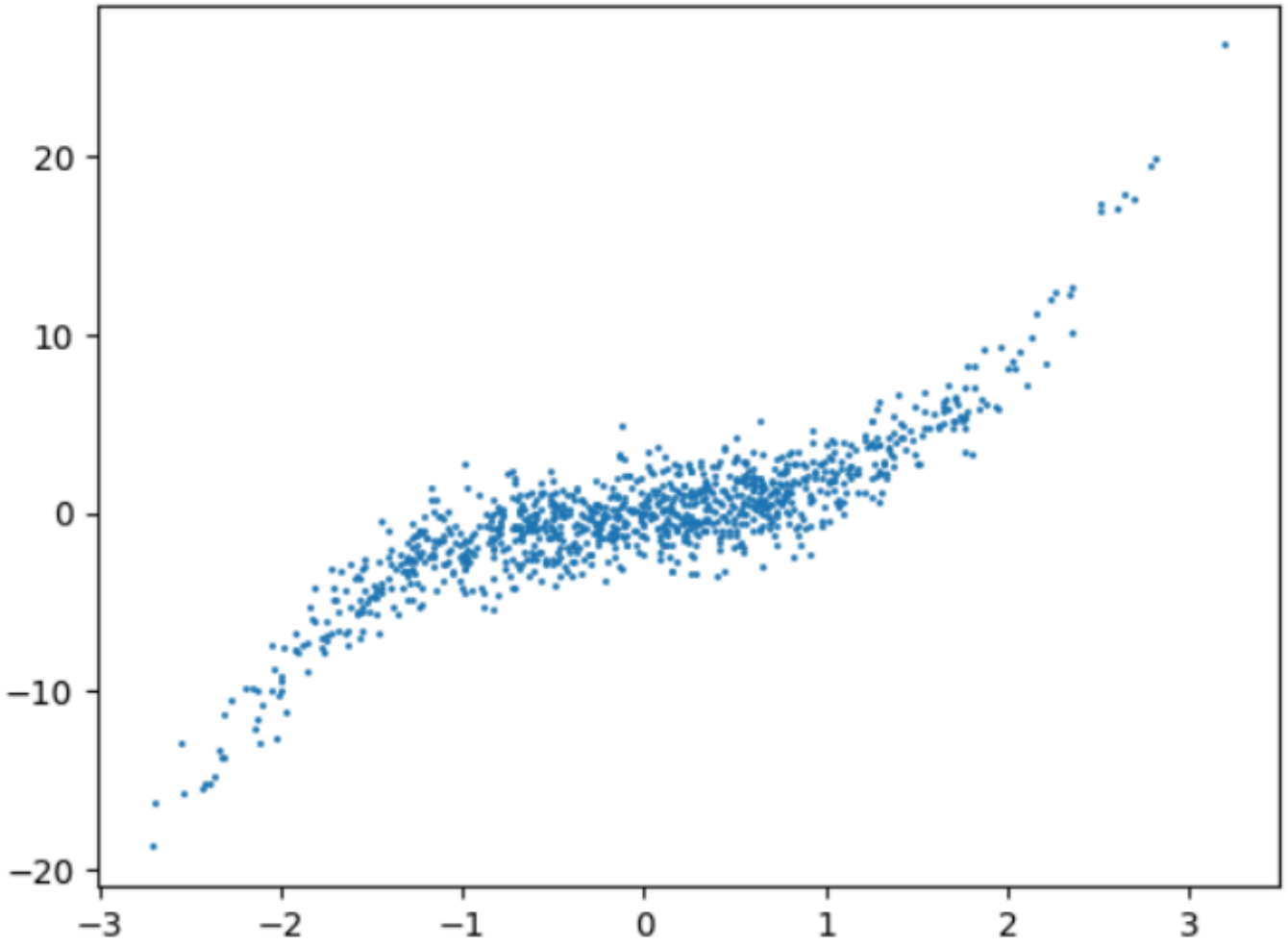
Гістограма показує розподіл числових даних, дозволяючи оцінити частоту появи значень у різних інтервалах. Бін (Bin) — це інтервал значень, у межах якого підраховується кількість спостережень. Занадто мала кількість інтервалів приховує деталі розподілу, тоді як занадто велика може зробити гістограму шумною та ускладнити її аналіз.

2D дані

Точкові діаграми — числові x числові (Scatter plots — numeric x numeric)

Якщо обидва виміри даних є числовими, найприроднішим першим типом діаграми, який слід розглянути, є точкова діаграма: нанесення точок, які просто відповідають різним координатам даних. (Діаграма розсіювання)

```
x = np.random.randn(1000)
y = 0.8*x**3 + x + 1.5*np.random.randn(1000)
plt.scatter(x,y,s=1)
```

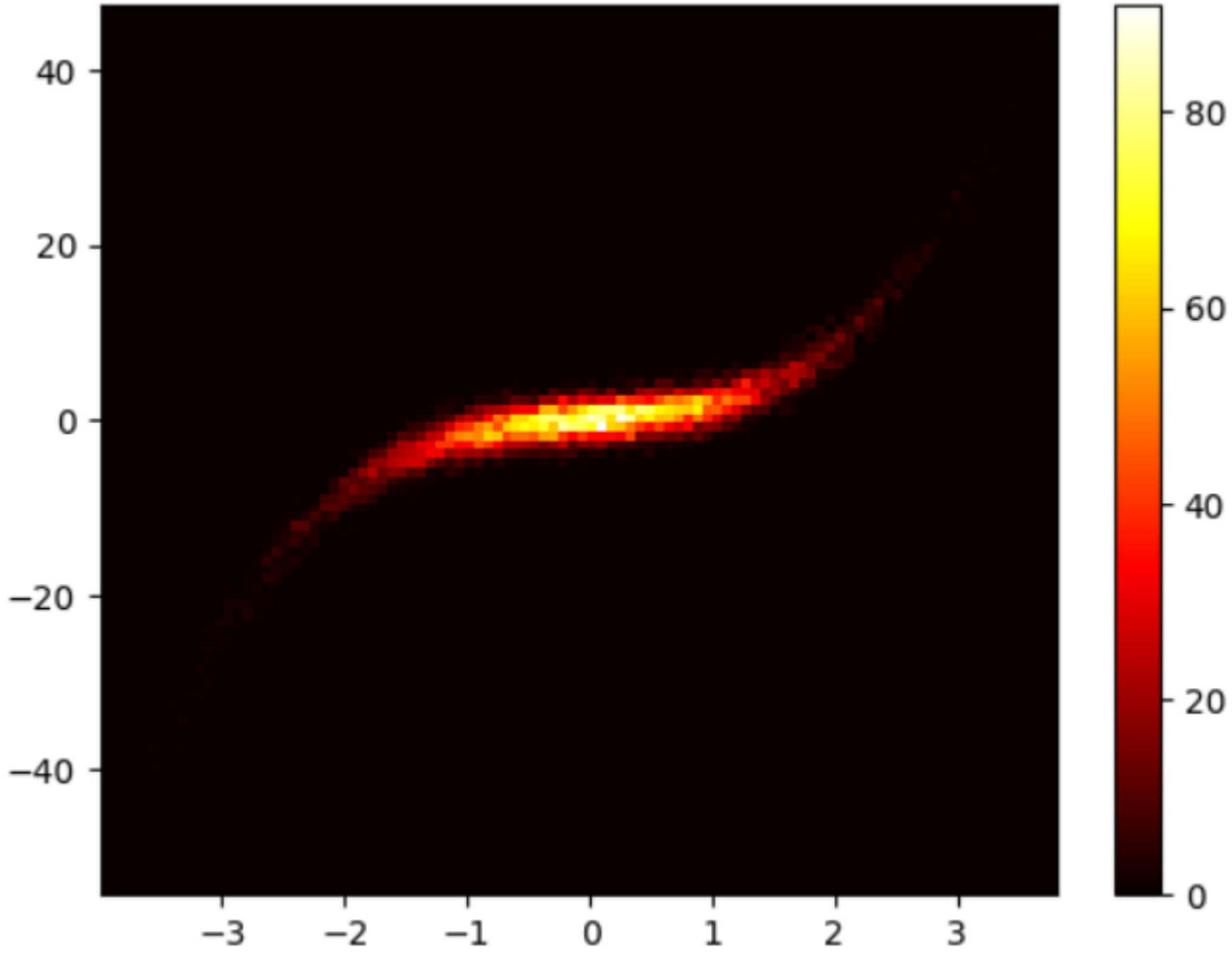


Однак існує також природний режим відмови, коли точок занадто багато, щоб їх можна було чітко розділити, і графік втрачає здатність показувати щільність даних. Наприклад, якщо у нас в 10 разів більше точок, графік вже не так чітко показує щільність у внутрішній частині «щільності».

У цьому випадку надлишку даних ми також можемо створити 2D гістограму даних (яка групує дані за обома вимірами) і вказати «висоту» кожного блоку за допомогою кольорової карти. Такі графіки можуть чіткіше вказувати щільність точок у регіонах, які в оригінальному графіку розсіювання мають суцільний колір. Ці 2D гістограми іноді також називають *тепловими картами* (heat maps), але ця назва часто конфліктує з подібними версіями, що використовуються для побудови 2D категоріальних даних, тому тут ми будемо використовувати термін *2D гістограма*.

```
x = np.random.randn(10000)
y = 0.8*x**3 + x + 1.5*np.random.randn(10000)
#plt.scatter(x,y,s=10)

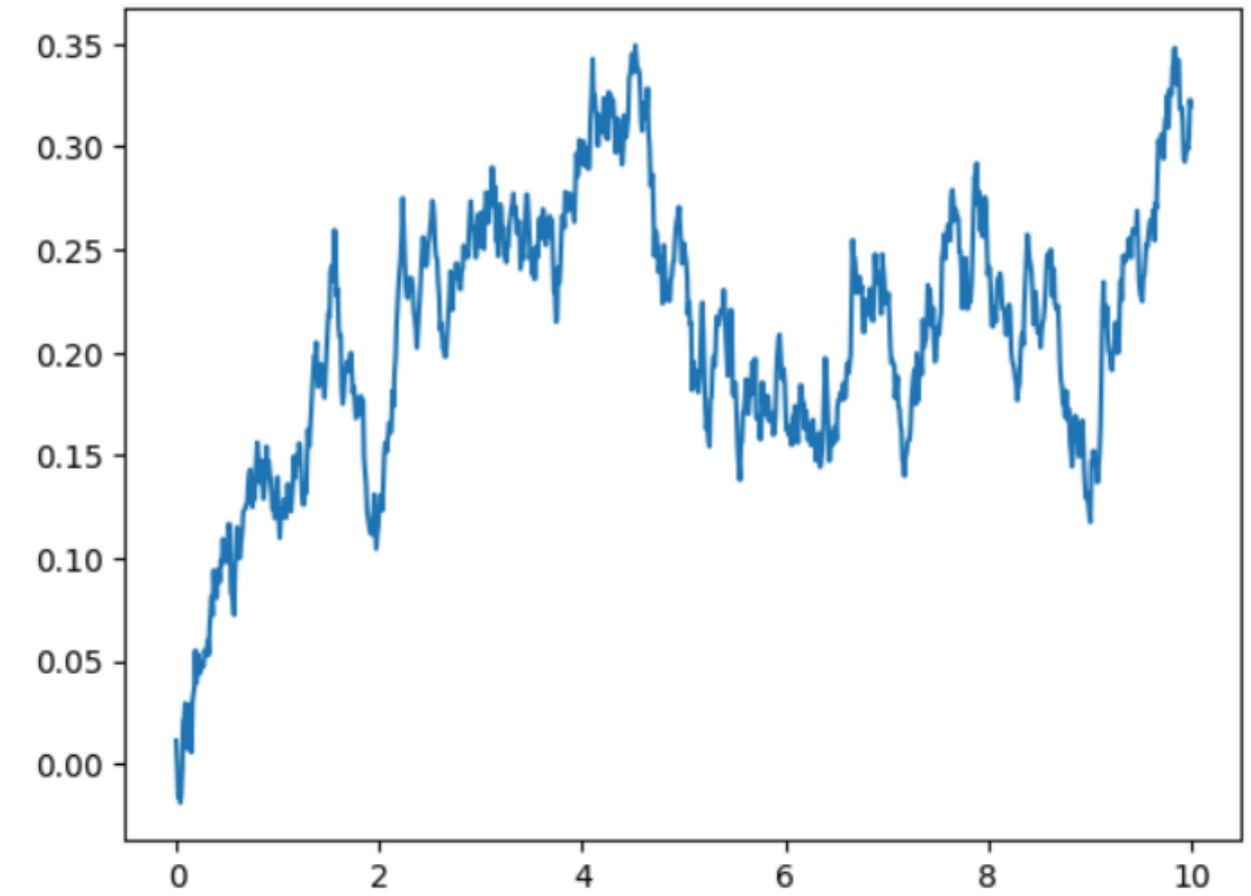
plt.hist2d(x,y,bins=100);
plt.colorbar();
plt.set_cmap('hot')
```



Лінійна діаграма (line plot) - numeric x numeric (sequential)

```
x = np.linspace(0,10,1000)
y = np.cumsum(0.01*np.random.randn(1000))
plt.plot(x,y)
```

```
[<matplotlib.lines.Line2D at 0x1957fbbcb50>]
```

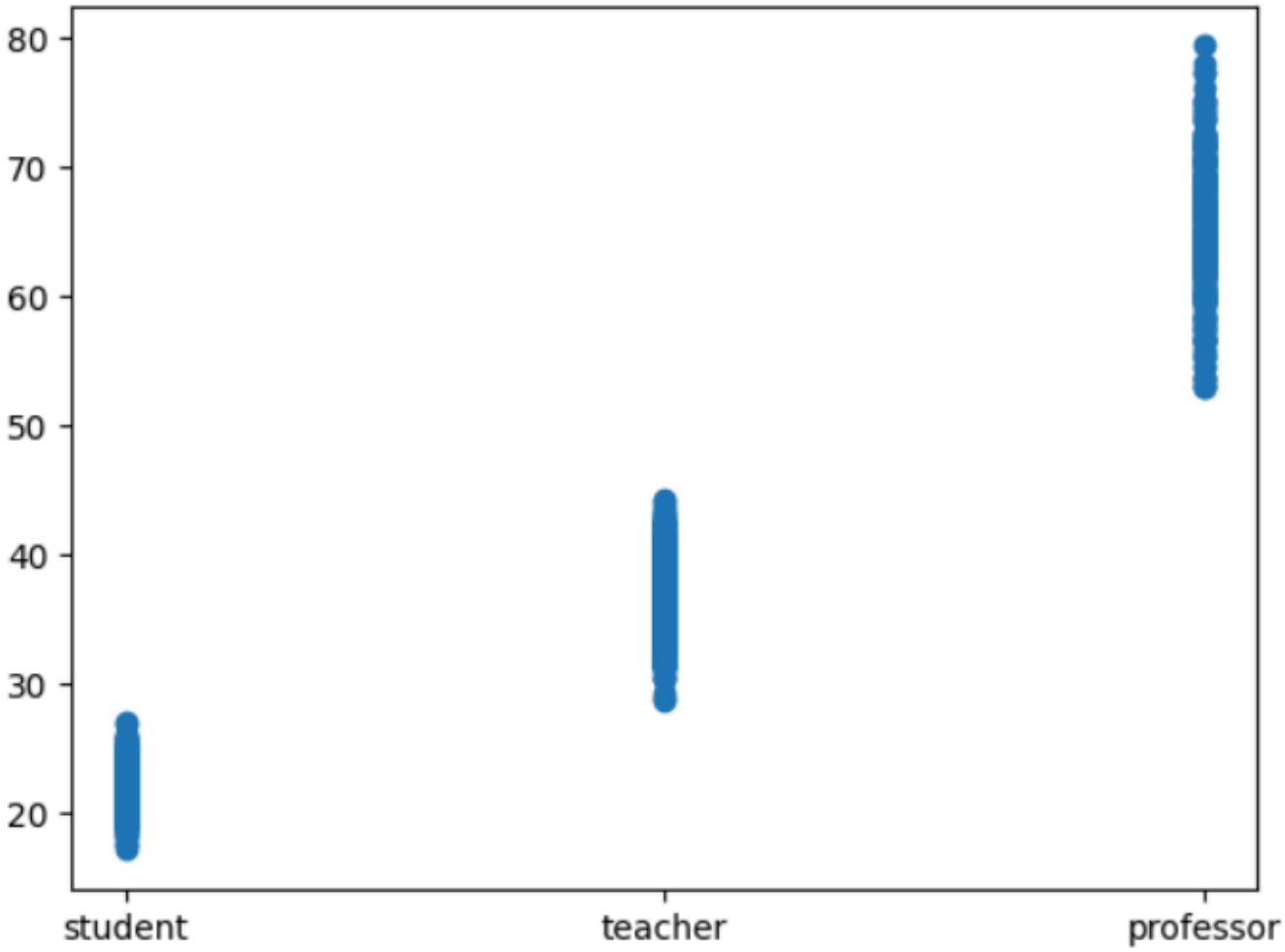


Коробкова (Box and whiskers) і скрипкові діаграми (Violin plots) — категоріальні x числові

Розглянемо простий приклад, де (тільки з вигаданими даними) ми будуємо графік учасників навчального процесу та їхній вік.

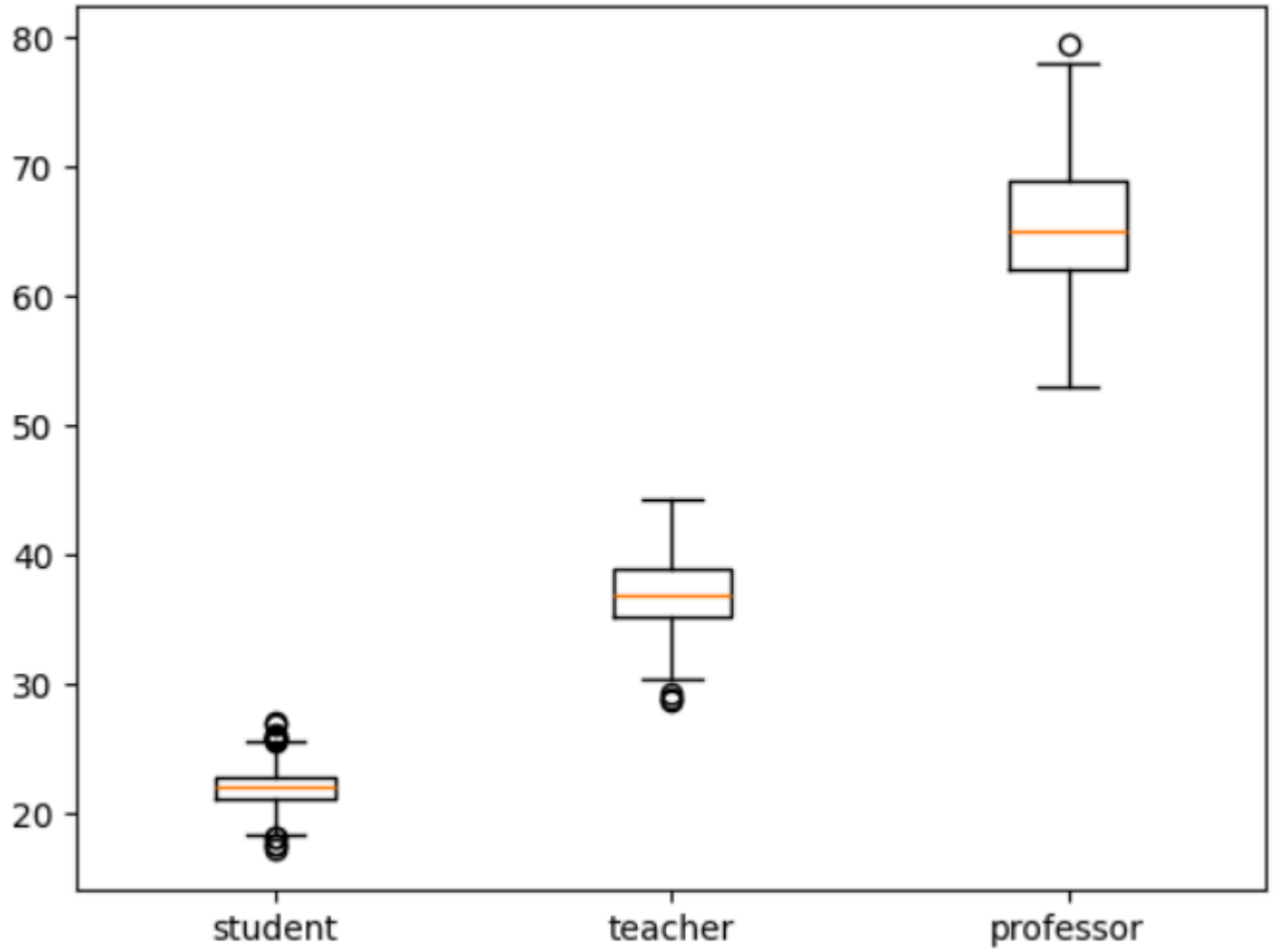
```
import numpy as np
import matplotlib.pyplot as plt
import scipy.linalg as la

data= {"student": 22 + 1.4*np.random.randn(1000),
       "teacher": 37 + 2.5*np.random.randn(900),
       "professor": 65 + 5.5*np.random.randn(300)}
plt.scatter(np.concatenate([i*np.ones(len(x)) for i,x in
enumerate(data.values())]),
            np.concatenate(list(data.values())))
plt.xticks(range(len(data)), data.keys());
```



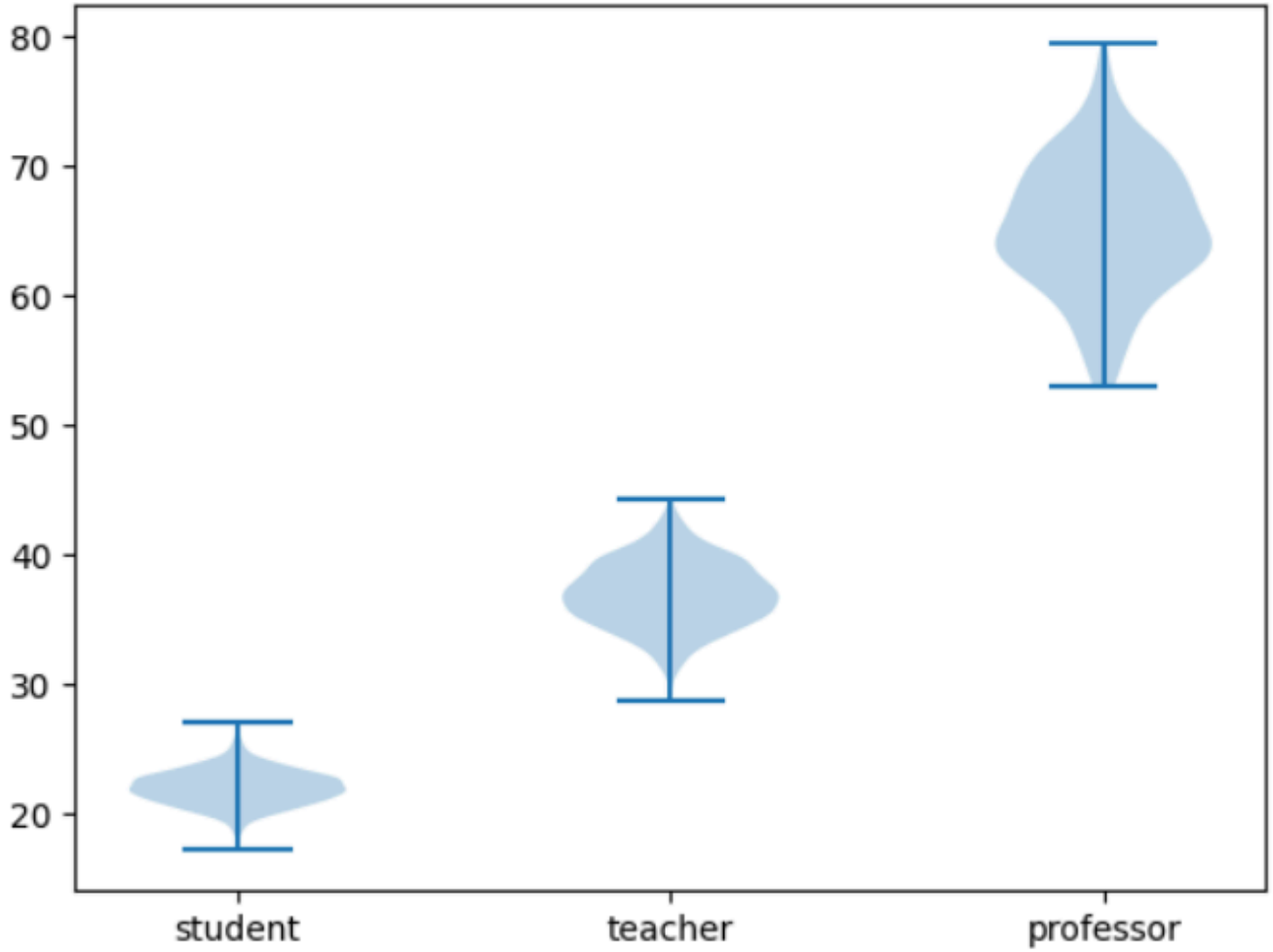
Очевидно, що, дивлячись тільки на цей графік, можна визначити дуже мало, оскільки в щільній лінії точок недостатньо інформації, щоб дійсно зрозуміти розподіл числової змінної для кожної точки. Поширеною стратегією тут є використання графіка коробкова, або коробка з вусами (box and whiskers), яка відображає медіану даних (як лінію в середині коробки), 25-й і 75-й процентилі даних (як нижня і верхня межі коробки), "вуса" встановлюються за допомогою ряду різних можливих угод (за замовчуванням Matplotlib використовує 1,5-кратний інтерквартильний розмах, відстань між 25-м і 75-м процентилем), а будь-які точки поза цим діапазоном ("випадкові значення", або "викиди") відображаються окремо.

```
plt.boxplot(data.values())
plt.xticks(range(1,len(data)+1), data.keys())
```



Звичайно, так само як середні значення та стандартні відхилення не описують повністю набір даних, статистика бокс-плат і вусів не відображає повністю розподіл даних. З цієї причини також часто використовуються скрипкові діаграми, які створюють міні-гістограми.

```
plt.violinplot(data.values())
plt.xticks(range(1,len(data)+1), data.keys())
```



Теплова карта (Heatmap) та бульбашкові діаграми (Bubble plots) — категоріальні x категоріальні

Теплова карта (Heatmap) — це спосіб візуалізації даних, у якому значення відображаються за допомогою кольорів, що дозволяє швидко оцінити їхню величину та закономірності. За допомогою теплових карт найчастіше візуалізують матриці кореляцій, статистичні таблиці, матриці відстаней, частоти спостережень та інші двовимірні числові дані. Теплова карта кореляцій використовується для наочного відображення сили та напряму кореляції між числовими ознаками, що допомагає виявити сильно пов'язані або незалежні змінні.

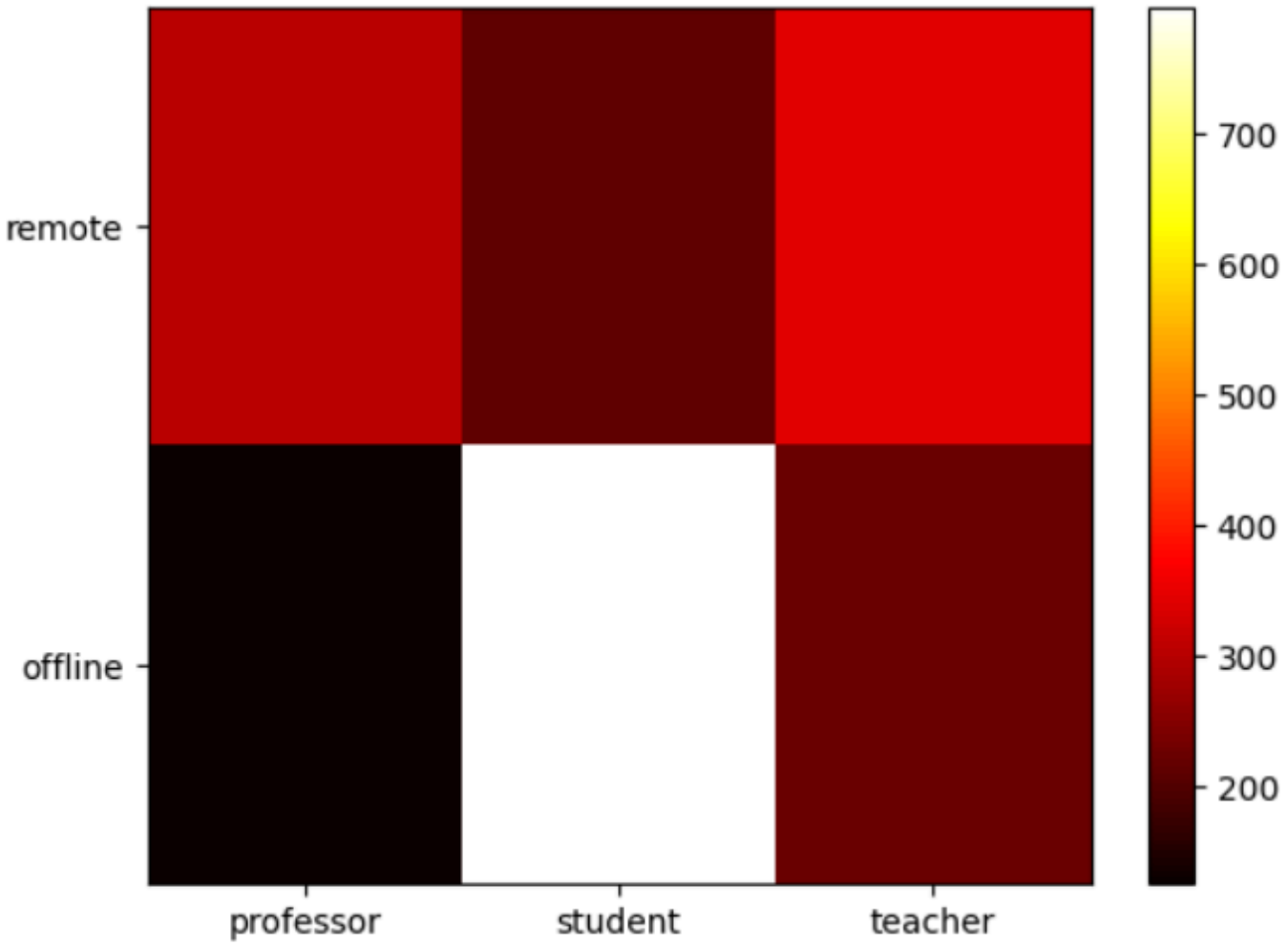
Коли обидва виміри наших 2D-даних є категоріальними, ми маємо ще менше інформації для використання. Знову ж таки, метою буде надання певної інформації про загальну кількість усіх можливих комбінацій між двома наборами даних. Наприклад, розглянемо вигаданий набір даних про учасників навчального процесу та їхній улюблений спосіб навчання.

```
types = np.array([('student', 'offline'), ('student', 'remote'),
                  ('teacher', 'offline'), ('teacher', 'remote'),
                  ('professor', 'offline'), ('professor', 'remote')])
data = types[np.random.choice(range(6), 2000, p=[0.4, 0.1, 0.12, 0.18, 0.05,
0.15]),:]

# print(data[100:])

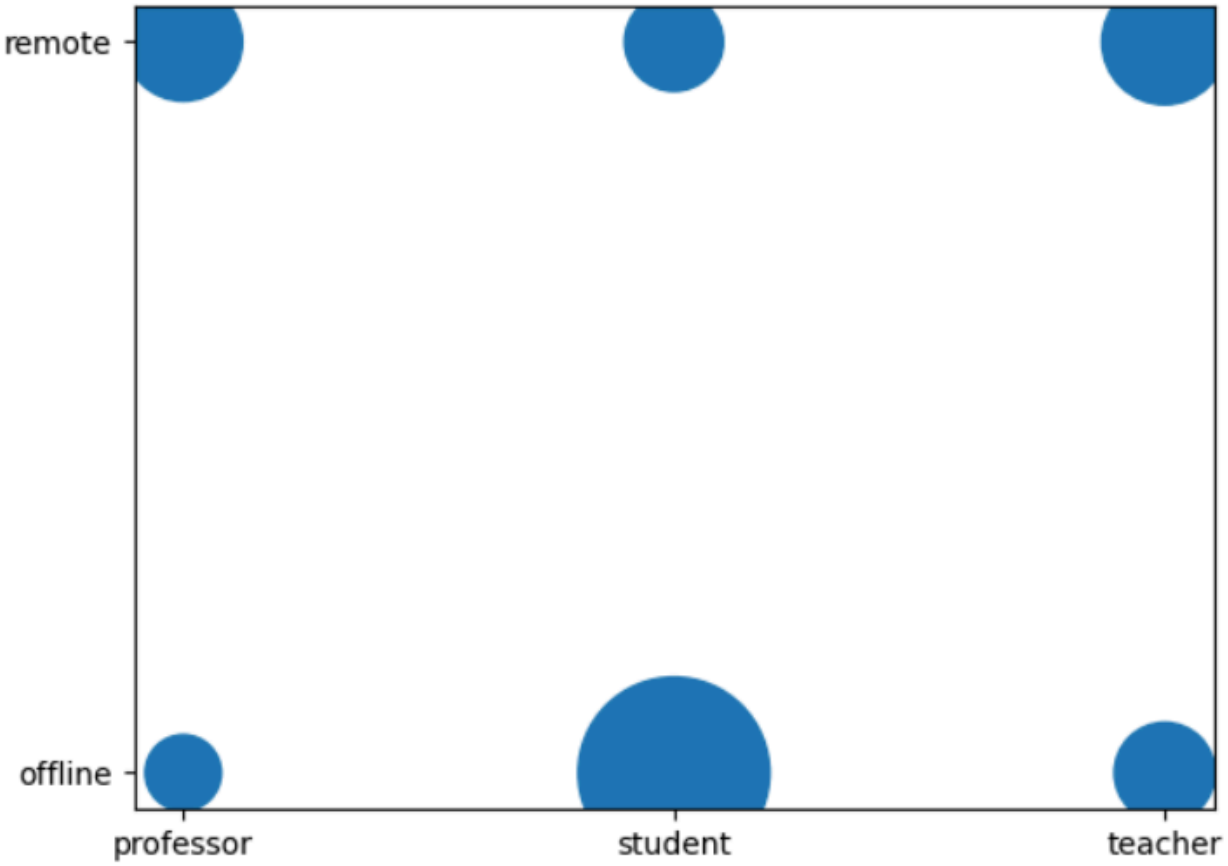
label_x, x = np.unique(data[:,0], return_inverse=True)
label_y, y = np.unique(data[:,1], return_inverse=True)
```

```
M, xt, yt, _ = plt.hist2d(x,y, bins=(len(label_x), len(label_y)))
plt.xticks((xt[:-1]+xt[1:])/2, label_x)
plt.yticks((yt[:-1]+yt[1:])/2, label_y)
plt.colorbar()
```



Хоча це може бути дещо корисним, діапазон кольорів, безперечно, не є дуже інформативним у деяких налаштуваннях, тому більш доречним може бути діаграма розсіювання з розмірами, пов'язаними з кожним типом даних (це також називається бульбашковою діаграмою).

```
xy, cnts = np.unique((x,y), axis=1, return_counts=True)
plt.scatter(xy[0], xy[1], s=cnts*5)
plt.xticks(range(len(label_x)), label_x)
plt.yticks(range(len(label_y)), label_y)
```



Бульбашкова діаграма додатково використовує розмір маркерів для відображення ще однієї числової змінної. Розмір бульбашок може відображати величину показника, наприклад населення, прибуток, обсяг продажів або іншу числову характеристику.

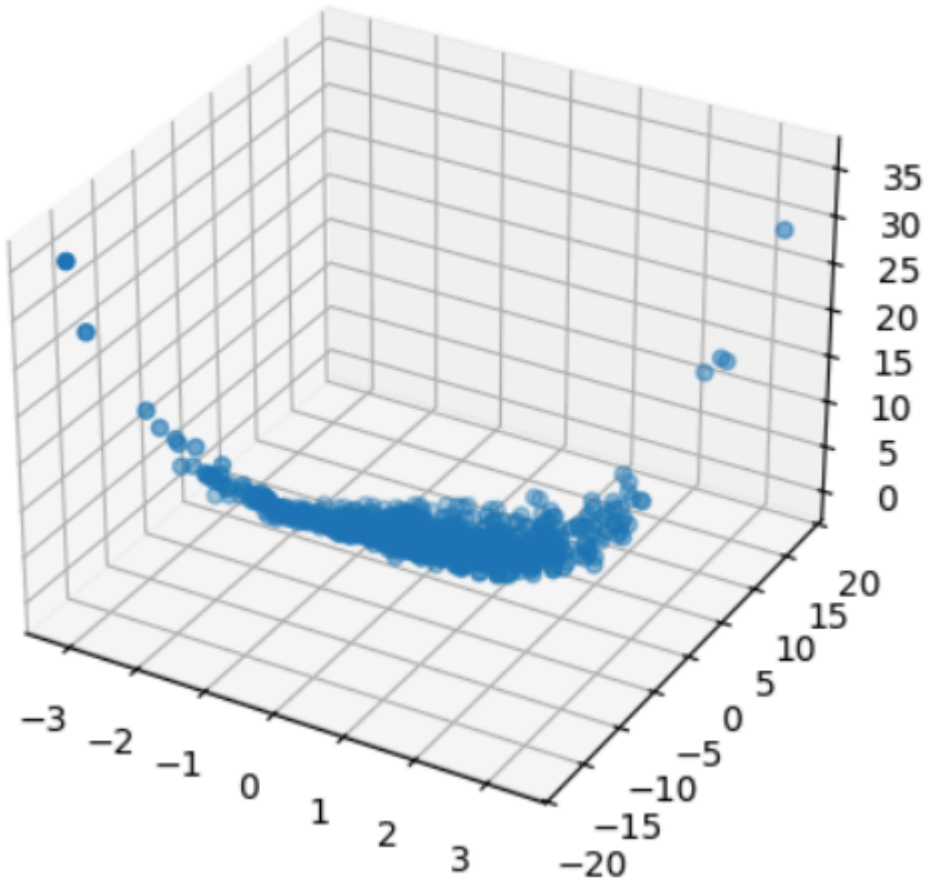
3D дані

Як тільки ми виходимо за межі двох вимірів, ефективна візуалізація стає набагато складнішою.

3D точкові графіки (3D scatter plots): слід уникати, якщо не можна взаємодіяти з 3D-графіком:

```
x = np.random.randn(1000)
y = 0.4*x**3 + x + 1.7*np.random.randn(1000)
z = 0.5 + 0.1*(y-1)**2 + 0.4*np.random.randn(1000)

from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(x,y,z)
```



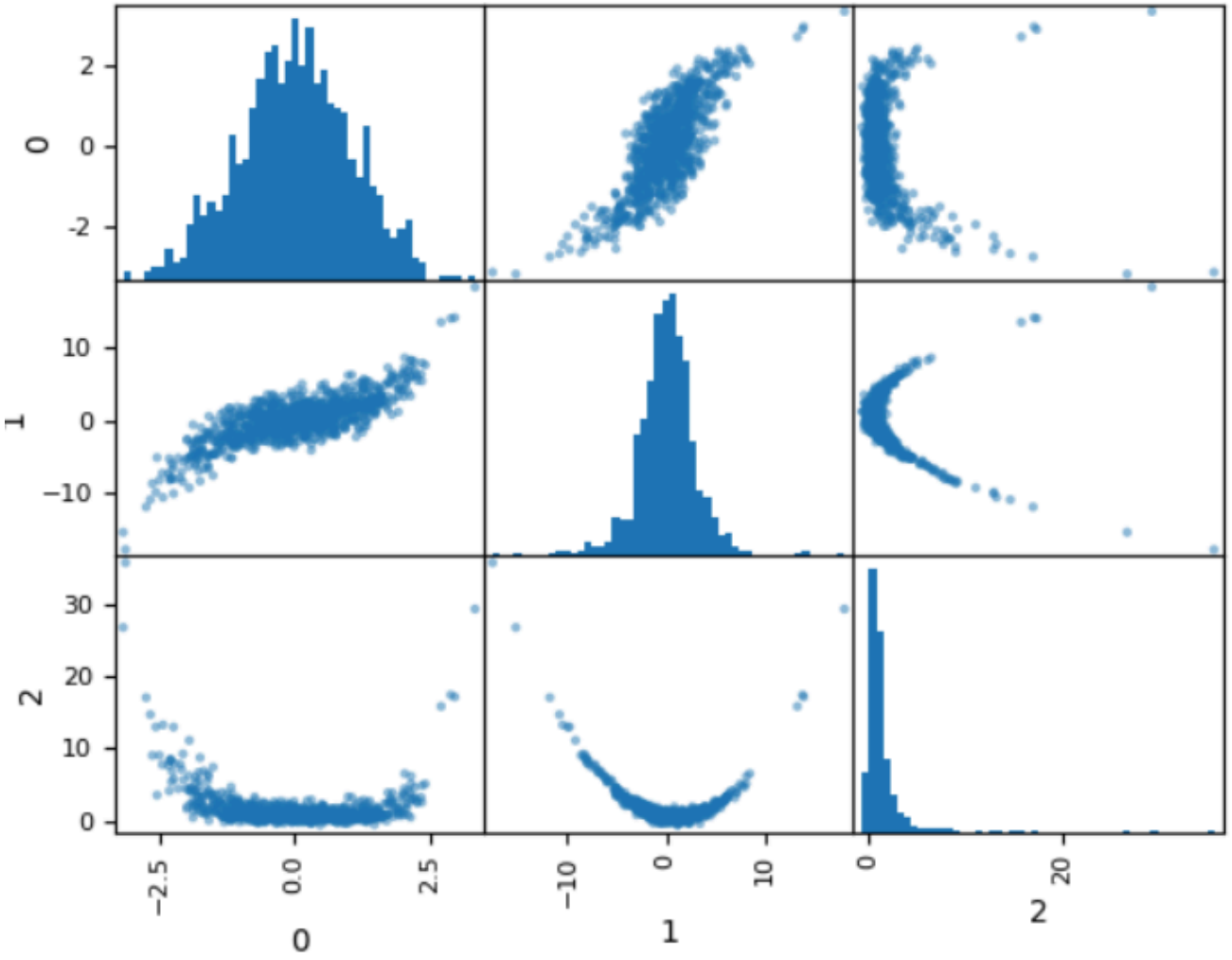
Матриці розсіювання (точкові матриці) (scatter matrices)

Універсальним засобом для візуалізації високорозмірних даних є матриця розсіювання.

Щодо матриць розсіювання слід зазначити один важливий момент: не намагайтеся використовувати їх для представлення даних.

```
import pandas as pd
df = pd.DataFrame([x,y,z]).transpose()
pd.plotting.scatter_matrix(df, hist_kwds={'bins':50})
```

```
array([[<Axes: xlabel='0', ylabel='0'>, <Axes: xlabel='1', ylabel='0'>,
        <Axes: xlabel='2', ylabel='0'>],
       [<Axes: xlabel='0', ylabel='1'>, <Axes: xlabel='1', ylabel='1'>,
        <Axes: xlabel='2', ylabel='1'>],
       [<Axes: xlabel='0', ylabel='2'>, <Axes: xlabel='1', ylabel='2'>,
        <Axes: xlabel='2', ylabel='2'>]], dtype=object)
```



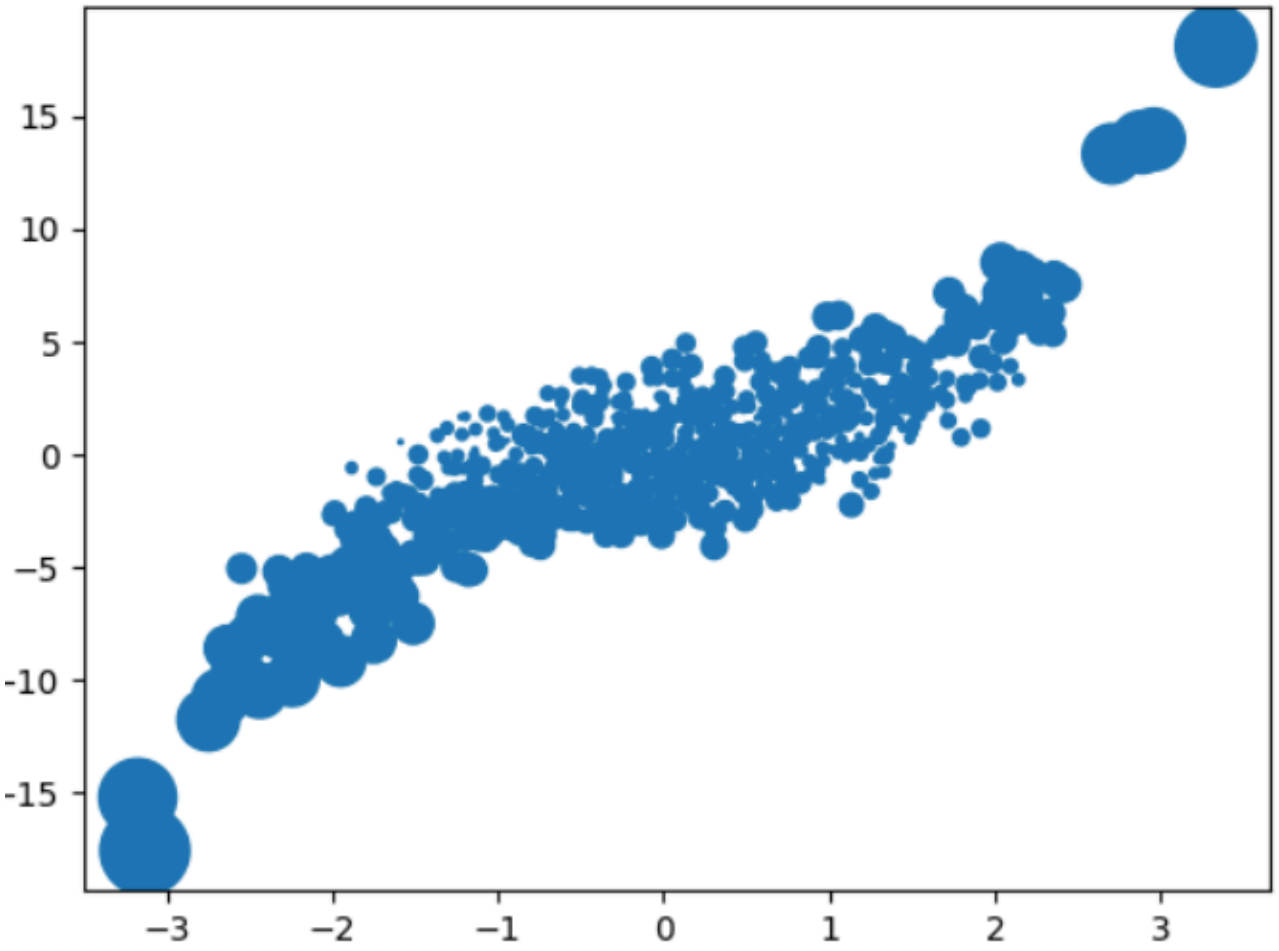
Матриця розсіювання використовується для одночасного аналізу взаємозв'язків між кількома числовими ознаками. За допомогою матриці розсіювання можна виявити кореляції, кластери, викиди (outliers) та можливі нелінійні залежності між ознаками.

Бульбашкові діаграми (Bubble plots)

Бульбашкова діаграма з попереднього розділу також може бути одним із засобів розуміння тривимірних даних. У цій діаграмі та її варіантах ми побудуємо двовимірну точкову діаграму, але додамо третій вимір за допомогою іншого атрибуту, наприклад розміру точок. Для наведених вище даних це буде виглядати наступним чином.

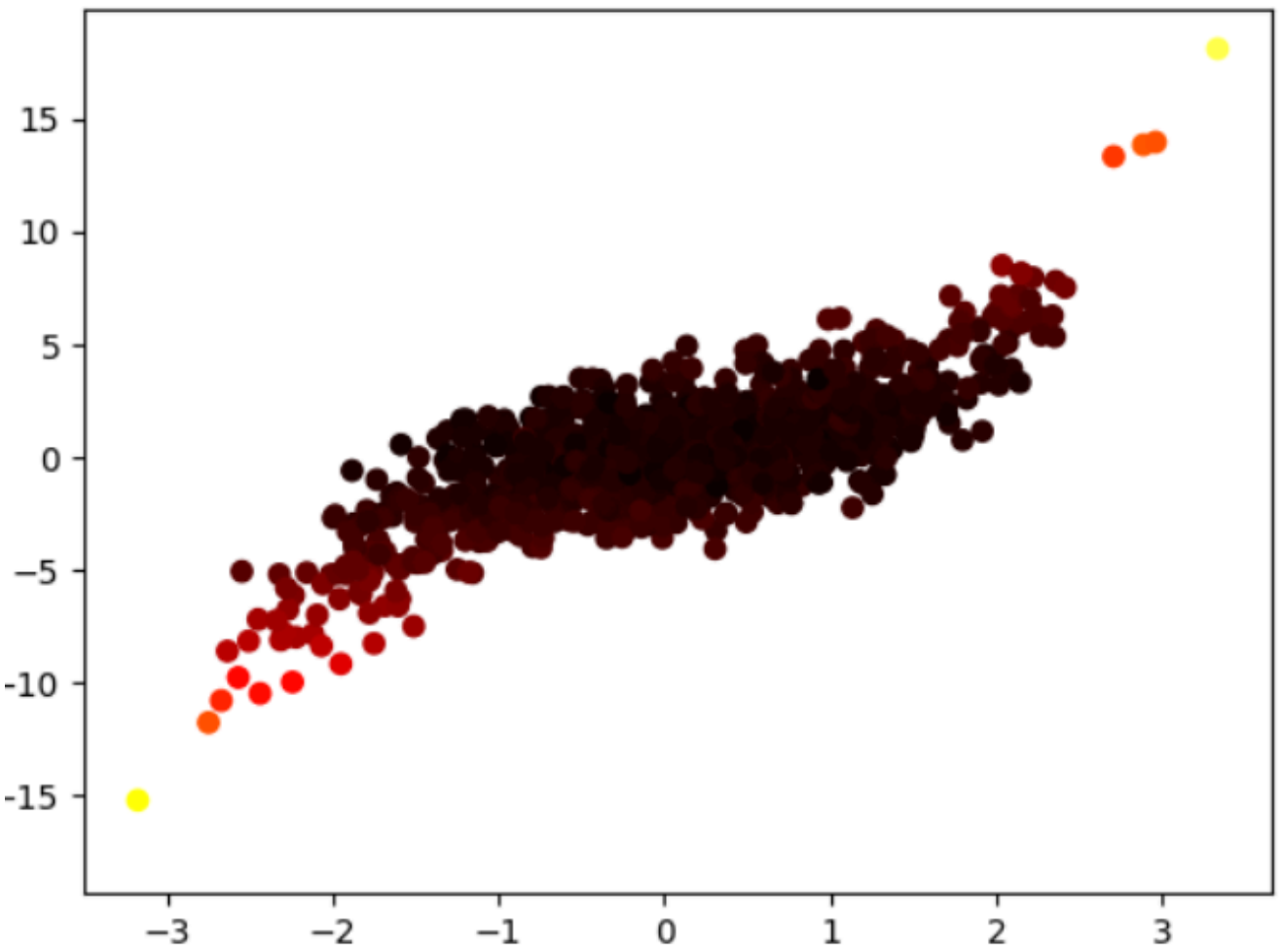
```
plt.scatter(x,y,s=z*20)
```

```
<matplotlib.collections.PathCollection at 0x2bfbdcfcd10>
```




```
plt.scatter(x,y,c=z)
```

```
<matplotlib.collections.PathCollection at 0x2bfbb610650>
```



Тип діаграми обирають залежно від типу даних, кількості змінних, мети аналізу та закономірностей, які необхідно виявити. Правильно обрана візуалізація робить дані зрозумілішими, дозволяє швидко знаходити закономірності, виявляти аномалії та оцінювати взаємозв'язки між змінними.

Питання для самоконтролю.

1. Що таке візуалізація даних і яке її значення в інтелектуальному аналізі даних?
2. Які основні цілі візуалізації даних?
3. Чому візуалізація є важливим етапом дослідницького аналізу даних (EDA)?
4. Які переваги має графічне представлення даних порівняно з табличним?
5. Які бібліотеки Python найчастіше використовуються для побудови графіків?
6. Які типи даних виділяють за шкалою вимірювання?
7. Що таке номінальні дані?
8. Наведіть приклади номінальних даних.
9. Що таке порядкові дані?
10. Наведіть приклади порядкових даних.
11. Що таке інтервальні дані?
12. Чим інтервальні дані відрізняються від даних співвідношення?
13. Що таке дані співвідношення (Ratio Data)?
14. Який тип діаграми найкраще підходить для номінальних даних?
15. Для яких типів даних доцільно використовувати гістограми?
16. Що таке стовпчаста діаграма (Bar Chart)?

17. Для яких задач використовується стовпчаста діаграма?
18. Чим стовпчаста діаграма відрізняється від гістограми?
19. Що таке кругова діаграма (Pie Chart)?
20. У яких випадках доцільно використовувати кругову діаграму?
21. Які недоліки має кругова діаграма?
22. Що таке гістограма?
23. Яку інформацію дозволяє отримати гістограма?
24. Що таке бін (Bin) у гістограмі?
25. Як вибір кількості інтервалів впливає на вигляд гістограми?
26. Що таке точкова діаграма (Scatter Plot)?
27. Для яких задач використовують точкові діаграми?
28. Як за допомогою точкової діаграми можна виявити кореляцію між змінними?
29. Що таке лінійна діаграма (Line Chart)?
30. Для яких типів даних найчастіше використовують лінійні діаграми?
31. Що таке коробкова діаграма (Box Plot)?
32. Які статистичні характеристики відображає коробкова діаграма?
33. Як за допомогою Box Plot можна виявити викиди?
34. Що таке скрипкова діаграма (Violin Plot)?
35. Чим скрипкова діаграма відрізняється від коробкової?
36. Що таке теплова карта (Heatmap)?
37. Які дані найчастіше візуалізують за допомогою теплової карти?
38. Для чого використовується теплова карта кореляцій?
39. Що таке бульбашкова діаграма (Bubble Chart)?
40. Чим бульбашкова діаграма відрізняється від звичайної точкової діаграми?
41. Яку інформацію може відображати розмір бульбашок?
42. Що таке матриця розсіювання (Scatter Matrix)?
43. Для чого використовується матриця розсіювання під час аналізу даних?
44. Які закономірності можна виявити за допомогою матриці розсіювання?
45. Що таке тривимірна (3D) візуалізація даних?
46. Які типи 3D-графіків підтримуються сучасними бібліотеками Python?
47. Які переваги має використання 3D-візуалізації?
48. Які недоліки та обмеження має 3D-візуалізація?
49. За якими критеріями слід обирати тип діаграми для конкретного набору даних?
50. Яким чином правильний вибір способу візуалізації допомагає виявляти закономірності, аномалії та взаємозв'язки у даних?

4. Лінійна алгебра та бібліотека NumPy

Вектори та матриці відіграють центральну роль в сучасному інтелектуальному аналізі даних. Матриці, це очевидний спосіб зберігання табличних даних. Сьогодні основа лінійної алгебри, є мовою всіх алгоритмів аналізу даних (машинне навчання, штучний інтелект та ін.).

Вектор - 1D масив даних

$$\mathbf{v} = \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix}$$

Матриця - 2D масив даних

$$\mathbf{A} = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1m} \\ x_{21} & x_{22} & \dots & x_{2m} \\ \dots & \dots & \dots & \dots \\ x_{n1} & x_{n2} & \dots & x_{nm} \end{bmatrix}$$

Існують також узагальнення матриць вищого порядку (так звані *тензори*), які представляють тривимірні або вищі масиви значень. Вони досить широко використовуються в сучасній науці про дані, хоча зазвичай (але, звичайно, не завжди) тензори використовуються лише в значенні «багатовимірного масиву», а не в їхньому справжньому лінійно-алгебраїчному значенні. Тензори як лінійні оператори, що діють, наприклад, на матриці або інші тензори вищого порядку, дещо рідше використовуються в базовій науці про дані.

Порядок рядків і стовпців

Матриці можуть бути розміщені в пам'яті за рядками або за стовпцями.

$$\mathbf{A} = \begin{bmatrix} 100 & 80 \\ 60 & 80 \\ 100 & 100 \end{bmatrix}$$

За рядками: 100, 80, 60, 80, 100, 100. За стовпцями: 100, 60, 100, 80, 80, 100. По замовчуванню в NumPy порядок за рядками

Основні операції

Додавання: Дві матриці A та B, $[n \times m]$, $A + B = C$ визначається: $C_{ij} = A_{ij} + B_{ij}$

Транспонування: Для матриці A, $[n \times m]$, транспонована матриця $C = A^T$ визначається: $C_{ij} = A_{ji}$

Множення: Для матриць A ($[n \times m]$) та B ($[m \times p]$) добуток $C = AB$ ($[n \times p]$) визначається: $C_{ij} = \sum_{k=1}^{m} A_{ik} B_{kj}$

Множення матриць є асоціативним ($(AB)C = A(BC)$), дистрибутивним $A(B + C) = AB + AC$, але не комутативним ($AB \neq BA$).

Одинична матриця: Одинична матриця I це квадратна матриця з одиницями на головній діагоналі та нулями в інших позиціях:

$$\mathbf{I} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & 1 \end{bmatrix}$$

Для будь-якої матриці A: $AI = IA = A$

Обернена матриця: Для квадратної матриці A, обернена матриця A^{-1} це така матриця, що

$A^{-1}A = AA^{-1} = I$

Не для кожної квадратної матриці існує обернена.

Деякі рівняння: $(AB)^T = B^T A^T$, $(AB)^{-1} = B^{-1} A^{-1}$

Скалярний добуток векторів: $x * y = x^T y = \sum_{i=1}^{n} x_i y_i$;

Норма вектора: $||x||_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$

Масиви Numpy

Створення масиву за допомогою команди `numpy.array` повертає тип `ndarray`. Ви також можете створювати масиви нулів, одиниць або випадкових чисел (у цьому випадку `np.randon.randn` створює матрицю зі стандартними випадковими нормальними елементами, а `np.random.rand` створює рівномірні випадкові елементи).

```
import numpy as np

b = np.array([-13,9])
A = np.array([[4,-5], [-2,3]])
print(b, "\n")
print(A, "\n")

print(np.ones(40), "\n")           # 1D масив одиниць
print(np.zeros(4), "\n")          # 1D масив нулів
print(np.random.rand(40), "\n")         # 1D масив сипадкових чисел нормального
розподілу

print(np.ones((3,4)), "\n")       # 2D масив одиниць
print(np.zeros((3,4)), "\n")      # 2D масив нулів
print(np.random.randn(3,4))       # 2D масив випадкових чисел нормального
розподілу

sigma, mu = 10, 50
print(sigma * np.random.randn(10,10) + mu)  # 2D масив випадкових чисел
нормального розподілу із середнім значенням mu та дисперсією sigma
```

Ви також можете створити матрицю ідентичності за допомогою команди `np.eye()`, а діагональну матрицю — за допомогою команди `np.diag()`.

```
print(np.eye(3),"\n")                    # створити масив для одиничної матриці
3x3
print(np.diag(np.random.randn(3)),"\n")   # створити діагональний масив випадкових
чисел нормального розподілу
```

Вивід:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

[[ 0.50651677  0.          0.        ]
```

```
 [ 0.         -0.35063699  0.        ]
 [ 0.          0.         -0.72949183]]
```

Індексація в масивах numpy

```
A = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
print(A, "\n")
print(A[1,1],"\n")           # вибрати один запис
print(A[1,:],"\n")           # вибрати ряд
print(A[1:3, :], "\n")       # slice
```

Вивід:

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]

5

[4 5 6]

[[4 5 6]
 [7 8 9]]
```

```
print(A[1:2,1:2], "\n")  # Вибрати A[1,1] як одиничний 2D масив
print(A[[1,2,3],:], "\n")  # вибрати рядки 1, 2, та 3
print(A[[2,1,2],:], "\n")  # вибрати рядки 2, 1, та 2 знову
print(A[[False, True, False, True],:], "\n")  # Вибрати 1-й та 3-й рядки
```

Вивід:

```
[[5]]

[[ 4  5  6]
 [ 7  8  9]
 [10 11 12]]

[[7 8 9]
 [4 5 6]
 [7 8 9]]
```

```
[[ 4  5  6]
 [10 11 12]]
```

Основні операції над масивами: масиви можна додавати/віднімати, множити/ділити та транспонувати, але ці операції **не є ідентичними** до відповідних операцій у лінійній алгебрі. Множення та ділення масивів виконуються **поелементно**, вони **не** є множенням матриць або чимось, пов'язаним з оберненням матриць.

```
A = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
B = np.array([[1, 1, 1], [1,2,1], [3, 1, 3], [1, 4, 1]])

print(A+B, "\n") # додати A та B поелементно
print(A-B, "\n") # відняти B від A поелементно (same as "standard" matrix
subtraction)
print(A*B, "\n") # поелементне множення, не(!) множення матриць
print(A/B, "\n") # поелементне ділення
```

Вивід:

```
[[ 2  3  4]
 [ 5  7  7]
 [10  9 12]
 [11 15 13]]

[[ 0  1  2]
 [ 3  3  5]
 [ 4  7  6]
 [ 9  7 11]]

[[ 1  2  3]
 [ 4 10  6]
 [21  8 27]
 [10 44 12]]

[[ 1.          2.          3.        ]
 [ 4.          2.5         6.        ]
 [ 2.33333333  8.          3.        ]
 [10.          2.75       12.        ]]
```

Ви можете транспонувати масиви, але зверніть увагу, що це має значення тільки для 2D (або вищих) масивів. Транспонування 1D масиву нічого не дає, оскільки Numpy не розрізняє стовпцеві вектори та рядкові вектори для 1D масивів.

```
print(A, "\n")
print(A.T, "\n")
```

Вивід:

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]

[[ 1  4  7 10]
 [ 2  5  8 11]
 [ 3  6  9 12]]
```

Трансляція Numpy (broadcasting). Справжня цікавість починається, коли ви додаєте/віднімаєте/множите/ділите масиви різних розмірів. Замість того, щоб видавати помилку, Numpy спробує зрозуміти вашу операцію, використовуючи правила трансляції Numpy. Це складна тема, яка часто бентежить новачків у Numpy, але з невеликою практикою правила стають досить інтуїтивними.

```
A = np.ones((4,3))          # A - 4x3
x = np.array([[1,2,3]])      # x - 1x3

print(A, '\n')
print(x, '\n')

print(A*x) # повторити x 4 рази, та помножити A
```

Вивід:

```
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]

[[1 2 3]]

[[1. 2. 3.]
 [1. 2. 3.]
 [1. 2. 3.]
 [1. 2. 3.]]
```

Операції лінійної алгебри. Починаючи з Python 3, між масивами numpy тепер існує оператор множення матриць `@` (раніше для цього доводилося використовувати більш громіздку функцію `np.dot()`).

```
A = np.random.randn(5,4)
C = np.random.randn(4,3)
x = np.random.randn(4)
```

```
y = np.random.randn(5)
z = np.random.randn(4)

print(A @ C, "\n")       # множення матриці на матрицю (вертає 2D масив)
print(A @ x, "\n")       # множення матриці на вектор (вертає 1D масив)
print(x @ z)       # внутрішній добуток (скалярний) - зверніть увагу на тип даних,
що вертаються - це скаляр, а не масив!
```

Вивід:

```
[[-0.77983164  0.86915701  0.74016041]
 [ 0.9441811  -1.13337787 -2.45126146]
 [ 1.06065113 -3.17950771 -3.50084458]
 [ 1.26732128 -1.3041138  -2.5499486 ]
 [ 1.60891994 -4.68541868 -4.6433185 ]]

[ 1.53587182 -2.9122003  -2.53401694 -2.88846645 -3.50775768]

2.236322572387176
```

Зверніть увагу! За замовчуванням оператор @ застосовується зліва направо, що може призвести до дуже неефективних порядків множення матриць.

```
A = np.random.randn(1000,1000)
B = np.random.randn(1000,2000)
x = np.random.randn(2000)

%timeit A @ B @ x
```

Вивід:

```
59.1 ms ± 7.45 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

Це виконує матричні добутки $(AB)x$, які спочатку обчислюють неефективне множення матриць. Якщо ми хочемо обчислити добуток у набагато ефективнішому (в даному прикладі) порядку $A(Bx)$, ми використаємо команду

```
%timeit A @ (B @ x)
```

```
2.13 ms ± 226 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

Нарешті, Numpy включає процедуру `np.linalg.inv()` для обчислення оберненої матриці A^{-1} та `np.linalg.solve()` для обчислення матричного рівняння $A^{-1}b$.

```
b = np.array([-13,9])
A = np.array([[4,-5], [-2,3]])

print(np.linalg.inv(A), "\n")   # обернена матриця
print(np.linalg.solve(A,b))     # обчислити A^{-1}b
```

Вивід:

```
[[1.5 2.5]
 [1.  2. ]]

[3. 5.]
```

Складність операцій

Припустимо, що A, B [$n \times n$], x, y - вектори [n].

Добуток матриць AB: $O(n^3)$

Добуток матриці та вектора Ax: $O(n^2)$

Внутрішній добуток векторів $x^T y$: $O(n)$

Обернена матриця/розв'язання: A^{-1}: $O(n^3)$

Розріджені матриці

Багато матриць є розрідженими - такими, що містять переважно нульові елементи, з лише декількома ненульовими елементами.

Приклади: матриці, утворені реальними графами, матриці підрахунку слів у документах (більше про це пізніше)

Зберігання всіх цих нулів у стандартному форматі матриці може бути великою тратою обчислювальних ресурсів і пам'яті.

Бібліотеки розріджених матриць надають ефективні засоби для обробки цих розріджених матриць, зберігаючи і оперуючи тільки ненульовими елементами

Існує кілька різних способів зберігання розріджених матриць, кожен з яких оптимізований для різних операцій

Формат координат (COO): зберігання кожного елемента як кортежу (row_index, col_index, value)

Наприклад, замість матриці:

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 3 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 4 & 0 & 1 & 0 \end{bmatrix}$$

Матимемо:
$values = [2, 4, 1, 3, 1, 1]$
$row - indices = [1, 3, 2, 0, 3, 1]$
$column - indices = [0, 0, 1, 2, 2, 3]$

Приклад використання бібліотеки `scipy` для роботи з розрідженими матрицями:

```
import scipy.sparse as sp


values = [2, 4, 1, 3, 1, 1]
row_indices = [1, 3, 2, 0, 3, 1]
column_indices = [0, 0, 1, 2, 2, 3]
A = sp.coo_matrix((values, (row_indices, column_indices)), shape=(4,4))
print(A.todense())
```

Вивід:

```
[[0 0 3 0]
 [2 0 0 1]
 [0 1 0 0]
 [4 0 1 0]]
```

5. Обробка Природної Мови (Natural Language Processing, NLP)

Велика кількість даних у багатьох реальних наборах даних представлена у вигляді так званого вільного тексту (free text). Це є, наприклад, коментарі користувачів, телеметрія програмних додатків, будь-які «неструктуровані» поля, та багато чого іншого.

Обчислювальна обробка природної мови полягає в тому, що ми пишемо комп'ютерні програми, які можуть розуміти природну мову. Обробка природної мови (NLP) займається тим, як перетворити текст — часто нечіткий і непідготовлений — у форму, зручну для аналізу та моделювання. В цьому розділі ми спробуємо отримати деяку значущу інформацію з неструктурованих текстових даних.

Першим кроком до роботи з такими даними зазвичай стає підготовка тексту: приведення символів до одного регістру, видалення розділових знаків і зайвих символів, очищення чисел і зведення виробленого шуму у більш стандартизований вигляд.

Приклад очищення тексту (clean_text):

```
import re
import string
```

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import nltk

# Завантаження ресурсів NLTK (виконується один раз)
nltk.download('stopwords')
nltk.download('punkt')

def clean_text(text):
        # Перевести в нижній регістр
        text = text.lower()

        # Видалити розділові знаки
        text = text.translate(str.maketrans('', '', string.punctuation))

        # Видалити числа
        text = re.sub(r'\d+', '', text)

        # Видалити спеціальні символи (залишаємо лише англійські літери та
пробіли)
        text = re.sub(r'[^a-z\s]', '', text)

        # Видалити стоп-слова
        stop_words = set(stopwords.words('english'))
        text = ' '.join(word for word in word_tokenize(text) if word not in
stop_words)

        return text

sample_text = "Hello! This is an example text with numbers 123 and punctuation!!!"
cleaned_text = clean_text(sample_text)
print(cleaned_text)  # очікуваний вивід: hello example text numbers
```

Після базового очищення йде токенізація — розбиття тексту на слова або речення, що дозволяє оперувати окремими «будівельними блоками» змісту. Далі часто застосовують стемінг або лематизацію: стемінг обрізає закінчення й повертає корінь слова, іноді утворюючи несловникові форми, тоді як лематизація намагається відновити словникову форму з урахуванням контексту. Нормалізація може також включати виправлення орфографії, розгортання абревіатур і узгодження синонімів — усе задля того, щоб одна й та сама ідея не розпадалася на багато еквівалентних представлень.

Токенізація (word/sentence tokenization):

```
from nltk.tokenize import word_tokenize, sent_tokenize

def tokenize_text(text):
        words = word_tokenize(text)
        sentences = sent_tokenize(text)
        return words, sentences

sample_text = "Natural Language Processing (NLP) is exciting! It enables computers
to understand human language."
```

```
words, sentences = tokenize_text(sample_text)
print('Words:', words)
print('Sentences:', sentences)
```

Стемінг та лематизація:

```
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.tokenize import word_tokenize
import nltk

# Завантаження словника для лематизації
nltk.download('wordnet')

def apply_stemming(text):
        stemmer = PorterStemmer()
        words = word_tokenize(text)
        stemmed_words = [stemmer.stem(word) for word in words]
        return stemmed_words

def apply_lemmatization(text):
        lemmatizer = WordNetLemmatizer()
        words = word_tokenize(text)
        lemmatized_words = [lemmatizer.lemmatize(word) for word in words]
        return lemmatized_words

sample_text = "The children are playing with their toys."
print('Stemmed:', apply_stemming(sample_text))
print('Lemmatized:', apply_lemmatization(sample_text))
```

Нормалізація (спрощені приклади: виправлення орфографії, абревіатури):

```
from nltk.tokenize import word_tokenize
from autocorrect import Speller
import nltk

nltk.download('punkt')

def normalize_text(text):
        spell = Speller(lang='en')
        normalization_dict = {
                "u": "you",
                "ur": "your",
                "r": "are",
                "pls": "please",
                "thx": "thanks"
        }
        words = word_tokenize(text.lower())
        normalized_words = [normalization_dict.get(word, spell(word)) for word in
words]
```

```
        normalized_text = ' '.join(normalized_words)
        normalized_text = normalized_text.replace('!', '').replace('.',
'').strip()
        return normalized_text

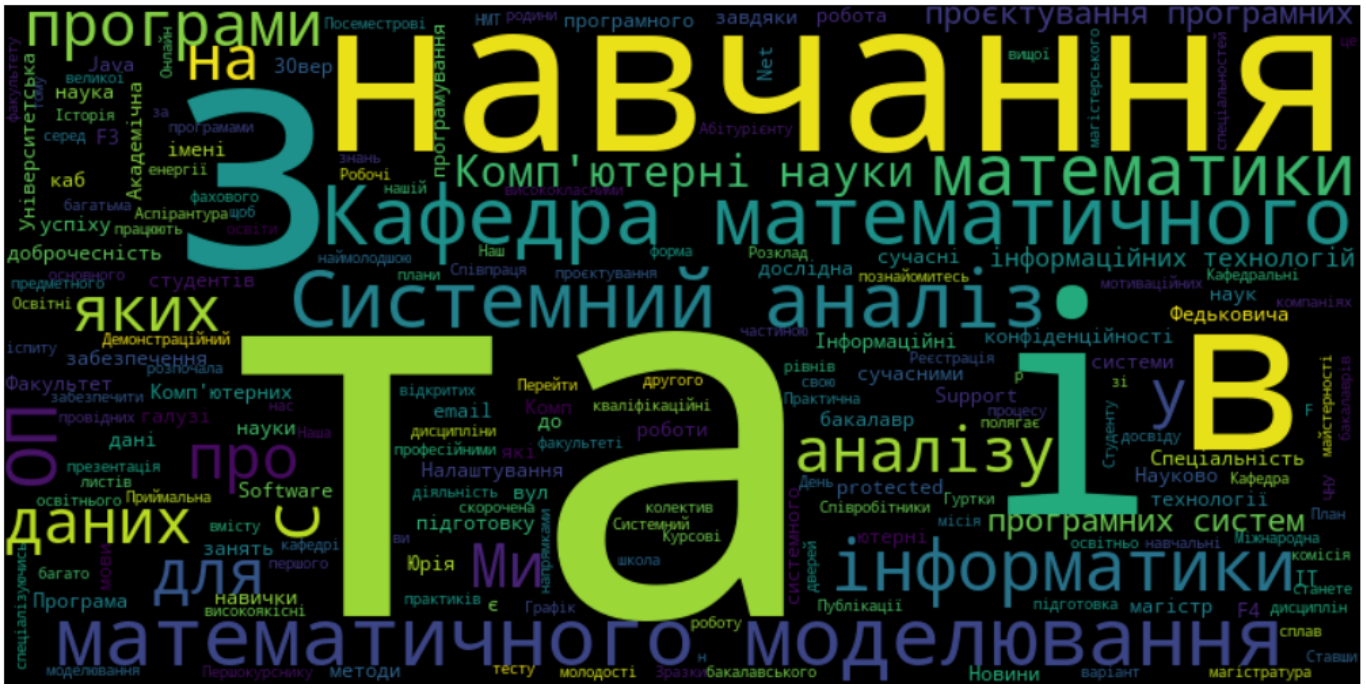
sample_text = "U r the best! Pls help me with this assignment. Thx!"
print(normalize_text(sample_text))
```

Після підготовки тексту його потрібно представити чисельно. Простий і часто ефективний підхід — модель «мішка слів», при якій кожен документ кодується вектором частот слів. Щоб зменшити вплив дуже частих, але малоінформативних слів, використовують вагу зворотної частоти документа: $idf_i = \log\left(\frac{N_{documents}}{N_{documents\text{-}with\text{-}word_i}}\right)$. Поєднання частоти терміна та зворотної частоти документа дає відому міру TF–IDF: $tfidf_{i,j} = tf_{i,j} \times idf_i$, яка підсилює рідкісні й інформативні слова й послаблює надто загальні.

Генерація wordcloud (приклад збору тексту зі сторінки та візуалізації):

```
from wordcloud import WordCloud
from bs4 import BeautifulSoup
import requests
import re

response = requests.get("https://mathmod.chnu.edu.ua/")
root = BeautifulSoup(response.content, "lxml")
wc = WordCloud(width=800, height=400).generate(re.sub(r"\s+", " ", root.text))
wc.to_image()
```



TF (bag-of-words) — приклад побудови словника та матриці частот:

```
documents = [
        "In Inception, the plot unfolds with thrilling action, keeping audiences
on edge through a series of intense and unexpected twists.",
        "The story of Inception keeps audiences hooked with its thrilling action
and surprising twists, making every moment unforgettable.",
        "The Matrix transformation preserves the vector's direction, illustrating
how linear combinations affect the overall system."
]

document_words = [doc.lower().split() for doc in documents]
vocab = sorted(set(sum(document_words, [])))
vocab_dict = {k: i for i, k in enumerate(vocab)}

import numpy as np
X_tf = np.zeros((len(documents), len(vocab)), dtype=int)
for i, doc in enumerate(document_words):
        for word in doc:
                X_tf[i, vocab_dict[word]] += 1

print(X_tf)
```

Косинусна подібність — простий спосіб виміряти схожість між двома векторними представленнями тексту: $CosineSim(x, y) = \frac{x \cdot y}{\|x\|_2 \|y\|_2}$. Вона добре працює для порівняння документів у просторі TF–IDF, але має й обмеження: в такому підході немає розуміння семантики слів, і різні слова, які мають близьке значення, залишаються віддаленими в просторі one-hot або частотних векторів.

Щоб виправити цю проблему, використовують вбудовування слів (word embeddings) — щільні вектори фіксованого розміру, у яких семантично близькі слова займають близькі позиції у просторі. Моделі на кшталт word2vec або переднавчені матриці типу GloVe навчають векторні представлення таким чином, що арифметичні операції над векторами можуть відображати мовні відношення: наприклад, операція $\mathrm{king} - \mathrm{man} + \mathrm{woman}$ зазвичай наближається до вектора, близького до queen. Це відкриває шлях до більш змістовних порівнянь і до знаходження синонімів, антонімів і тематичних зв'язків.

IDF, TF–IDF та косинусна подібність:

```
idf = np.log(X_tf.shape[0] / X_tf.astype(bool).sum(axis=0))
X_tfidf = X_tf * idf
X_tfidf_norm = X_tfidf / np.linalg.norm(X_tfidf, axis=1)[:, None]
M = X_tfidf_norm @ X_tfidf_norm.T
print('IDF:', idf)
print('TF-IDF matrix:\n', X_tfidf)
print('Cosine similarity matrix:\n', M)
```

Word embeddings (GloVe через gensim.downloader) та приклад аналогій:

```
import gensim.downloader as api
word_vectors = api.load("glove-wiki-gigaword-100")
vector = word_vectors['computer']
print('vector sample:', vector[:10])
print('similarity woman/man:', word_vectors.similarity('woman', 'man'))

king = word_vectors['king']
man = word_vectors['man']
woman = word_vectors['woman']
result_vector = king - man + woman
print('Analogy results:')
for word, similarity in word_vectors.similar_by_vector(result_vector, topn=5):
        print(word, similarity)
```

TF–IDF і схожість у Gensim (корпус, словник, модель TFIDF, MatrixSimilarity):

```
import gensim as gs
from gensim.test.utils import lee_corpus_list
from gensim.models import Word2Vec

documents = [
        "kyiv has some excellent new restaurants",
        "kharkiv is a city with great cuisine",
        "postgresql is a relational database management system"
]
document_words = [doc.split() for doc in documents]
dictionary = gs.corpora.Dictionary(document_words)
corpus = [dictionary.doc2bow(doc) for doc in document_words]
tfidf = gs.models.TfidfModel(corpus)
X_tfidf = gs.matutils.corpus2csc(tfidf[corpus])
print(X_tfidf.todense().T)

M = gs.similarities.MatrixSimilarity(tfidf[corpus])
print(M.get_similarities(tfidf[corpus]))
```

Далі стоять мовні моделі, які намагаються оцінити ймовірність слова, враховуючи контекст: $P(word_i \mid word_1, \ldots, word_{i-1})$. Простий, але корисний клас таких моделей — n-грами, що враховують обмежений контекст попередніх $n-1$ слів; їх можна вважати аппроксимацією повної ймовірнісної моделі мови. Сучасні нейронні та трансформерні моделі значно перевершують n-грами за здатністю захоплювати довготривалі залежності та контекст.

У практиці науковця з даних частина роботи полягає в доборі правильного представлення залежно від задачі: для швидкої класифікації рецензій часто достатньо TF–IDF і лінійних моделей; для складних завдань семантичного пошуку або витягнення сутностей корисні вбудовування слів і контекстні мовні моделі. Важливо пам'ятати, що підготовка даних і прості методи іноді дають вражаючі результати, і вибір інструменту має базуватись на доступності даних, вимогах до точності й обмеженнях по часу на навчання.

6. Теорія ймовірності в контексті аналізу даних

```
import random
import math
import numpy as np
from collections import Counter

pX = {1: 0.1, 2: 0.5, 3:0.4}
pWeather = {"sunny": 0.3, "rainy":0.2, "cloudy":0.4, "snowy":0.1}
print(pWeather["sunny"])
```

```
0.3
```

Приклад: Кидання монети

```
# Симудюємо 10 підкидувань
tosses = [random.choice(["H", "T"]) for _ in range(10)]
print(tosses)

# Емпірична ймовірність випадання «решки»
p_heads = tosses.count("H") / len(tosses)
print(f"P(Heads) ≈ {p_heads:.2f}")
```

```
['H', 'T', 'T', 'H', 'H', 'H', 'H', 'T', 'T', 'T']
P(Heads) ≈ 0.50
```

Закон великих чисел

Давайте подивимося, як емпірична ймовірність наближається до теоретичної ймовірності.

```
def simulate_coin(n_tosses=1000):
    tosses = [random.choice(["H", "T"]) for _ in range(n_tosses)]
    return tosses.count("H") / n_tosses

for n in [10, 100, 1000, 10000]:
    print(f"{n} tosses → P(Heads) ≈ {simulate_coin(n):.3f}")

# По мірі того, як число підкидувань зростає → ймовірність стабілізується около
0.5.
```

```
10 tosses → P(Heads) ≈ 0.200
100 tosses → P(Heads) ≈ 0.580
1000 tosses → P(Heads) ≈ 0.496
10000 tosses → P(Heads) ≈ 0.500
```

Використання NumPy для роботи з розподілами ймовірностей

Рівномірний розподіл:

```
import numpy as np

# Генеруємо випадкові числа між 0 та 1
samples = np.random.rand(5)
print(samples)

# Ймовірність значення < 0.5
p = np.mean(samples < 0.5)
print("P(X < 0.5) ≈", p)
```

```
[0.75955599 0.02132349 0.45415367 0.32714836 0.93499247]
P(X < 0.5) ≈ 0.6
```

Гаусівський розподіл (Normal (Gaussian) Distribution)

Гаусівський розподіл є розподілом по дійсних числах і, ймовірно, є найпоширенішим розподілом у науці про дані (я не кажу, що це найпоширеніший розподіл у самих даних, а в науці про дані як галузі). Це пояснюється багатьма приємними властивостями розподілу: він легко інтегрується, має природний багатовимірний аналог, а також тим фактом, про який ми поговоримо в декількох лекціях, що певні розподіли даних середніх значень вибірки мають тенденцію до гаусівського розподілу (так звана центральна гранична теорема). Гаусівський розподіл має «дзвоноподібну» функцію щільності, яка, ймовірно, знайома багатьом людям.

Згенеруємо гаусівський розподіл засобами NumPy:

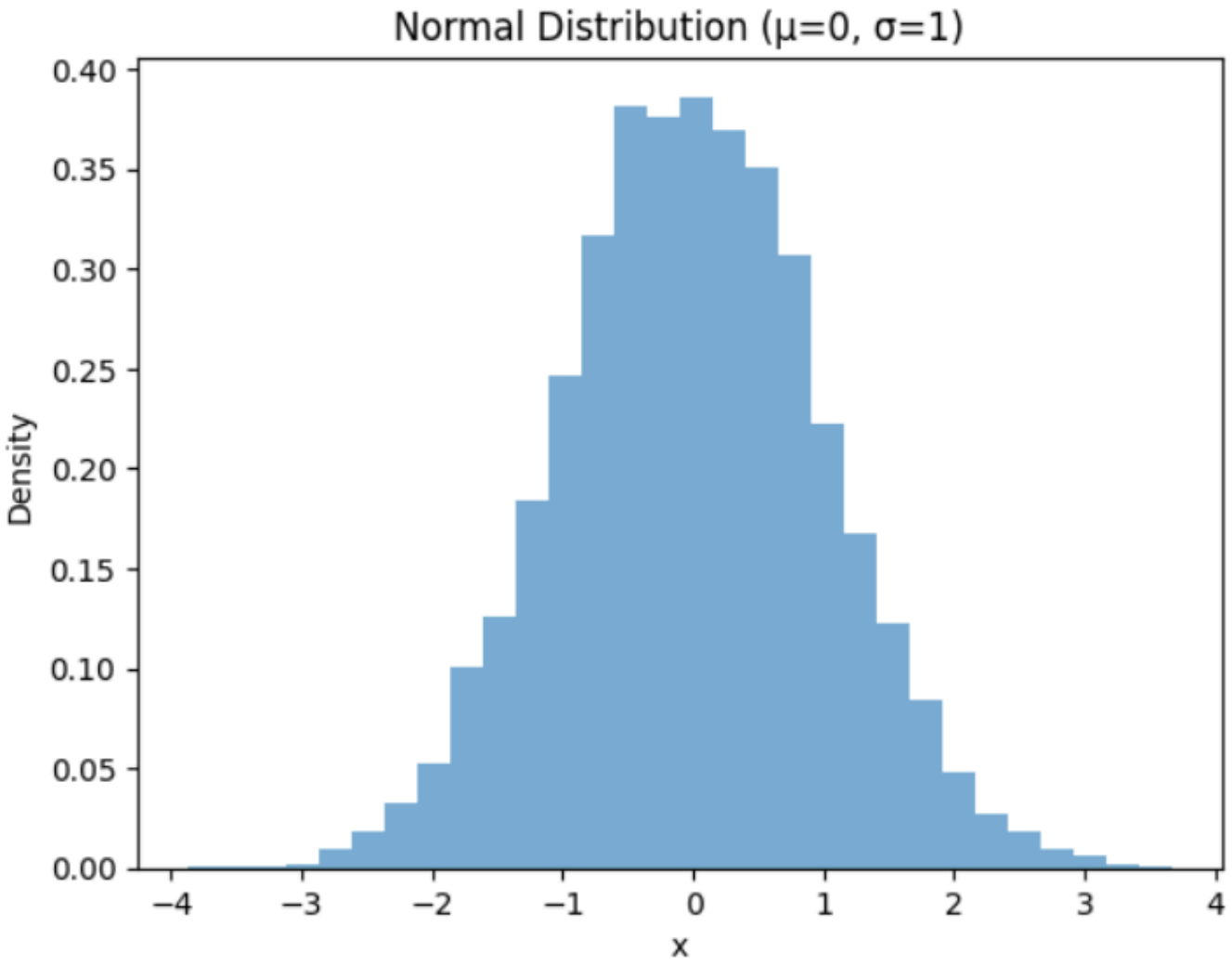
```
samples = np.random.normal(loc=0, scale=1, size=10000)  # середнє=0, відхилення=1
print("Mean:", np.mean(samples))
print("Std Dev:", np.std(samples))

import matplotlib.pyplot as plt

plt.hist(samples, bins=30, density=True, alpha=0.6)
plt.title("Normal Distribution (μ=0, σ=1)")
plt.xlabel("x")
```

```
plt.ylabel("Density")
plt.show()
```

```
Mean: -0.002846126267790618
Std Dev: 0.9992466721005764
```



Дискретні розподіли (приклад біноміального розподілу)

```
from scipy.stats import binom

# Параметри
n = 10   # кількість випробувань
p = 0.5  # ймовірність успіху

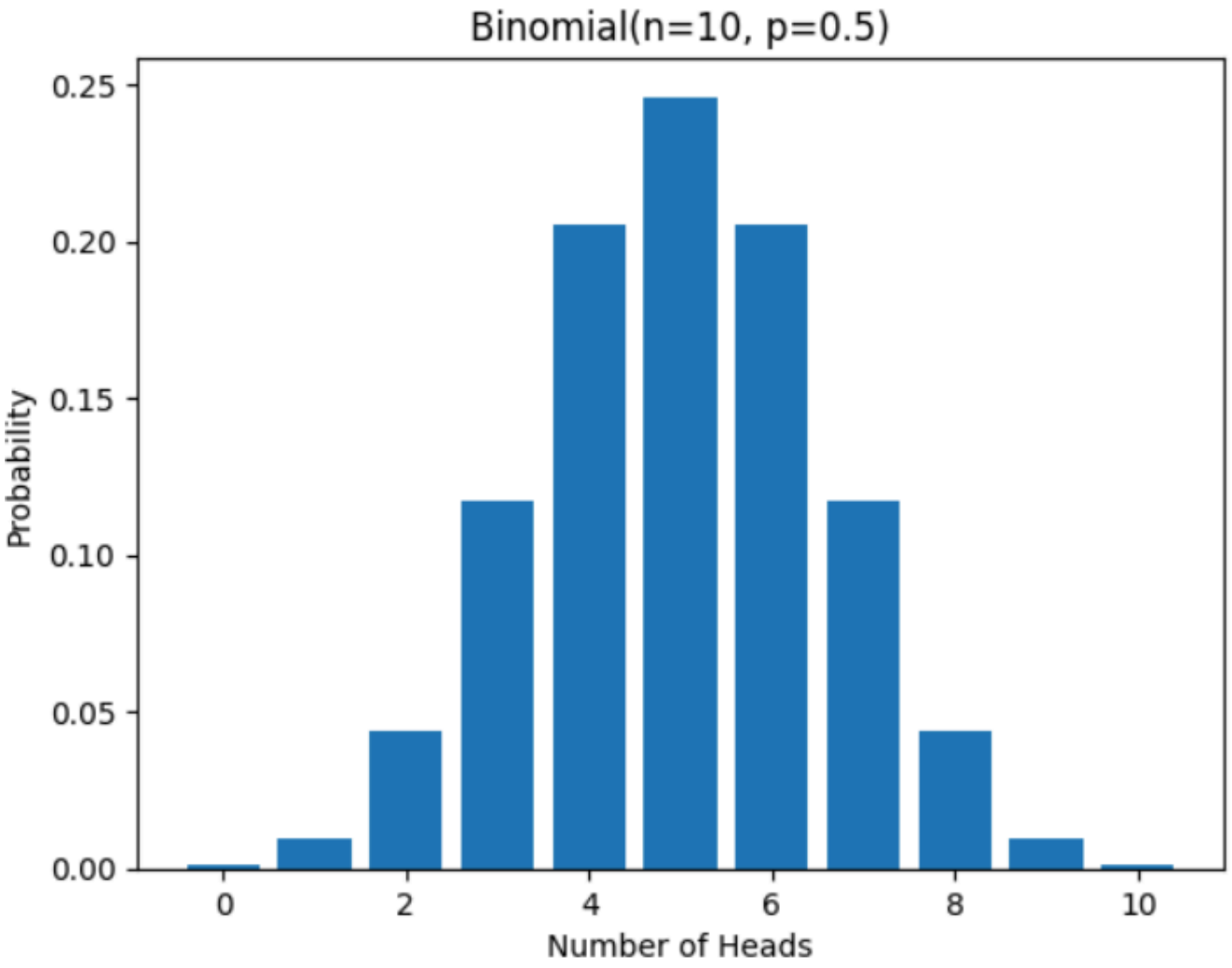
# Імовірність випадання рівно 3 «решок» за 10 підкидань
prob = binom.pmf(k=3, n=n, p=p)
print(f"P(X=3) = {prob:.4f}")

# Кумулятивна ймовірність P(X ≤ 3)
cum_prob = binom.cdf(k=3, n=n, p=p)
print(f"P(X ≤ 3) = {cum_prob:.4f}")
```

```
x = np.arange(0, 11)
y = binom.pmf(x, n, p)

plt.bar(x, y)
plt.xlabel("Number of Heads")
plt.ylabel("Probability")
plt.title("Binomial(n=10, p=0.5)")
plt.show()
```

```
P(X=3) = 0.1172
P(X ≤ 3) = 0.1719
```



Теорема Байєса

```
# Обчислимо ймовірність захворювання за умови позитивного результату тесту.
# 1 % населення хворіє → P(Захворювання) = 0,01
# Точність тесту становить 99 % →
# P(Позитивний результат | Захворювання) = 0,99
# P(Позитивний результат | Відсутність захворювання) = 0,01
# Обчислимо P(Захворювання | Позитивний результат):
```

```
P_D = 0.01
P_Pos_given_D = 0.99
P_Pos_given_notD = 0.01

# Загальна ймовірність позитивного результату тесту
P_Pos = P_Pos_given_D * P_D + P_Pos_given_notD * (1 - P_D)

# Формула Байєса
P_D_given_Pos = (P_Pos_given_D * P_D) / P_Pos
print(f"P(Disease | Positive Test) = {P_D_given_Pos:.3f}")

# Лише ~50%, навіть при тесті з «99% точністю» — базова частота має значення!
```

```
P(Disease | Positive Test) = 0.500
```

7. Вступ в машинне навчання

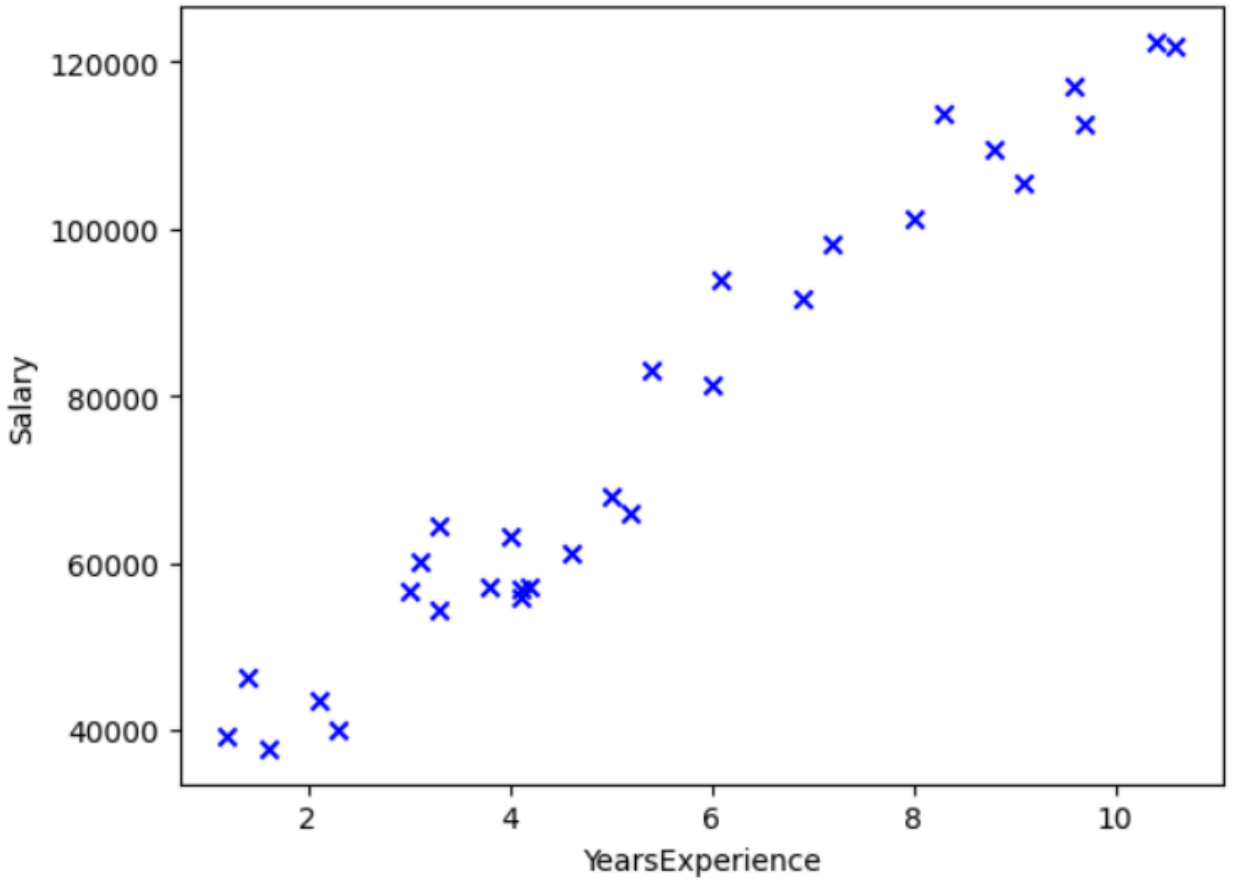
Машинне навчання є на даний момент швидкозростаючою галуззю, хоча слід підкреслити, що початкові методи, які ми тут розглянемо (а саме лінійна регресія), з'явилися приблизно на 200 років раніше, ніж сам термін (зазвичай вважається, що Гаусс розробив метод найменших квадратів близько 1800 року). Більшість пізніших методів, про які ми говоримо, були добре вивчені в статистиці на початку століття. Ми все одно будемо використовувати загальний термін «машинне навчання» для позначення всіх цих методів, але важливо знати, що ці ідеї не виникли в спільноті ML. Натомість тема машинного навчання розвинулася завдяки поєднанню трьох різних елементів:

- зростання обчислювальної потужності (машинне навчання, яке виникло з інформатики, завжди було фундаментально пов'язане з обчислювальними алгоритмами),
- величезні обсяги доступних даних (сировина для методів машинного навчання) та
- деякі значні досягнення в галузі алгоритмів, що відбулися за останні 30 років. Однак для початку ми розглянемо деякі з найпростіших алгоритмів, щоб систематизувати основні принципи.

```
import matplotlib.pyplot as plt
import pandas as pd

def build_plot(file_path, x, y):
    data = pd.read_csv(file_path)
    plt.scatter(data[x], data[y], c='blue', marker='x')
    plt.xlabel(x)
    plt.ylabel(y)
    return plt

build_plot('../resources/Salary_dataset.csv', 'YearsExperience', 'Salary').show()
```



Ось простий приклад залежності між стажем роботи співробітників та їхньою заробітною платою. Чи можна передбачити другий параметр на основі значення першого?

Можна припустити, що залежність відповідає лінійній моделі.

$$y \approx \theta_1 \cdot x + \theta_2$$

Тут

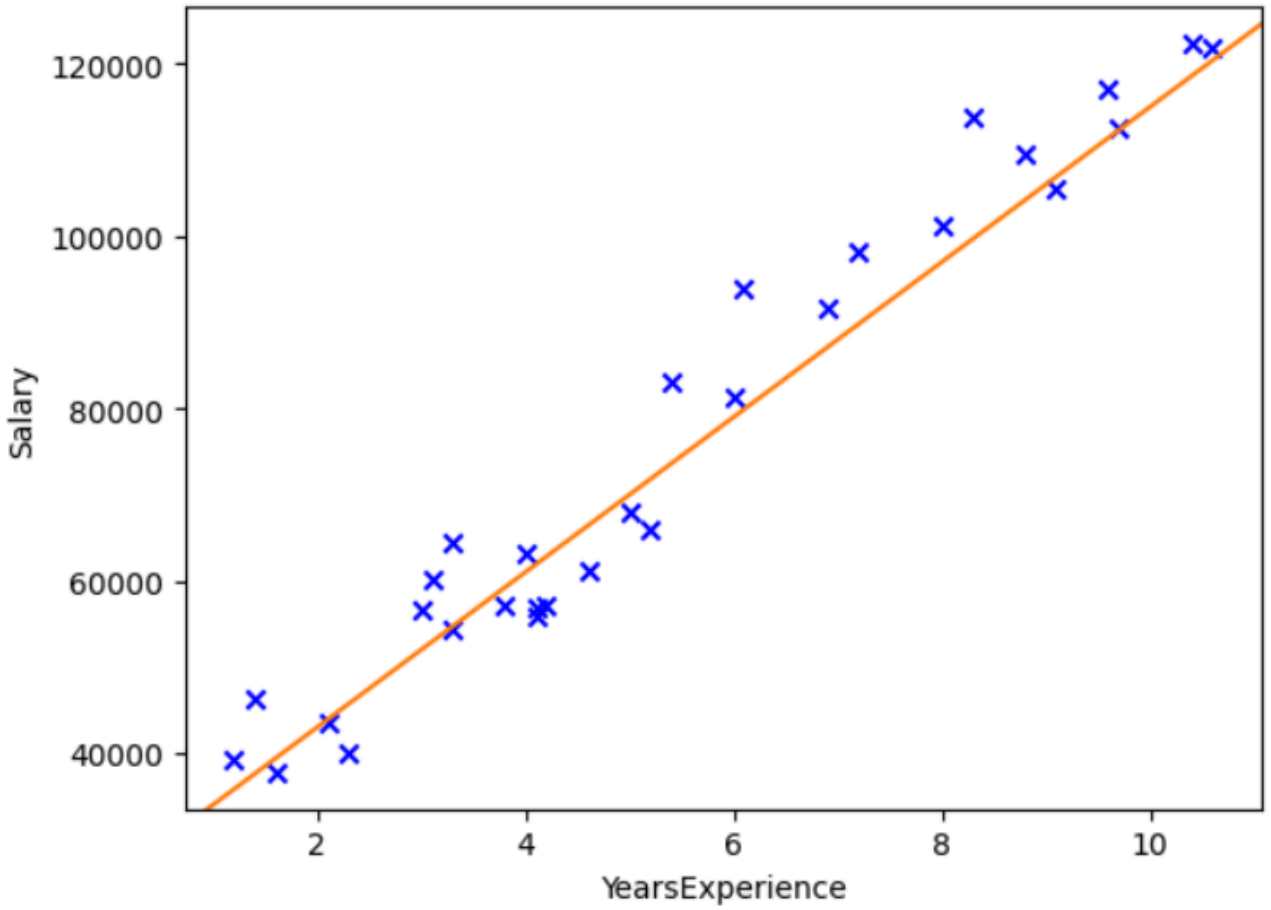
- θ_1 це “нахил” лінії, та (slope)
- θ_2 зсув. (intercept)

Чи можемо ми робити передбачення даних?

```
import numpy as np

def build_line(plt, theta):
    xlim, ylim =(plt.gca().get_xlim(), plt.gca().get_ylim())
    plt.plot(xlim, [theta[0]*xlim[0]+theta[1], theta[0]*xlim[1]+theta[1]], 'C1')
    plt.xlim(xlim)
    plt.ylim(ylim)

plt = build_plot('../resources/Salary_dataset.csv', 'YearsExperience', 'Salary')
build_line(plt, np.array([9000, 25000]))
```



Градієнтний спуск (Gradient descent)

Припустимо, що прогнозована зарплата відповідає лінійній моделі. Як ми можемо знайти θ?

Є багато можливостей, але природною метою є мінімізація деякої різниці між цією лінією і спостережуваними даними, наприклад, квадратична втрата:

$$E(\theta) = \sum_i (\theta_1 * x_i + \theta_2 - y_i)^2$$

Тепер нам потрібно знайти параметри, які мінімізують значення функції $E(\theta)$.

$$minimize_\theta E(\theta)$$

Найкращий спосіб знайти значення функції $f(x)$, в якій її значення є мінімальним, — це знайти похідну. Ми починаємо з деякої точки і змінюємо вхідне значення в напрямку від'ємної похідної. У випадку багатозмінних функцій ми будемо використовувати часткові похідні.

Щоб знайти хороше значення θ, ми можемо повторно робити кроки в напрямку негативних похідних для кожного значення. Ітерація:

$$\theta_1 := \theta_1 - \alpha \frac{\delta}{\delta \theta_1} E(\theta_1, \theta_2)$$

$$\theta_2 := \theta_2 - \alpha \frac{\delta}{\delta \theta_2} E(\theta_1, \theta_2)$$

де α — це невелике додатне число, яке називається розміром кроку. Це алгоритм градієнтного спуску, основа сучасного машинного навчання.

Тут:

$$\frac{\delta}{\delta\theta_1}E(\theta_1,\theta_2) = \sum_i 2(\theta_1 * x_i + \theta_2 - y_i) * x_i$$

$$\frac{\delta}{\delta\theta_2}E(\theta_1,\theta_2) = \sum_i 2(\theta_1 * x_i + \theta_2 - y_i)$$

Застосуємо градієнтний спуск до наших даних, спочатку нормалізувавши їх.

```
def build_plot_normalized(file_path, x, y):
    data = pd.read_csv(file_path)
    x_nor = (data[x] - min(data[x])) / (max(data[x]) - min(data[x]))
    y_nor = (data[y] - min(data[y])) / (max(data[y]) - min(data[y]))
    plt.scatter(x_nor, y_nor, c='blue', marker='x')
    plt.xlabel(x)
    plt.ylabel(y)
    return plt

build_plot_normalized('../resources/Salary_dataset.csv', 'YearsExperience',
'Salary').show()
```

Візуалізуємо процес градієнтного спуску.

```
# Visualizing the gradient descent:

def get_data(file_path, x_name, y_name):
    data = pd.read_csv(file_path)
    return data[x_name], data[y_name]

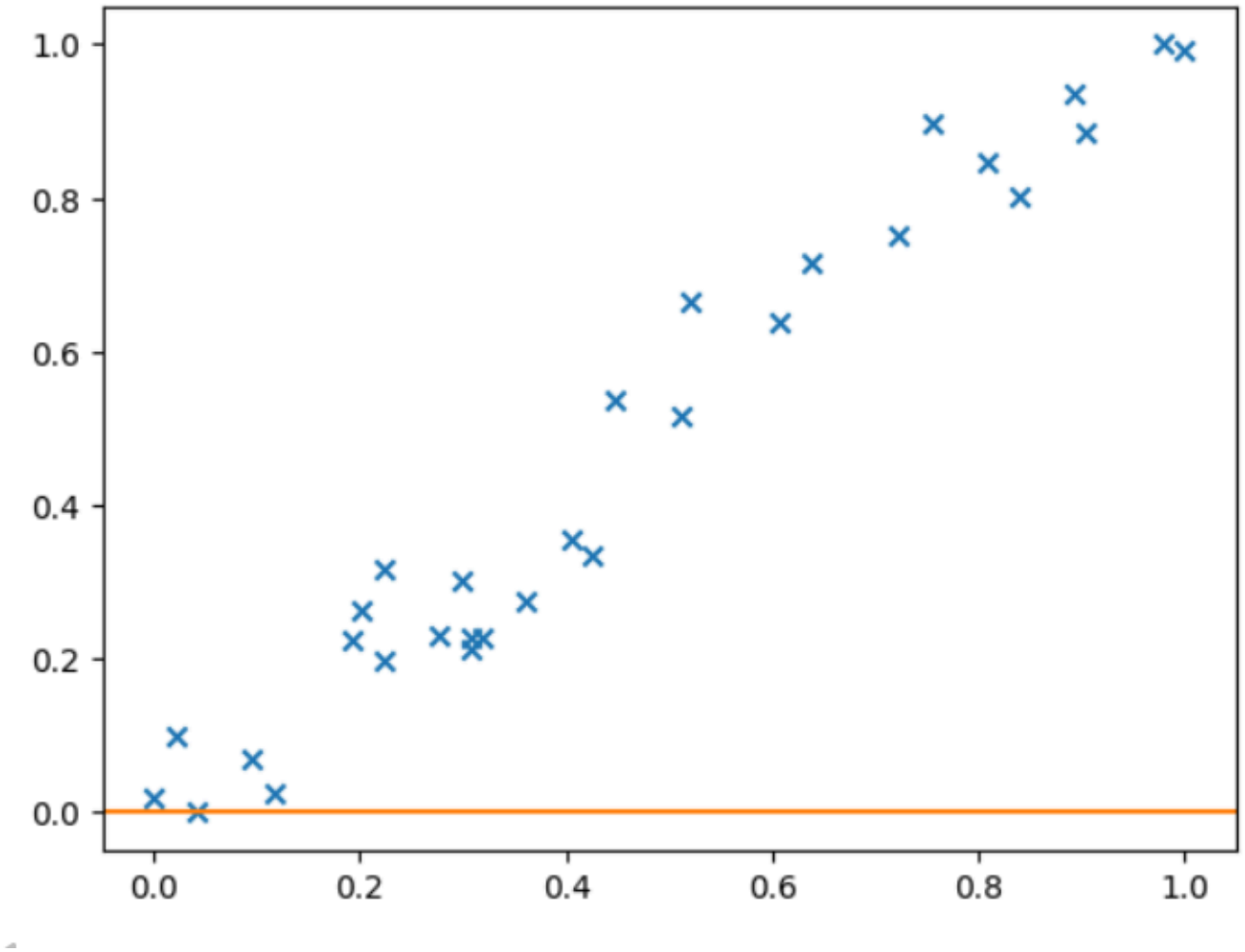
def normalize_data(x):
    x_normalized = (x - min(x)) / (max(x) - min(x))
    return x_normalized

def gradient_descent(x, y, iters, alpha):
    theta = np.array([0., 0.])
    for t in range(iters):
        theta[0] -= alpha/len(x) * 2 * sum((theta[0] * x + theta[1] - y)*x)
        theta[1] -= alpha/len(x) * 2 * sum((theta[0] * x + theta[1] - y) )
    return theta

def plot_fit(x, y, theta):
    plt.scatter(x, y, marker = 'x')
    xlim, ylim =(plt.gca().get_xlim(), plt.gca().get_ylim())
    plt.plot(xlim, [theta[0]*xlim[0]+theta[1], theta[0]*xlim[1]+theta[1]], 'C1')
    plt.xlim(xlim)
    plt.ylim(ylim)
```

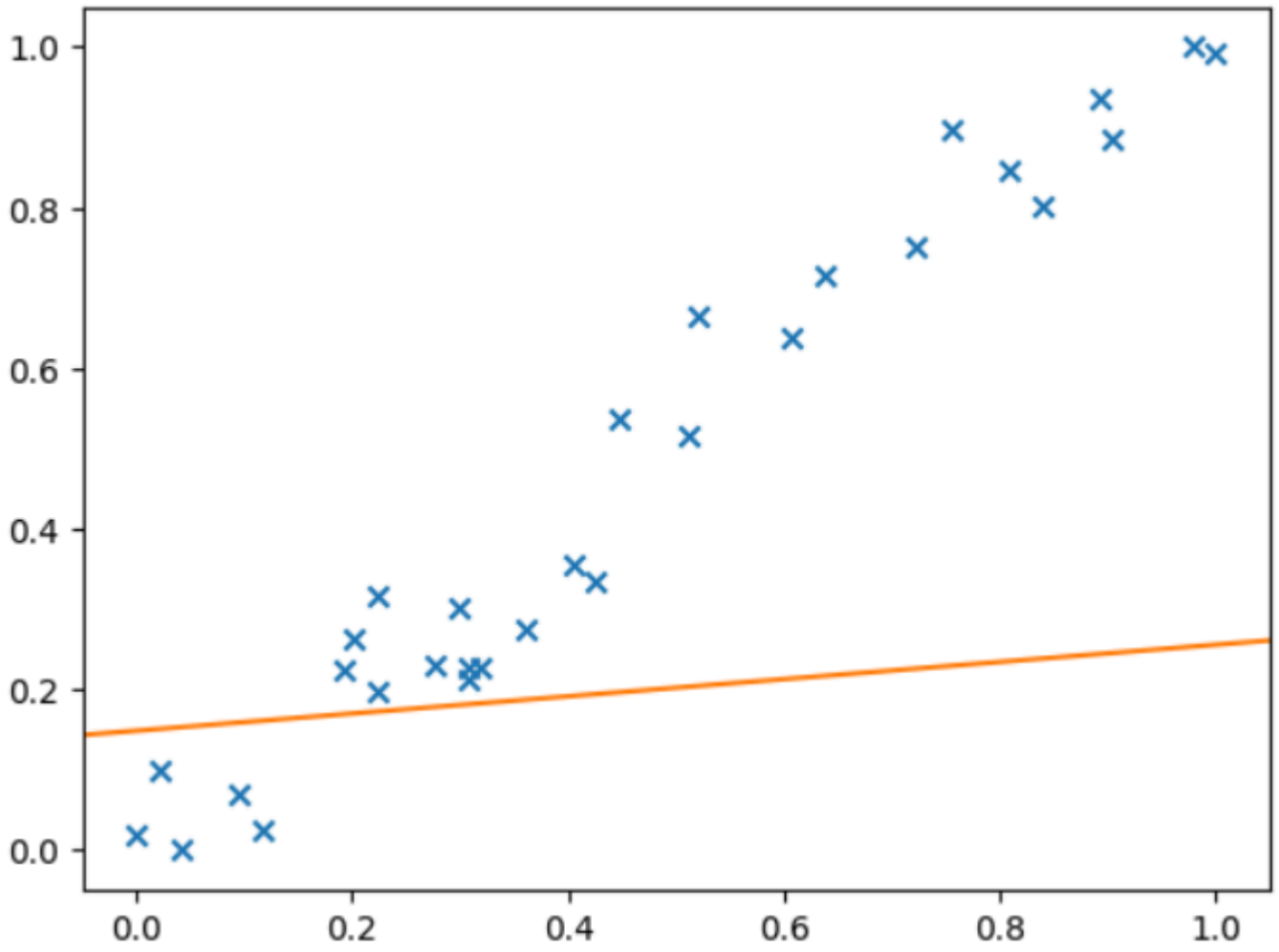
```
x, y = get_data('../resources/Salary_dataset.csv', 'YearsExperience', 'Salary')
x_normalized, y_normalized = normalize_data(x), normalize_data(y)

#first iteration
theta = gradient_descent(x_normalized, y_normalized, 0, 0.1)
plot_fit(x_normalized, y_normalized, theta)
```

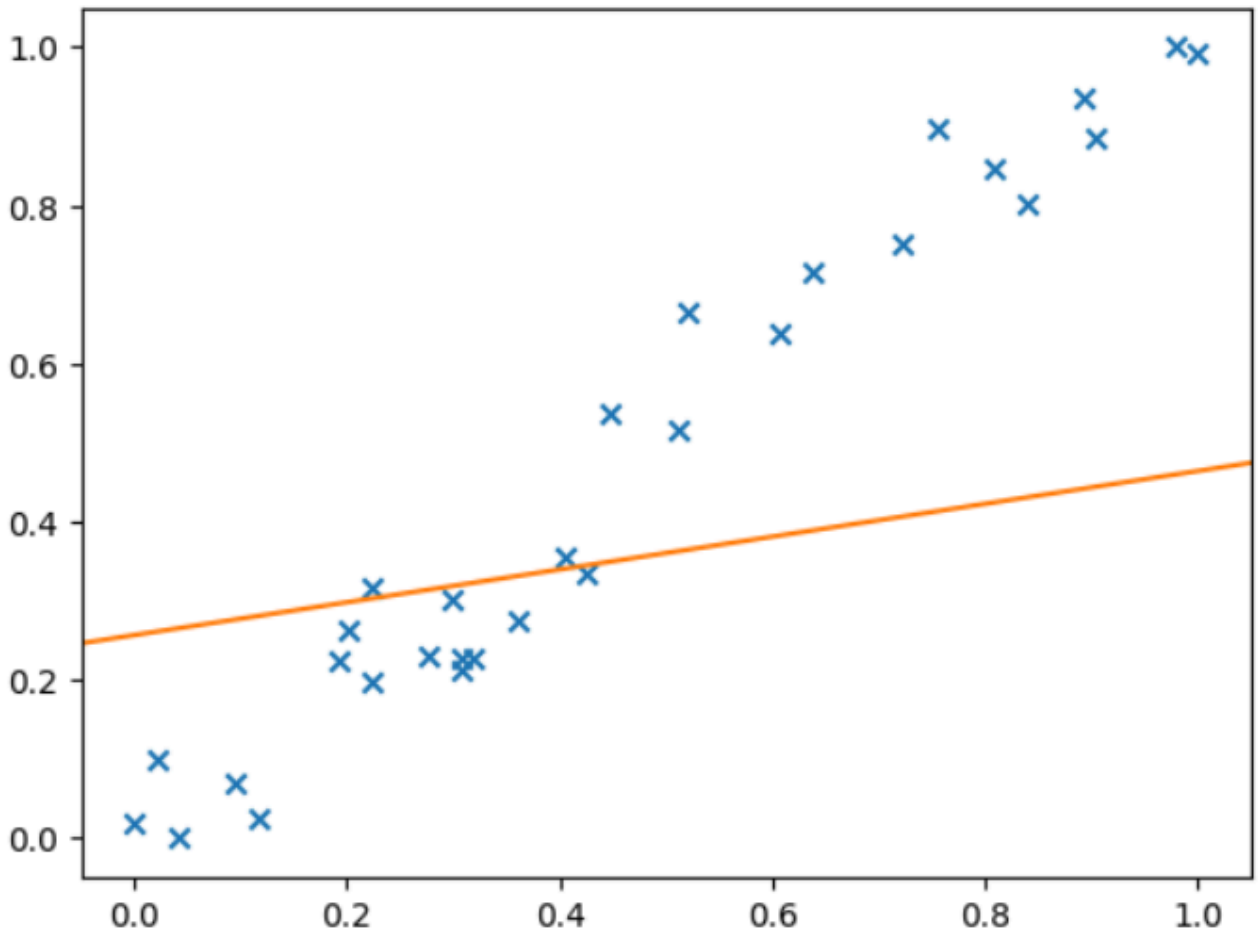


Як бачимо, на першій ітерації значення параметрів θ віддалені від бажаних. Проведемо іще декілька ітерацій.

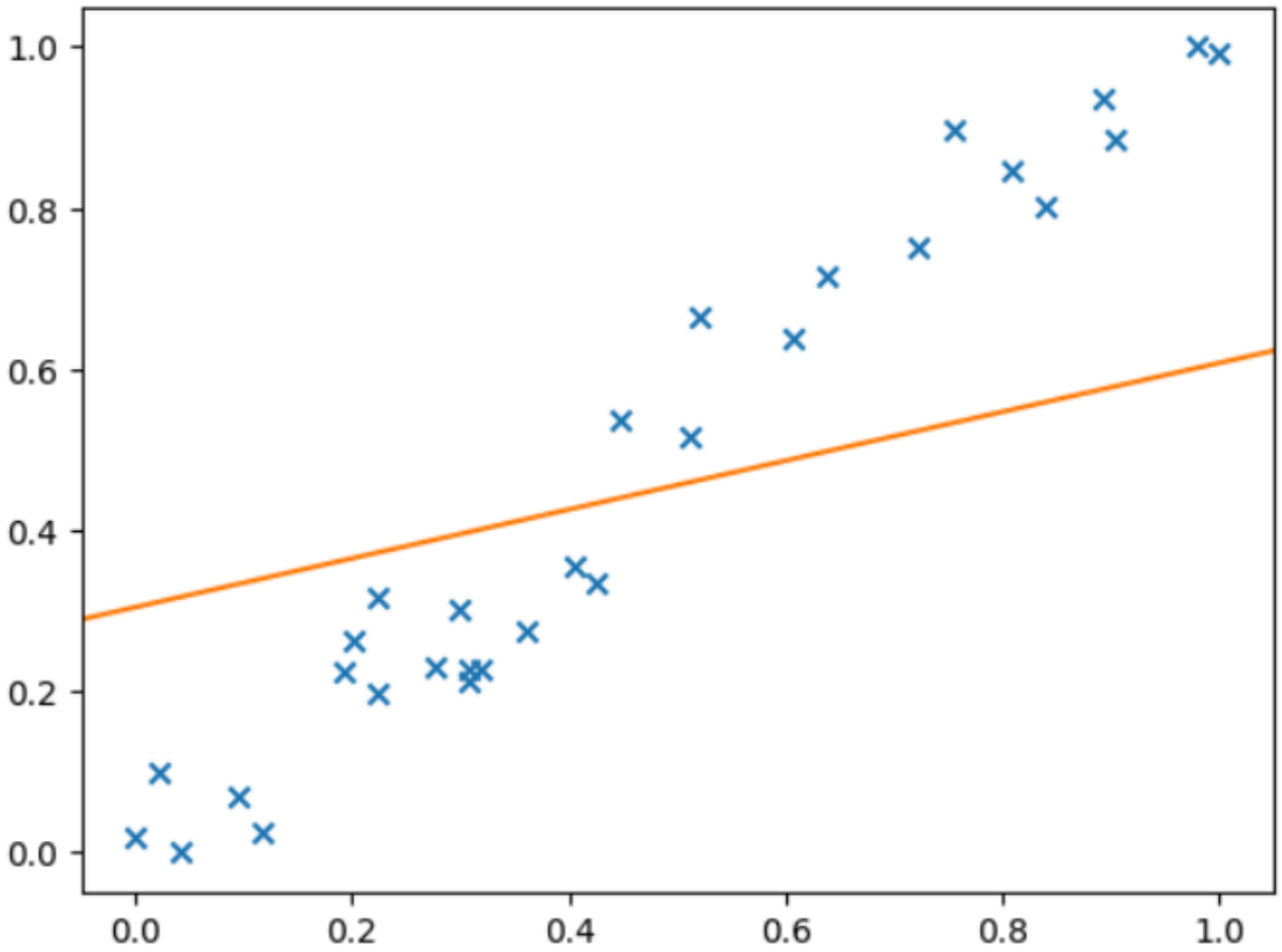
```
theta = gradient_descent(x_normalized, y_normalized, 2, 0.1) # change third
parameter to see more iterations
plot_fit(x_normalized, y_normalized, theta)
```



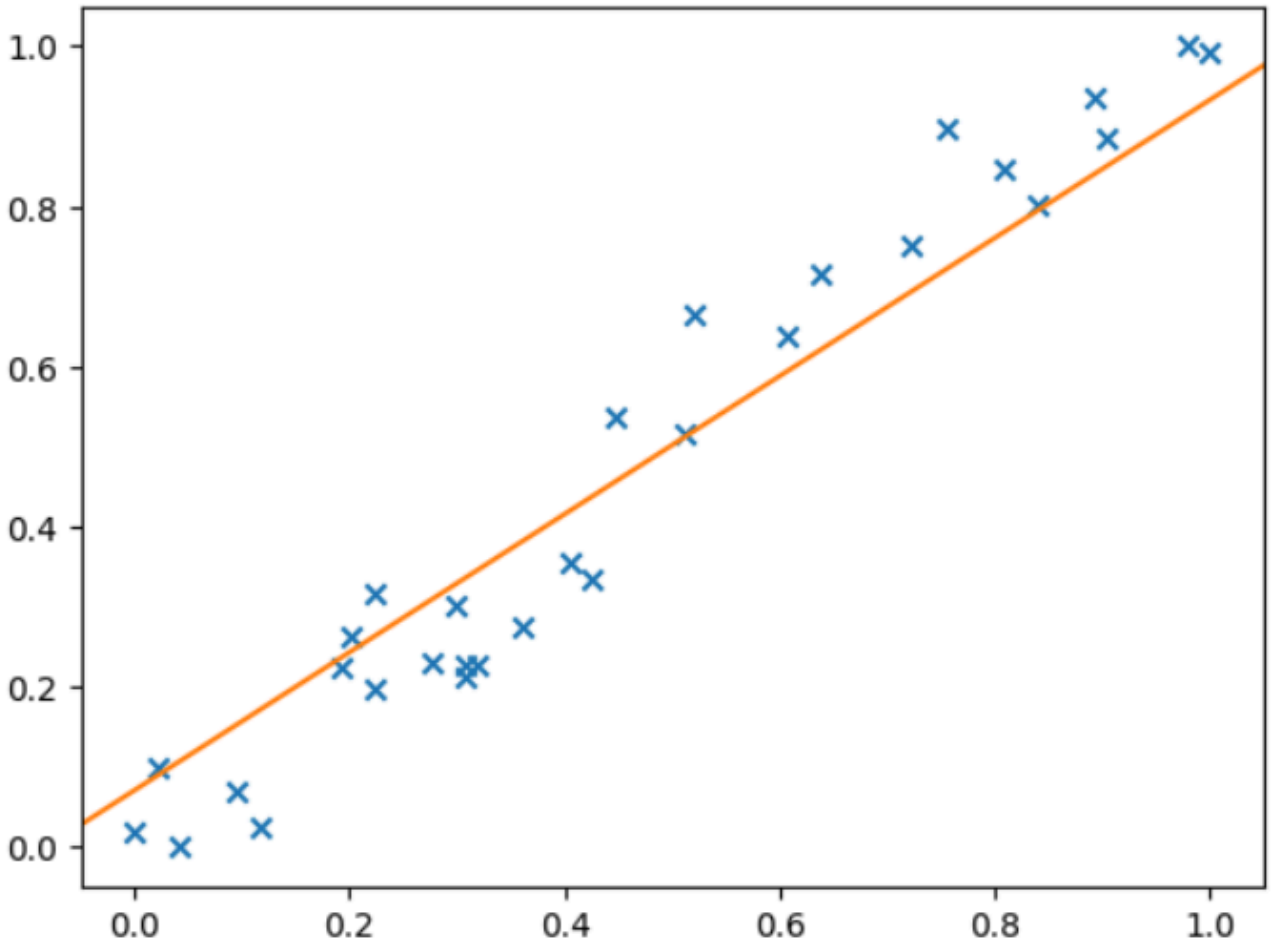

```
theta = gradient_descent(x_normalized, y_normalized, 5, 0.1) # change third
parameter to see more iterations
plot_fit(x_normalized, y_normalized, theta)
```




```
theta = gradient_descent(x_normalized, y_normalized, 10, 0.1) # change third
parameter to see more iterations
plot_fit(x_normalized, y_normalized, theta)
```




```
theta = gradient_descent(x_normalized, y_normalized, 100, 0.1) # change third
parameter to see more iterations
plot_fit(x_normalized, y_normalized, theta)
```



Після 100-ї ітерації ми наблизилися із нашою лінійною моделлю до реального розподілу.

Ми також можемо візуалізувати процес зміни параметрів θ:

```
# Visualizing parameter updates

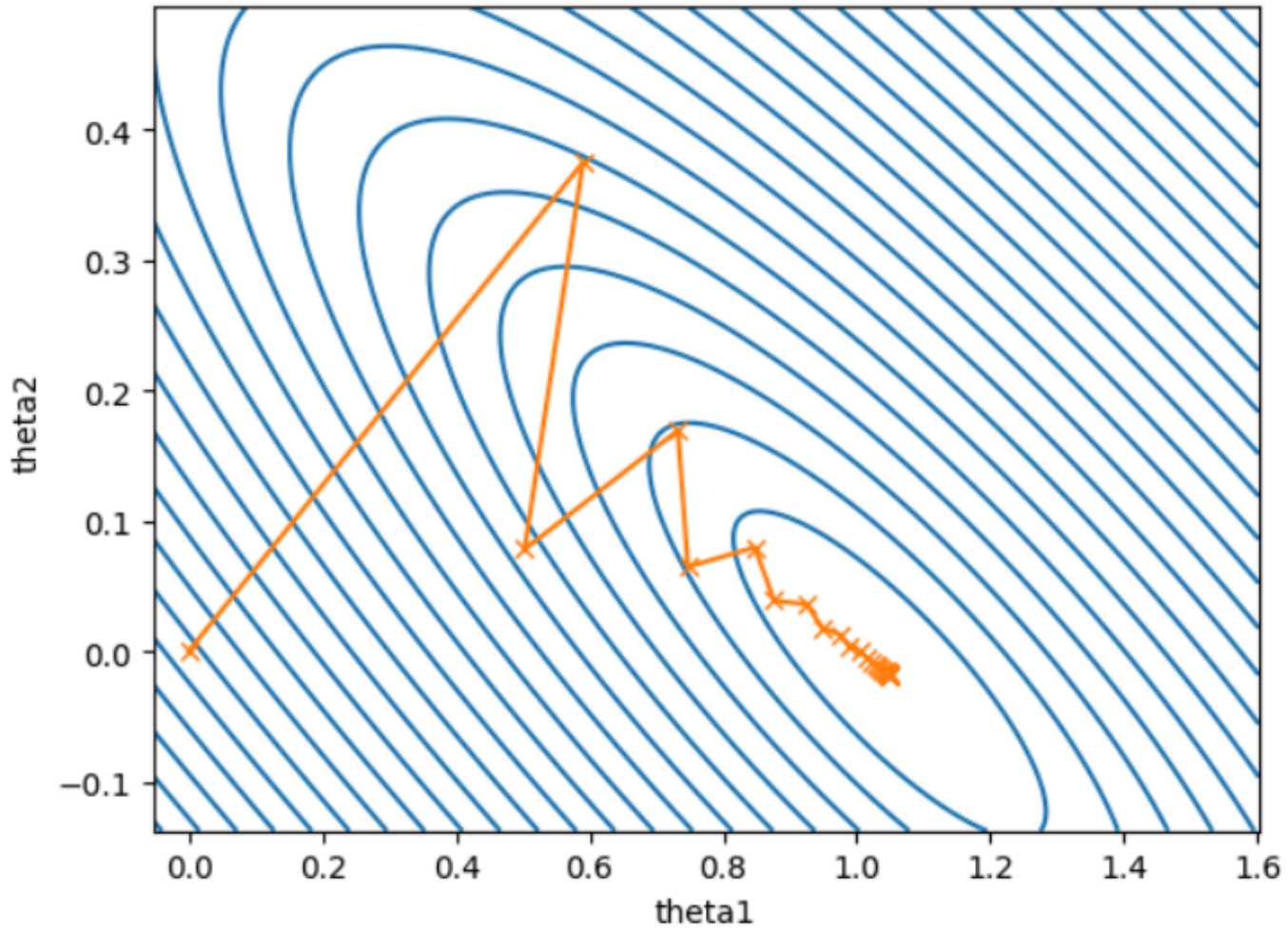
def gradient_descent_params(x, y, iters):
    thetas = []
    theta = np.array([0., 0.])
    alpha = 1.0
    for t in range(iters):
        thetas.append(theta.copy())
        theta[0] -= alpha/len(x) * 2 * sum((theta[0] * x + theta[1] - y)*x)
        theta[1] -= alpha/len(x) * 2 * sum((theta[0] * x + theta[1] - y) )
    return np.array(thetas)

def err(x, y, theta):
    return np.mean((np.outer(x, theta[:,0]) + theta[:,1] - y[:,None])**2,axis=0)

thetas = gradient_descent_params(x_normalized, y_normalized, 100)
plt.plot(thetas[:,0], thetas[:,1])
xlim, ylim =(np.array(plt.gca().get_xlim()), np.array(plt.gca().get_ylim()))
xlim += np.array([0,0.5])
ylim += np.array([-0.1, 0.1])

XX,YY = np.meshgrid(np.linspace(xlim[0],xlim[1],200), np.linspace(ylim[0],
```

```
ylim[1],200))
ZZ = err(x_normalized.to_numpy(), y_normalized.to_numpy(), np.hstack([np.ravel(XX)
[:,None], np.ravel(YY)[:,None]])).reshape(XX.shape)
#V = np.logspace(np.log(np.min(ZZ)), np.log(np.max(ZZ)), 30)
V = np.linspace(np.sqrt(np.min(ZZ)), np.sqrt(np.max(ZZ)), 25)**2
plt.clf()
plt.contour(XX,YY,ZZ, V, colors=('C0',))
plt.plot(thetas[:,0], thetas[:,1], 'C1-x')
plt.xlabel("theta1")
plt.ylabel("theta2")
```



Зробимо те саме лінійне прогнозування, використовуючи бібліотеку scikit-learn:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

data = pd.read_csv('../resources/Salary_dataset.csv')
X = np.array(data['YearsExperience']).reshape(-1, 1)
y = np.array(data['Salary'])

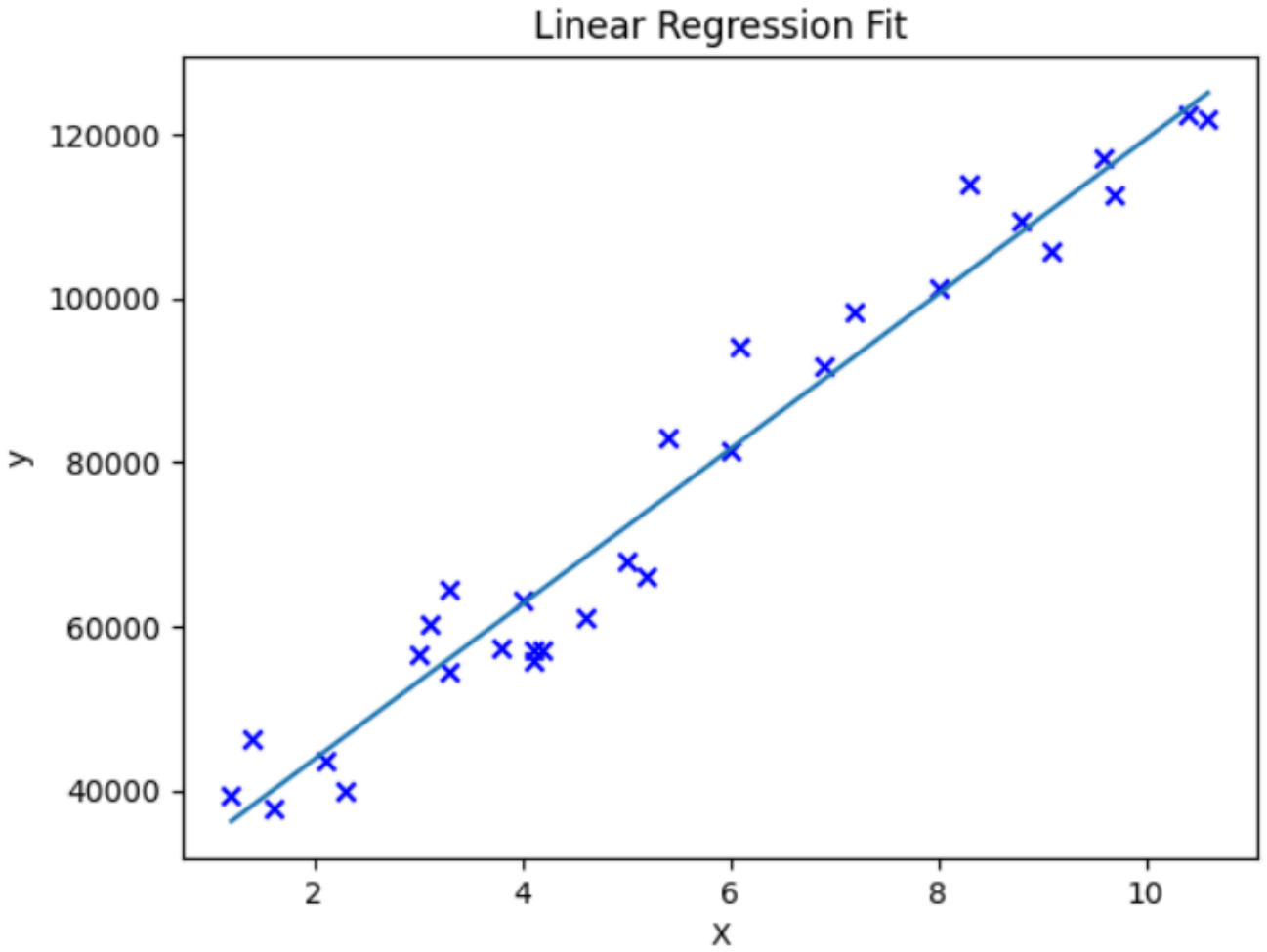
# етап тренування моделі
model = LinearRegression()
```

```
model.fit(X, y)

# отримуємо параметри theta
slope = model.coef_[0]       # theta_1
intercept = model.intercept_ # theta_2
print("slope:", slope, "intercept:", intercept)

# будуємо лінію
X_line = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
y_line = model.predict(X_line)

# plot
plt.scatter(X, y, c='blue', marker='x')          # початкові дані
plt.plot(X_line, y_line)   # лінія
plt.xlabel("X")
plt.ylabel("y")
plt.title("Linear Regression Fit")
plt.show()
```



Трохи більше про машинне навчання.

Машинне навчання надає можливість автоматично встановлювати прогнозну модель на основі даних.

Термінологія. Функція втрат: $l : y \times y \to \mathbb{R}$, вимірює різницю між прогнозом і фактичним результатом

$$l(y', y) = (y' - y)^2$$

Канонічна задача оптимізації машинного навчання:

$$minimize_\theta \sum_i l(h_\theta(x^i), y^i)$$

Практично кожен алгоритм машинного навчання має таку форму, просто вкажіть

- Що таке функція гіпотези?
- Що таке функція втрат?
- Як вирішити задачу оптимізації?

Приклади алгоритмів машинного навчання

- Найменші квадрати: лінійна гіпотеза, квадратична втрата, (зазвичай) аналітичне рішення.
- Лінійна регресія: лінійна гіпотеза.
- Машина опорних векторів (support vector machine): {лінійна або ядрова гіпотеза, втрата шарніра, *}
- Нейронна мережа: {складена нелінійна функція, *, (зазвичай) градієнтний спуск)
- Дерево рішень (decision trees): {ієрархічні півплощини, вирівняні по осі, *, жадібна оптимізація}
- Наївний Байєс: {лінійна гіпотеза, спільна ймовірність за певних припущень незалежності, аналітичне рішення}

8. Лінійна Класифікація

- Завдання регресії: прогнозування реальної величини $y \in R$
- Завдання класифікації: прогнозування дискретної величини y
- Бінарна класифікація: $y \in \{-1, 1\}$
- Багатокласова класифікація: $y \in \{1, 2,, ..., k\}$

Приклад лінійної класифікації: класифікація раку молочної залози

Відомий приклад класифікації: використання машинного навчання для діагностики того, чи є пухлина молочної залози доброякісною чи злоякісною [Street et al., 1992]

Умови: лікар бере зразок рідини з пухлини, фарбує клітини, а потім окреслює декілька клітин (обробка зображення уточнює контури)

Система обчислює характеристики кожної клітини, такі як площа, периметр, увігнутість, текстура (всього 10); обчислює середнє/стандартне/максимальне значення для всіх характеристик

```
import numpy as np
import sklearn.datasets
import matplotlib.pyplot as plt
dat = sklearn.datasets.load_breast_cancer()

ix = np.where(dat["feature_names"] == "mean area")[0][0]
iy = np.where(dat["feature_names"] == "mean concave points")[0][0]

def plot_cells():
```

```
    plt.scatter(dat["data"][:,ix][dat["target"]==1], dat["data"][:,iy]
[dat["target"]==1], marker='x', color='C0')
    plt.scatter(dat["data"][:,ix][dat["target"]==0], dat["data"][:,iy]
[dat["target"]==0], marker='+', color='C3')
    plt.xlim([0,2600])
    plt.ylim([0,0.21])
    plt.xlabel("Mean Area")
    plt.ylabel("Mean Concave Points")
    plt.legend(['Benign', 'Malignant'])

plot_cells()
```

```
```

```
```

9. Нелінійні Алгоритми Машинного Навчання

До цього моменту ми ілюстрували лінійну регресію як «проведення лінії через дані», але насправді це було функцією наших вхідних характеристик.

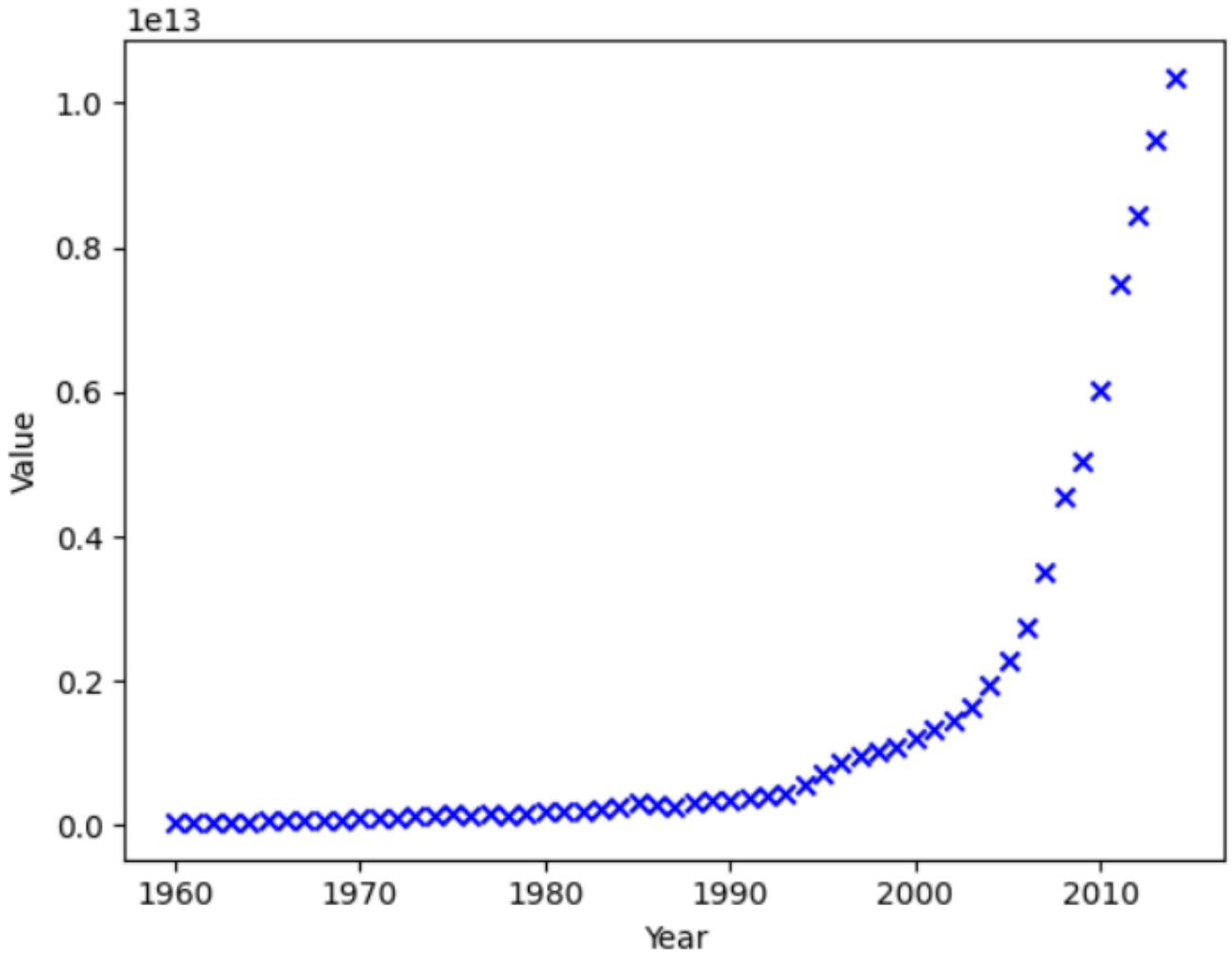
Хоча це може здаватися обмеженим, алгоритми лінійної регресії є досить потужними, коли застосовуються до нелінійних характеристик вхідних даних.

Розглянемо, для прикладу, дані про ріст ВВП Китаю за останні 60 років, і спробуємо побудувати модель, що дає найкраще передбачення.

```
import matplotlib.pyplot as plt
import pandas as pd

def build_plot(file_path, x, y):
    data = pd.read_csv(file_path)
    plt.scatter(data[x], data[y], c='blue', marker='x')
    plt.xlabel(x)
    plt.ylabel(y)
    return plt
    # plt.show()

build_plot('../resources/china_gdp.csv', 'Year', 'Value').show()
```



Якщо спробувати провести лінію через цей графік, отримаємо наступне:

```
plt = build_plot('../resources/china_gdp.csv', 'Year', 'Value')
data = pd.read_csv('../resources/china_gdp.csv')
x, y = data['Year'], data['Value']
x_reshaped, y_reshaped = x.values.reshape(-1, 1), y.values.reshape(-1, 1)

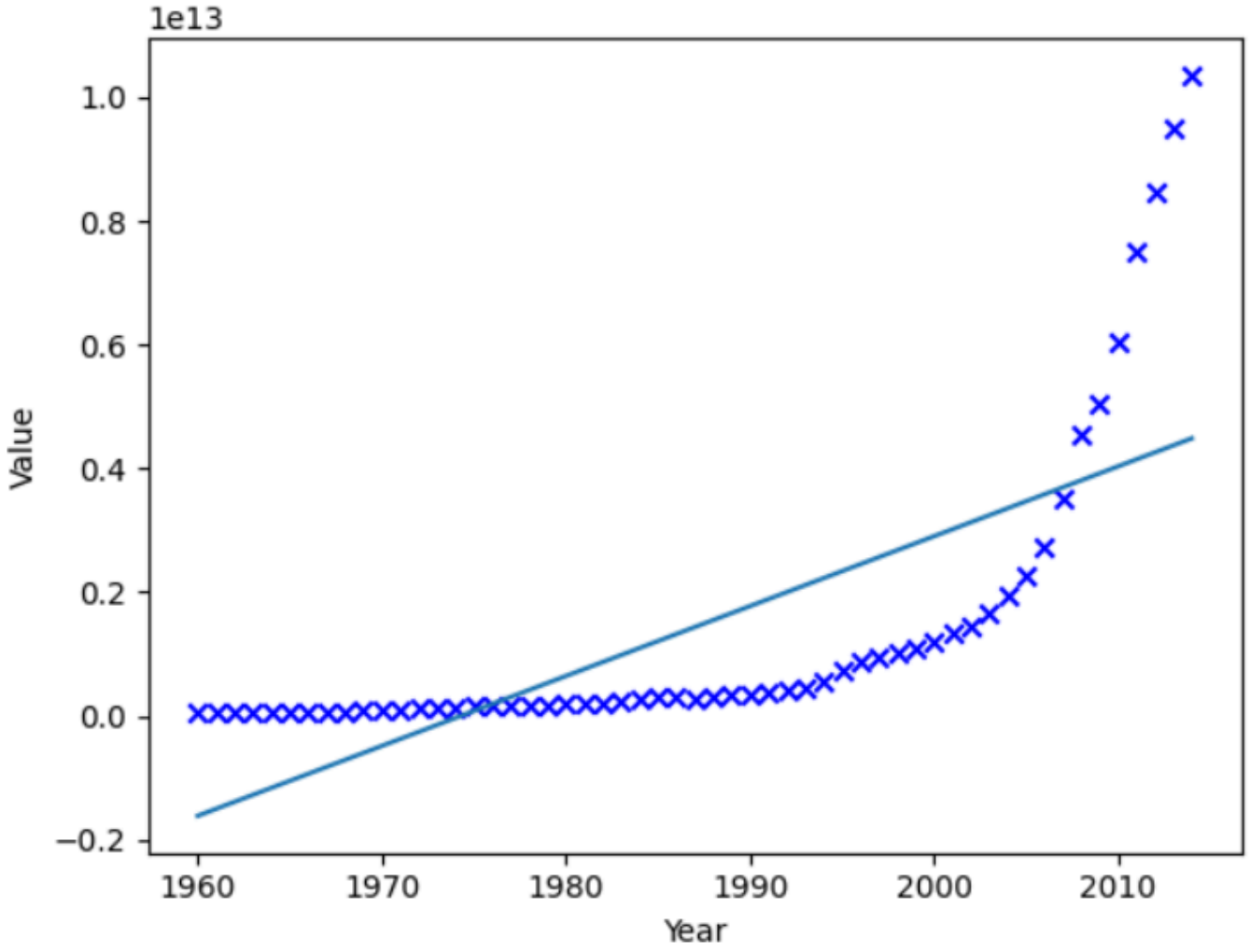
from sklearn.linear_model import LinearRegression
import numpy as np

model = LinearRegression(fit_intercept=True)
model.fit(x_reshaped, y_reshaped)
Xnew = np.array([[2025], [2030]])
print(model.predict(Xnew))

theta = [ model.coef_[0], model.intercept_ ]
print(theta)

X_line = np.linspace(x_reshaped.min(), x_reshaped.max(), 100).reshape(-1, 1)
y_line = model.predict(X_line)

plt.plot(X_line, y_line)
```



Очевидно, що лінійна модель не найкращим чином наближає ці дані.

Поліноміальна Регресія

Поліноміальна регресія — це різновид лінійної регресії, в якому взаємозв'язок між x та y моделюється у вигляді полінома n-го ступеня:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + ... + \beta_n x^n + \epsilon$$

Це дозволяє моделювати нелінійні залежності шляхом додавання поліноміальних членів ознак.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

plt = build_plot('../resources/china_gdp.csv', 'Year', 'Value')
data = pd.read_csv('../resources/china_gdp.csv')
x, y = data['Year'], data['Value']
x_reshaped, y_reshaped = x.values.reshape(-1, 1), y.values.reshape(-1, 1)

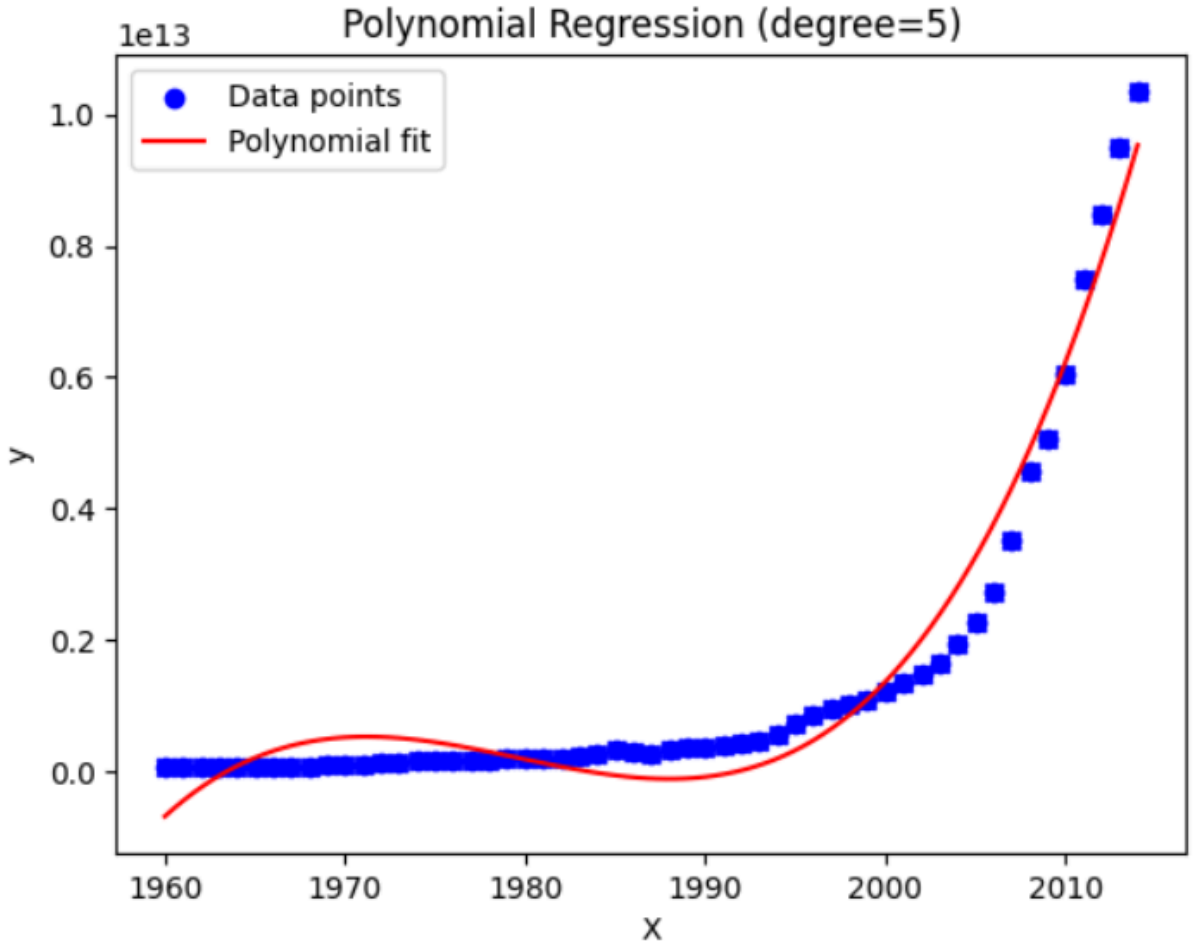
# трансформуємо вхідні ознаки в поліноміальні
poly = PolynomialFeatures(degree=5)
X_poly = poly.fit_transform(x_reshaped)
```

```
# підбираємо під модель
model = LinearRegression()
model.fit(X_poly, y_reshaped)

# проводимо передбачення та візуалізуємо
X_plot = np.linspace(min(x_reshaped), max(x_reshaped), 100).reshape(-1, 1)
y_pred = model.predict(poly.transform(X_plot))

plt.scatter(x, y, color='blue', label='Data points')
plt.plot(X_plot, y_pred, color='red', label='Polynomial fit')
plt.xlabel("X")
plt.ylabel("y")
plt.title("Polynomial Regression (degree=5)")
plt.legend()
plt.show()

# coefficients
print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)
```




```
Intercept: [-2.1028827e+17]
Coefficients: [[ 0.00000000e+00  1.20746941e+03  2.72901110e+05  2.71121389e+08
  -2.05442021e+05  4.15121802e+01]]
```

Узагальнення та перенавчання

Проблема з канонічною задачею машинного навчання полягає в тому, що нас насправді не цікавить мінімізація цієї цілі на заданому наборі даних.

Нас насправді цікавить, наскільки добре наша функція узагальниться на нових прикладах, які ми не використовували для навчання системи (але які взяті з «того самого розподілу», що й приклади, які ми використовували для навчання)

Поліноми вищого ступеня демонстрували перенавчання: вони фактично мають дуже низькі втрати на даних навчання, але створюють функції, які, як ми очікуємо, не будуть добре узагальнюватися.

У міру ускладнення моделі втрати навчання завжди зменшуються; втрати узагальнення зменшуються до певної точки, а потім починають збільшуватися

Хоча важко кількісно оцінити справжню помилку узагальнення (тобто помилку цих алгоритмів щодо повного розподілу можливих прикладів), ми можемо наблизити її за допомогою *перехресної валідації*

Основна ідея полягає в тому, щоб розділити набір даних на навчальний набір і набір для перевірки. Навчіть алгоритм на навчальному наборі і оцініть на наборі для перевірки.

Ми називаємо змінні θ *параметрами* алгоритму машинного навчання. Але є й інші величини, які також впливають на класифікатор: ступінь полінома, кількість регуляризації тощо; їх сукупно називають *гіперпараметрами* алгоритму.

Основна ідея перехресного валідації: використовувати навчальний набір для визначення параметрів, використовувати резервний набір для визначення гіперпараметрів.

k-кратна перехресна валідація (k-folds validation):

Більш складна (але насправді дещо більш поширена) версія перехресної валідації. Розділіть набір даних на k роз'єднаних підмножин (складок); навчіть на $k-1$ і оцініть на залишковій складці; повторіть k разів, утримуючи кожну складку один раз. Повідомте середню похибку для всіх утриманих складок.

Варіанти

Перехресна валідація «залиш один»: обмеження k-кратної перехресної валідації, де кожна складка є лише одним прикладом (тож ми навчаємося на всіх інших прикладах, тестуючи цей один приклад) [Дещо дивно, але для найменших квадратів це можна обчислити ефективніше, ніж k-кратну перехресну валідацію, з такою ж складністю вирішення для оптимального θ за допомогою матричного рівняння]

Стратифікована перехресна валідація: збереження приблизно рівного відсотка позитивних/негативних прикладів (або будь-якої іншої ознаки) у кожному згині

Попередження: k-кратна перехресна валідація не завжди є кращою (наприклад, у прогнозуванні часових рядів бажано, щоб набір утриманих даних був повністю сформований після набору навчання)

Поліноміальна класифікація.

Наведемо приклад поліномільної класифікації на тому ж датасеті пухлин молочної залози.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score

# Завантажуємо датасет (лише дві ознаки для класифікації)
data = load_breast_cancer()
X = data.data[:, [3, 25]]  # середній радіус, середній периметр
y = data.target

# Розбиття на тренувальні та тестові дані
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Скейлимо дані
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# Додаємо поліноміальні ознаки (нелінійна трансформація)
poly = PolynomialFeatures(degree=10)
X_train_poly = poly.fit_transform(X_train_scaled)
X_test_poly = poly.transform(X_test_scaled)

# Тренуємо логістичну регресію на поліноміальних ознаках
clf = LogisticRegression(max_iter=10000)
clf.fit(X_train_poly, y_train)

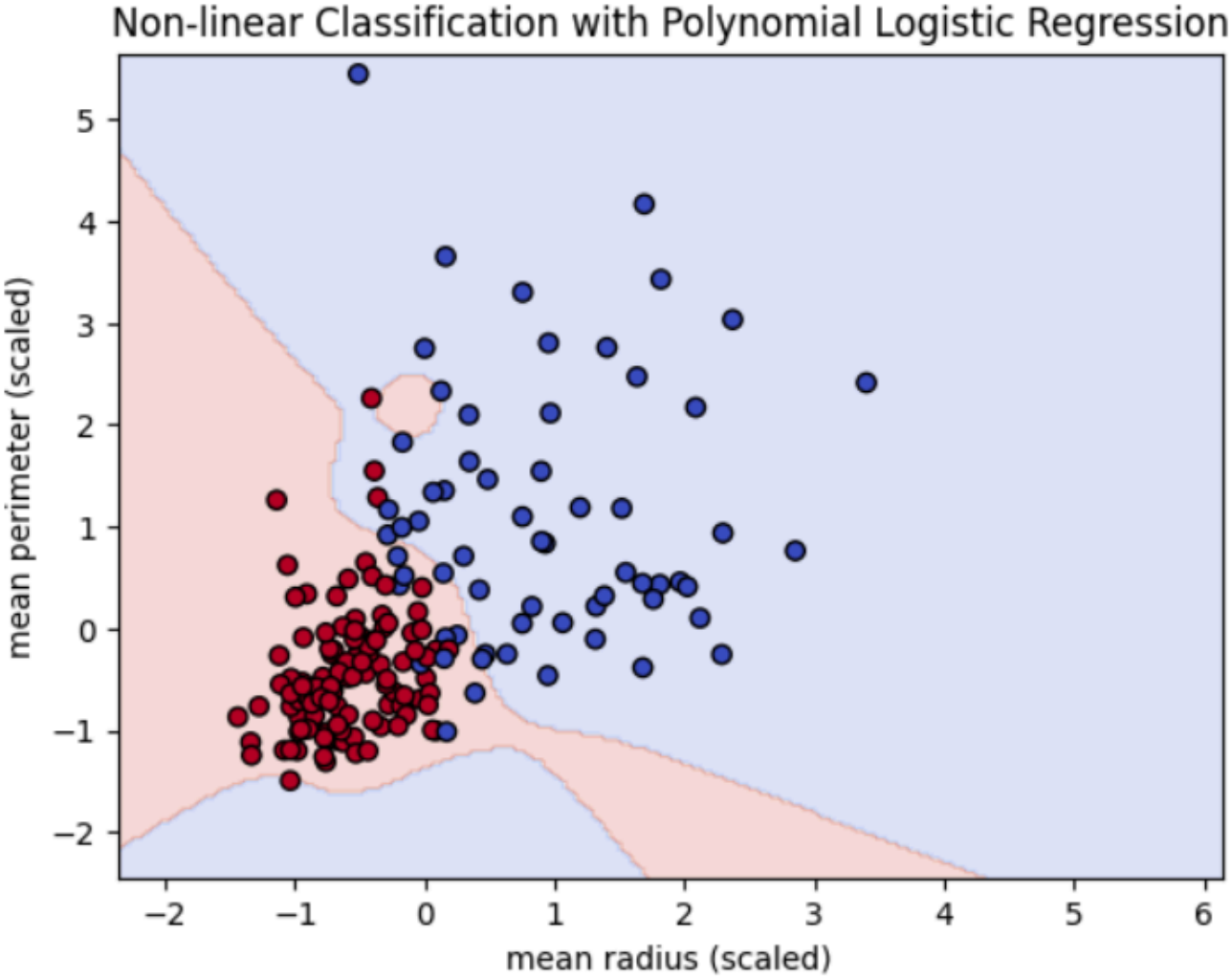
# Евалюємо
y_pred = clf.predict(X_test_poly)
print("Accuracy:", accuracy_score(y_test, y_pred))

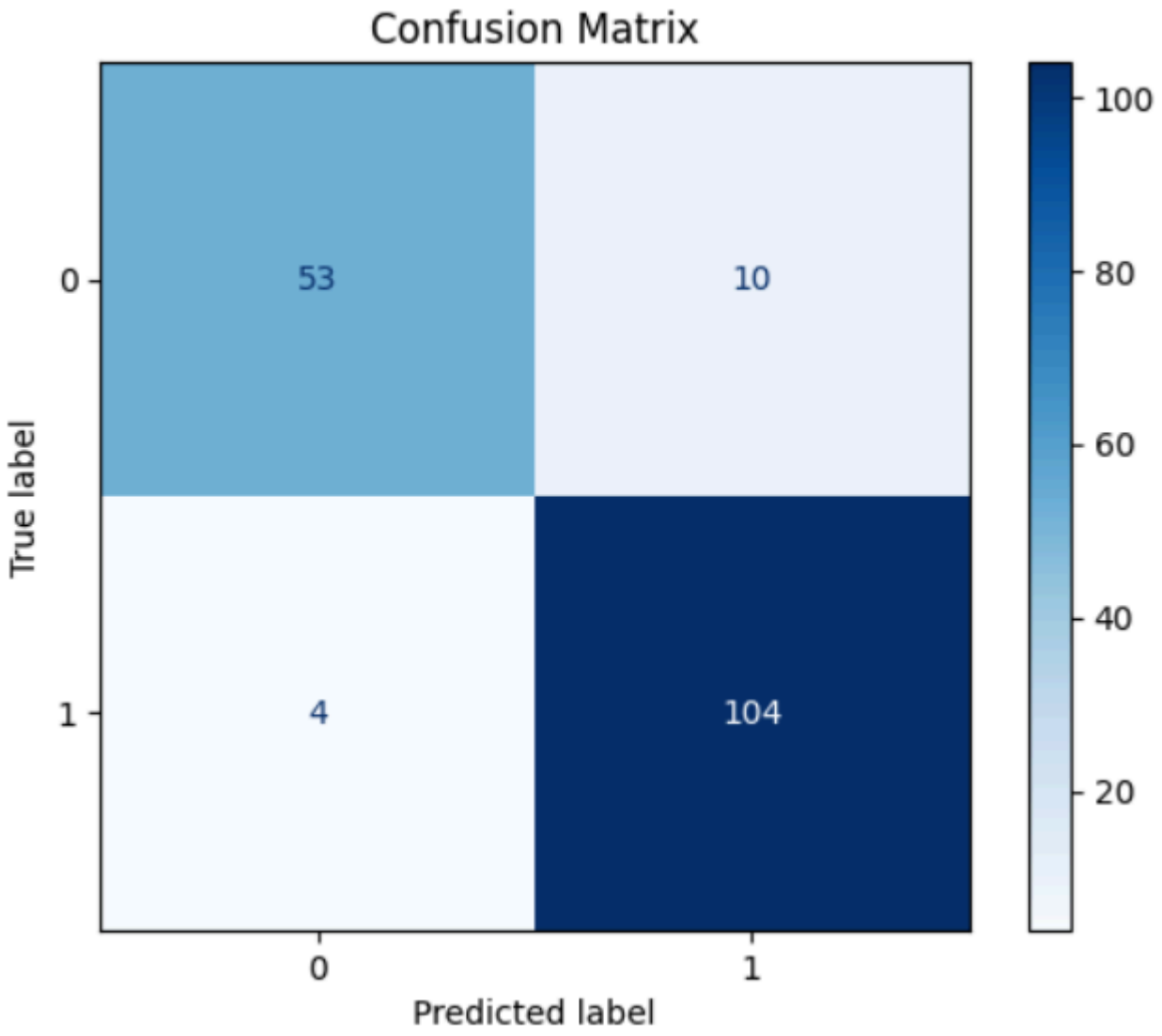
# Візуалізація (Межа між класами)
x_min, x_max = X_train_scaled[:, 0].min() - 1, X_train_scaled[:, 0].max() + 1
y_min, y_max = X_train_scaled[:, 1].min() - 1, X_train_scaled[:, 1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200),
                     np.linspace(y_min, y_max, 200))

Z = clf.predict(poly.transform(np.c_[xx.ravel(), yy.ravel()]))
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.2, cmap='coolwarm')
plt.scatter(X_test_scaled[:, 0], X_test_scaled[:, 1], c=y_test, cmap='coolwarm',
edgecolors='k')
plt.xlabel('mean radius (scaled)')
plt.ylabel('mean perimeter (scaled)')
plt.title('Non-linear Classification with Polynomial Logistic Regression')
plt.show()

# Матриця похибок (confusion matrix)
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, cmap='Blues')
plt.title("Confusion Matrix")
plt.show()
```


Non-linear Classification with Polynomial Logistic Regression
mean perimeter (scaled)
mean radius (scaled)
5
4
3
2
1
0
−1
−2
−2
−1
0
1
2
3
4
5
6



Метод Опорних Векторів (Support Vector Machine)

Метод опорних векторів — це алгоритм класифікації, який шукає оптимальну гіперплощину, що розділяє дані двох класів із максимальним зазором (margin).

Нехай ми маємо навчальну вибірку:

$$(x_i, y_i)_{i=1}^n, \quad x_i \in \mathbb{R}^d, \quad y_i \in -1, +1$$

Ми шукаємо гіперплощину, яка має вигляд:

$$w^T x + b = 0$$

де

- w — вектор ваг (нормаль до гіперплощини),
- b — зсув (bias).

Умова коректної класифікації: Для всіх навчальних точок повинно виконуватись:

$$y_i(w^T x_i + b) \geq 1$$

Це означає, що всі точки знаходяться по правильний бік гіперплощини, причому на відстані не менше 1 від неї.

Відстань (margin): Відстань від точки до гіперплощини дорівнює:

$$\frac{|w^T x + b|}{|w|}$$

Ми хочемо максимізувати цей зазор між класами. Для двох крайніх точок зазор дорівнює:

$$\text{margin} = \frac{2}{|w|}$$

Отже, щоб його максимізувати, потрібно мінімізувати $|w|^2$.

Оптимізаційна задача (лінійно роздільний випадок):

$$\min_{w,b} \quad \frac{1}{2}|w|^2 \text{ за умови:} \qquad y_i(w^T x_i + b) \geq 1, \quad i = 1, \ldots, n$$

Це *опукла квадратична задача оптимізації*.

Нероздільні дані — м'який зазор: У реальних даних можливі помилки класифікації, тому вводять змінні послаблення $\xi_i \geq 0$:

$$\min_{w,b,\xi} \quad \frac{1}{2}|w|^2 + C\sum_{i=1}^{n} \xi_i \text{ за умови:} \qquad y_i(w^T x_i + b) \geq 1 - \xi_i, \quad i = 1, \ldots, n$$

де $C > 0$ — параметр, що задає компроміс між шириною зазору та кількістю помилок.

Ядровий трюк (kernel trick): Уявимо собі, що перед нами стоїть така задача: потрібно провести класифікацію температури на комфортну та некомфортну для людини. Якщо для людини комфортною температурою є 20 градусів за Цельсієм, то некомфортною є як -10 так і 40 градусів. Провести таку гіперплощину через ці дані, щоб всі точки з комфортною температурою були з однієї сторони, є неможливим.

Для того, щоб провести гіперплощину в цьому випадку, ми можемо застосувати перетворення $y(x) = (x - 20)^2$. Тоді всі некомфортні температури переходять у значення, більші за якесь a, тоді як всі комфортні температури відображаються у точки $x \leq a$. Це і є ядровий трюк.

Щоб працювати з нелінійно роздільними даними, SVM використовує перетворення простору:

$$\phi : \mathbb{R}^d \to \mathbb{R}^m$$

але замість явного обчислення $\phi(x)$ застосовується ядро $K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$.

Типові ядра:

- Лінійне: $K(x, x') = x^T x'$
- Поліноміальне: $K(x, x') = (x^T x' + 1)^d$
- Радіально-базисне (RBF): $K(x, x') = \exp(-\gamma |x - x'|^2)$ - використовується найчастіше.

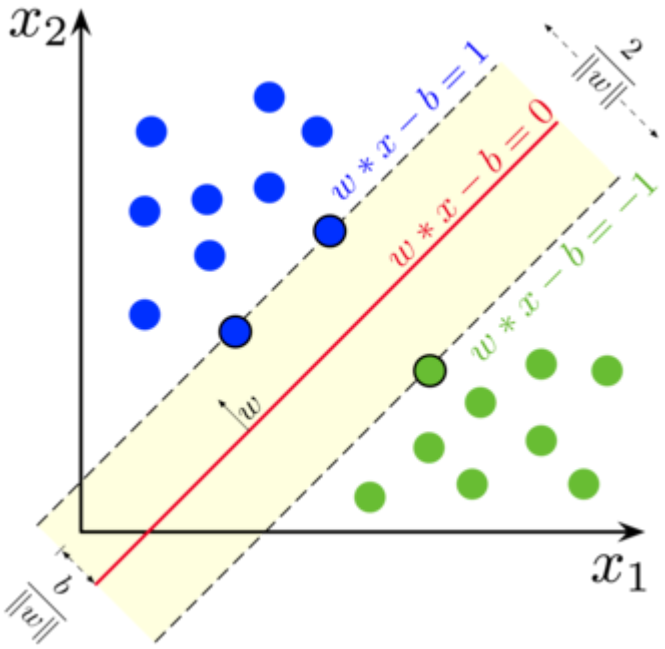
Рішення у вигляді опорних векторів: У результаті оптимізації маємо:

$$f(x) = \text{sign}\left(\sum_{i=1}^{n} \alpha_i y_i K(x_i, x) + b\right)$$

де α_i — коефіцієнти лагранжа. Точки, для яких $\alpha_i > 0$, називаються *опорними векторами* — саме вони визначають гіперплощину.

Геометрична інтерпретація

- SVM шукає межу, яка найбільше віддалена від найближчих точок обох класів.
- Тільки кілька опорних векторів визначають положення цієї межі.




```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score

# Завантажуємо датасет (лише дві ознаки для класифікації)
data = load_breast_cancer()
X = data.data[:, [3, 25]]  # середній радіус, середній периметр
y = data.target

# Розбиття на тренувальні та тестові дані
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Скейлимо дані
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Тренуємо модель SVM із RBF (радіально-базисним) ядром
clf = SVC(kernel='rbf', gamma='scale', C=1.0)
clf.fit(X_train_scaled, y_train)
```

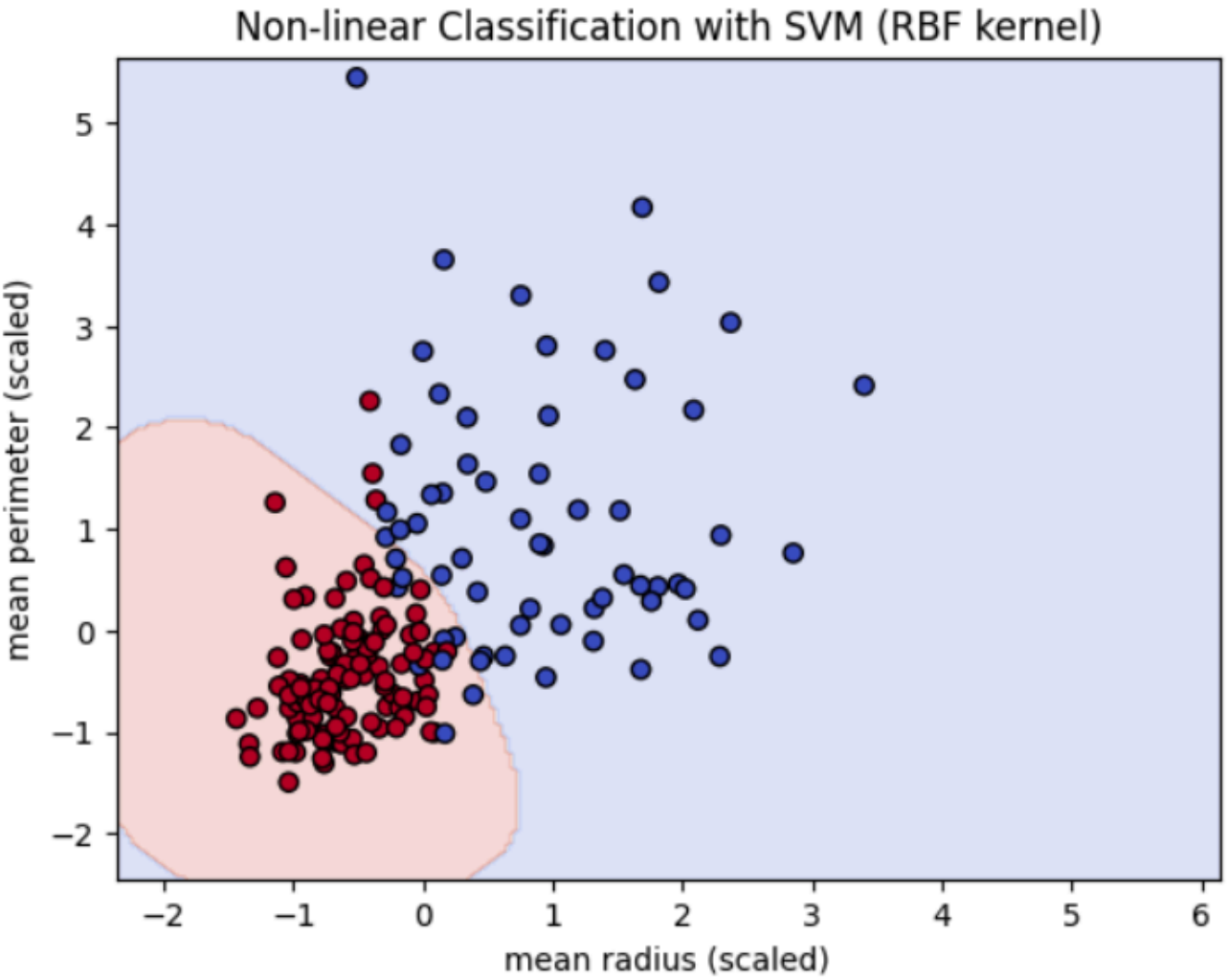
```
# Евалюємо
y_pred = clf.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))

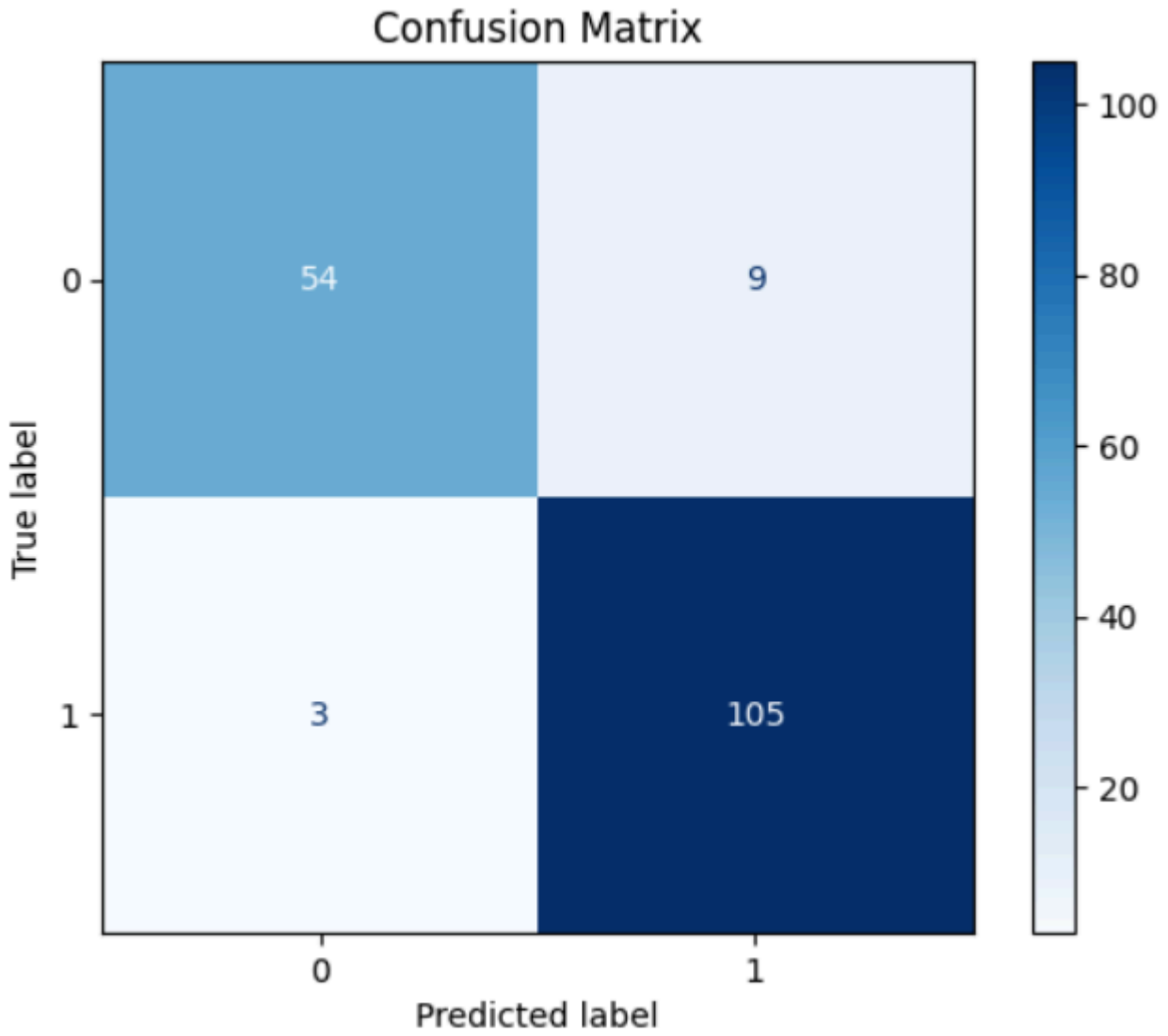
# Візуалізація (Межа між класами)
x_min, x_max = X_train_scaled[:, 0].min() - 1, X_train_scaled[:, 0].max() + 1
y_min, y_max = X_train_scaled[:, 1].min() - 1, X_train_scaled[:, 1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200),
                     np.linspace(y_min, y_max, 200))

Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.2, cmap='coolwarm')
plt.scatter(X_test_scaled[:, 0], X_test_scaled[:, 1], c=y_test, cmap='coolwarm',
edgecolors='k')
plt.xlabel('mean radius (scaled)')
plt.ylabel('mean perimeter (scaled)')
plt.title('Non-linear Classification with SVM (RBF kernel)')
plt.show()

# Матриця похибок (confusion matrix)
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, cmap='Blues')
plt.title("Confusion Matrix")
plt.show()
```


Non-linear Classification with SVM (RBF kernel)
mean perimeter (scaled)
mean radius (scaled)
5
4
3
2
1
0
−1
−2
−2
−1
0
1
2
3
4
5
6



Наївний баєсів класифікатор (Naive Bayes classifier)

Наївний баєсівський класифікатор — це ймовірнісний метод класифікації, який базується на теоремі Байєса. Він оцінює, до якого класу належить об'єкт x, обчислюючи ймовірність $P(C_k|x)$ для кожного класу C_k.

Теорема Байєса: основна формула

$$P(C_k|x) = \frac{P(x|C_k), P(C_k)}{P(x)}$$

де:

- $P(C_k|x)$ — апостеріорна ймовірність класу після спостереження ознак;
- $P(x|C_k)$ — правдоподібність (як імовірно отримати ознаки, якщо відомий клас);
- $P(C_k)$ — апріорна ймовірність класу;
- $P(x)$ — нормувальний коефіцієнт (спільна ймовірність ознак).

"*Наївне*" *припущення незалежності*: Метод вважає, що всі ознаки $x_1, x_2, \ldots, x_n$ незалежні одна від одної, якщо відомо, до якого класу належить об'єкт.

$$P(x|C_k) = \prod_{j=1}^{n} P(x_j|C_k)$$

Це "*наївне*" *припущення* значно спрощує обчислення, навіть якщо насправді ознаки частково залежать.

Формула класифікації: Підставимо у формулу Байєса:

$$P(C_k|x) \propto P(C_k) \prod_{j=1}^{n} P(x_j|C_k)$$

Оскільки $P(x)$ однакова для всіх класів, її можна не враховувати при порівнянні.

Отже, ми вибираємо клас з максимальною апостеріорною ймовірністю:

$$\hat{C} = \arg\max_{C_k}; P(C_k) \prod_{j=1}^{n} P(x_j|C_k)$$

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score

# Завантажуємо датасет (лише дві ознаки для класифікації)
data = load_breast_cancer()
X = data.data[:, [3, 25]]  # середній радіус, середній периметр
y = data.target

# Розбиття на тренувальні та тестові дані
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Скейлимо дані
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Тренуємо Gaussian Naive Bayes модель
clf = GaussianNB()
clf.fit(X_train_scaled, y_train)

# Евалюємо
y_pred = clf.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))

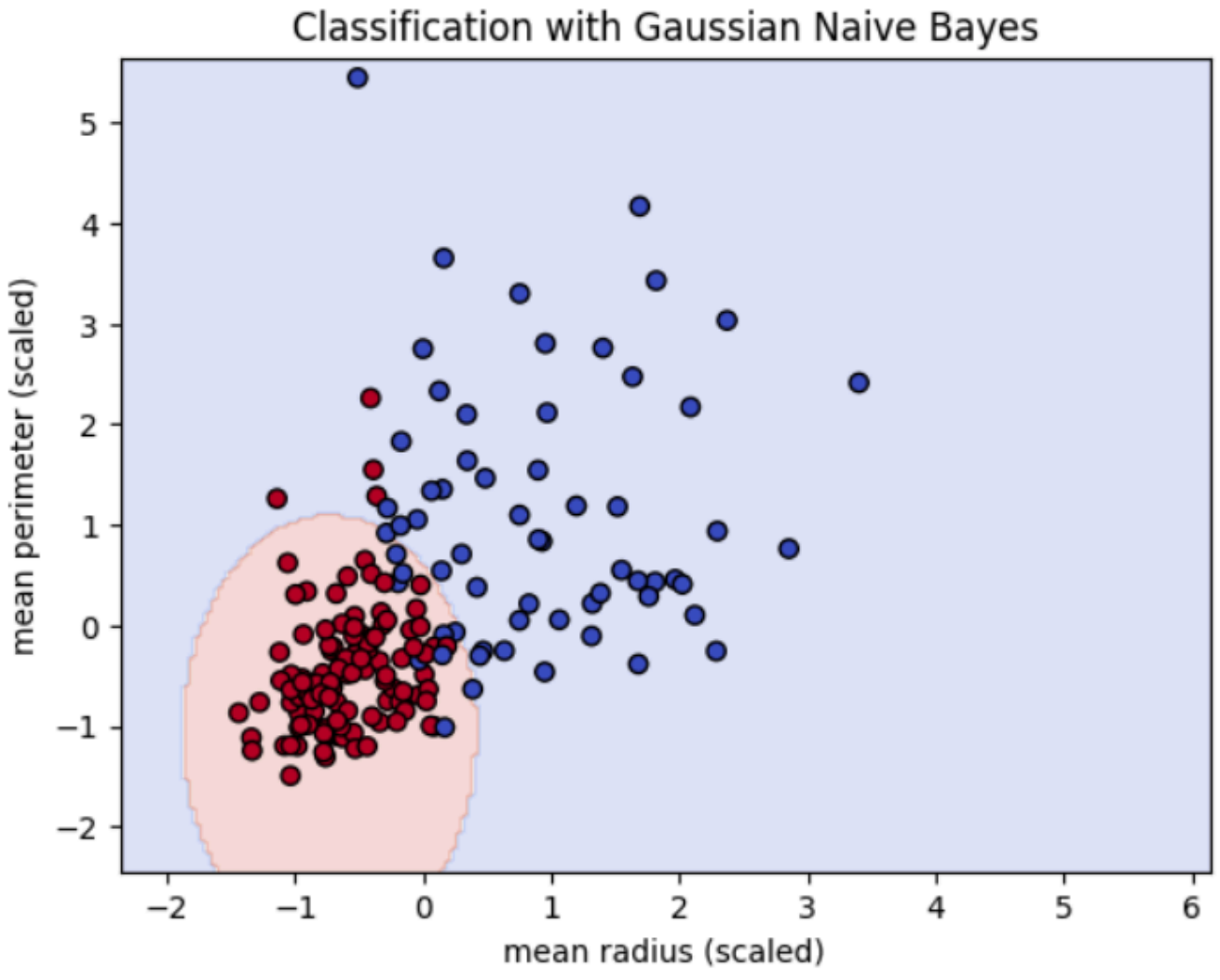
# Візуалізація (Межа між класами)
x_min, x_max = X_train_scaled[:, 0].min() - 1, X_train_scaled[:, 0].max() + 1
y_min, y_max = X_train_scaled[:, 1].min() - 1, X_train_scaled[:, 1].max() + 1
```

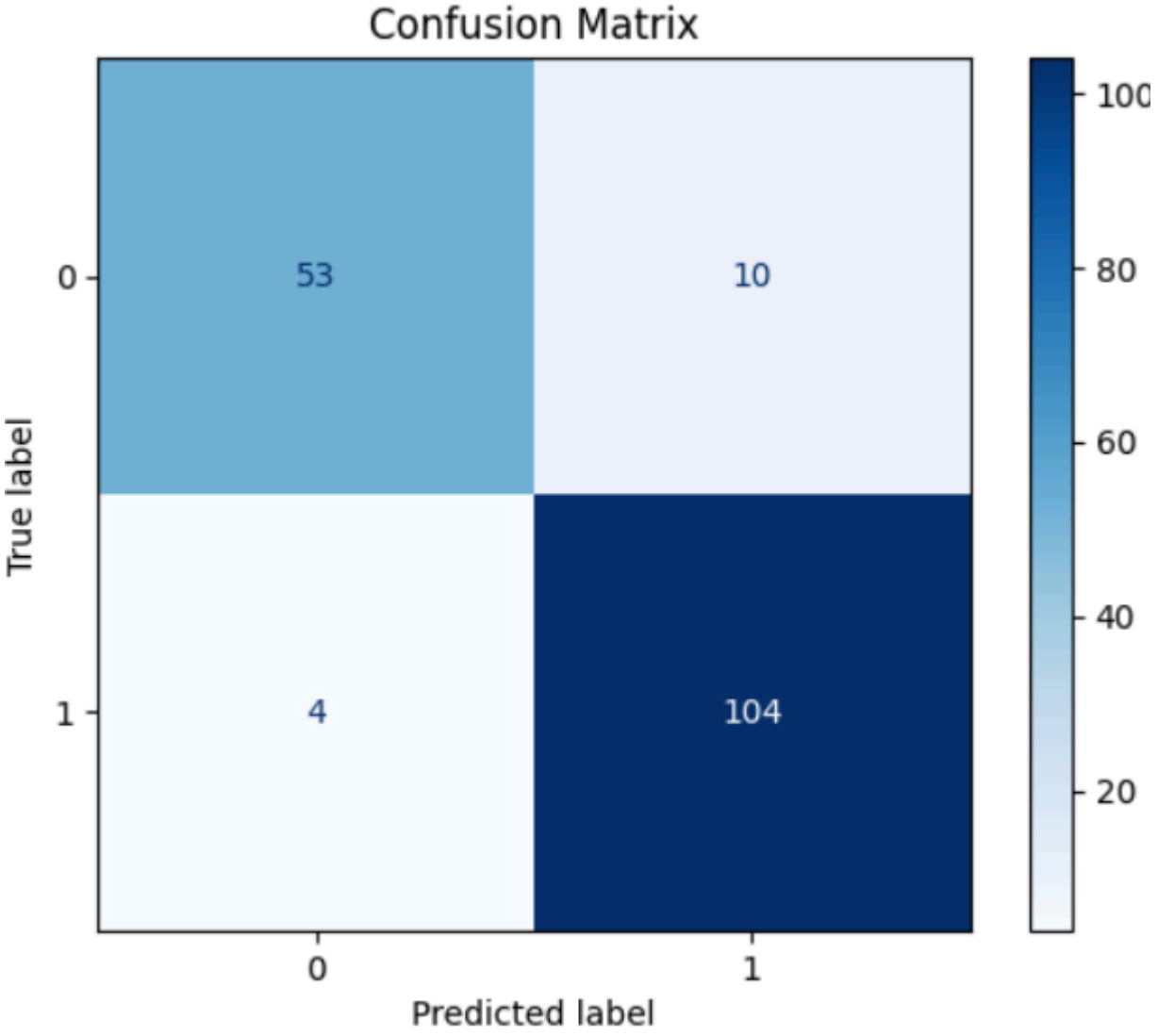
```
xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 200),
    np.linspace(y_min, y_max, 200)
)

Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.2, cmap='coolwarm')
plt.scatter(X_test_scaled[:, 0], X_test_scaled[:, 1], c=y_test, cmap='coolwarm',
edgecolors='k')
plt.xlabel('mean radius (scaled)')
plt.ylabel('mean perimeter (scaled)')
plt.title('Classification with Gaussian Naive Bayes')
plt.show()

# Матриця похибок (confusion matrix)
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, cmap='Blues')
plt.title("Confusion Matrix")
plt.show()
```





Дерева прийняття рішень (Decision trees)

Алгоритм дерева прийняття рішень використовується для класифікації або регресії. Його ідея — розділяти простір ознак на підмножини так, щоб отримати групи об'єктів, максимально однорідні за цільовою змінною.

Дерево прийняття рішень — це ієрархічна структура, де:

- вузол (node) містить умову розділення (наприклад, $x_i < t$)
- гілки (branches) — це варіанти переходу (так/ні);
- листки (leaves) — фінальні рішення (клас або числове значення)

Маємо навчальну вибірку:

$$(x_i, y_i)_{i=1}^{n}, \quad x_i \in \mathbb{R}^d, ; y_i \in 1, 2, \ldots, K$$

Мета — знайти функцію $f(x)$, яка розбиває простір ознак на області R_m, і кожній області відповідає передбачення класу:

$$f(x) = \sum_{m=1}^{M} c_m \cdot I(x \in R_m)$$

де:

- R_m — регіон (листок дерева);
- c_m — значення або клас, який передбачається для цього регіону;
- $I(\cdot)$ — індикаторна функція (1, якщо умова виконується).

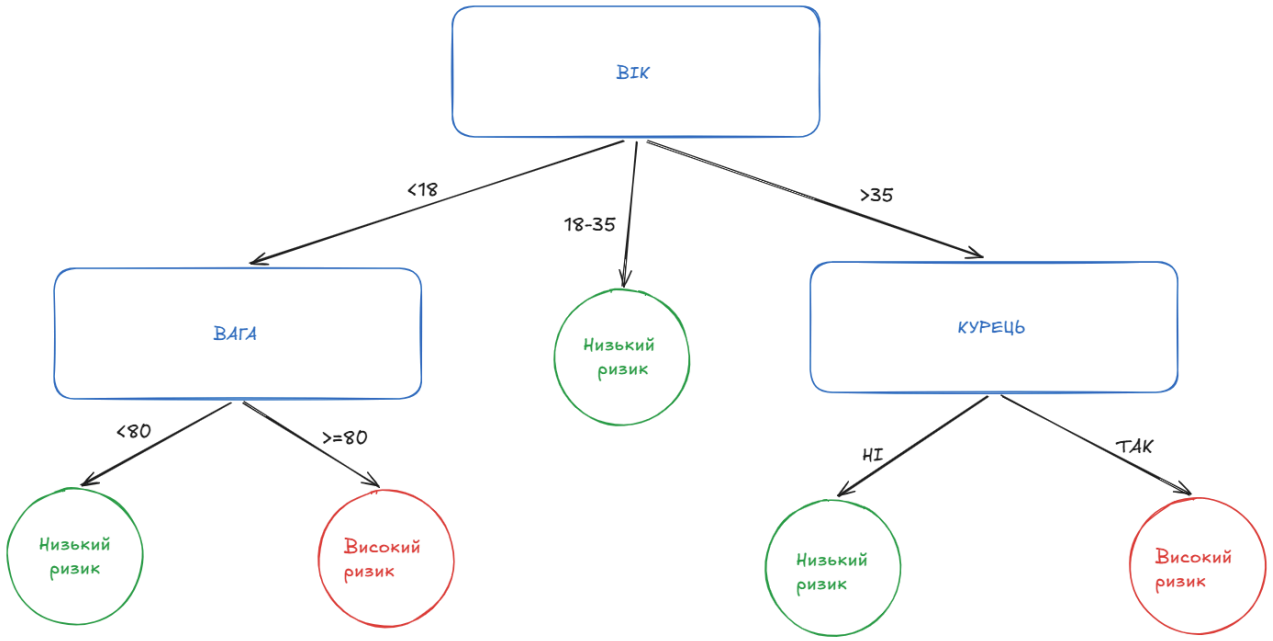
На кожному кроці вибирається ознака x_j та поріг t, які найкраще поділяють дані на дві частини:

$$R_1(j,t) = x \mid x_j \leq t, \quad R_2(j,t) = x \mid x_j > t$$

Ми вибираємо такі j, t, що мінімізують певну функцію втрат L.

Тут можна навести приклад із класифікацією пацієнтів за ризиком серцевих захворювань. Нехай перед нами стоїть задача класифікувати пацієнтів таким чином. Тут логічно будувати дерево, в якому на вузлах пацієнти будуть розділятися за віком (вік, більший за певне число - підвищений ризик), вагою (вага, більша за певне число - також ризик захворювань), та схильністю до куріння. Питання лише в підборі конкретних чисел (гранична вага та вік) таким чином, щоб мінімізувати ентропію (пацієнти, що будуть неправильно класифіковані).

Як бачимо, дерево прийняття рішень є цілком інтуїтивно зрозумілим алгоритмом, а значною перевагою цієї моделі є її інтерпретованість.



Приклад з використанням sklearn:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split

# Завантажуємо датасет
data = load_breast_cancer()
```

```
# Вибираємо тільки дві ознаки: під індексом 3 та 25
X = data.data[:, [3, 25]]
y = data.target

# Розділяємо дані на тренувальні та тестові
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Навчаємо модель
clf = DecisionTreeClassifier(max_depth=5, criterion='gini', random_state=42)
clf.fit(X_train, y_train)

# Точність
print("Accuracy on test set:", clf.score(X_test, y_test))

# Будуємо межу розподілу класів
x_min, x_max = X[:, 0].min() - 0.01, X[:, 0].max() + 0.01
y_min, y_max = X[:, 1].min() - 0.01, X[:, 1].max() + 0.01
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
                     np.linspace(y_min, y_max, 300))

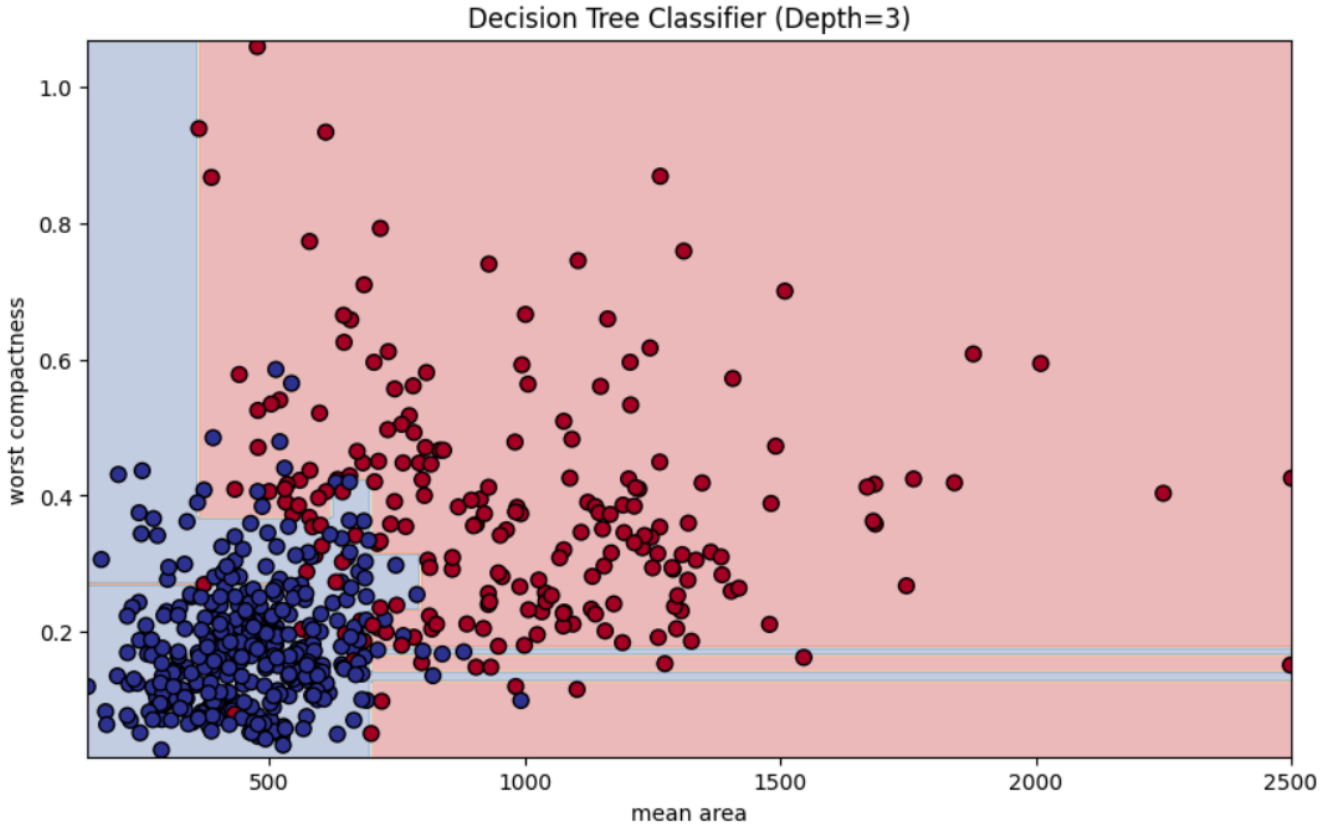
Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

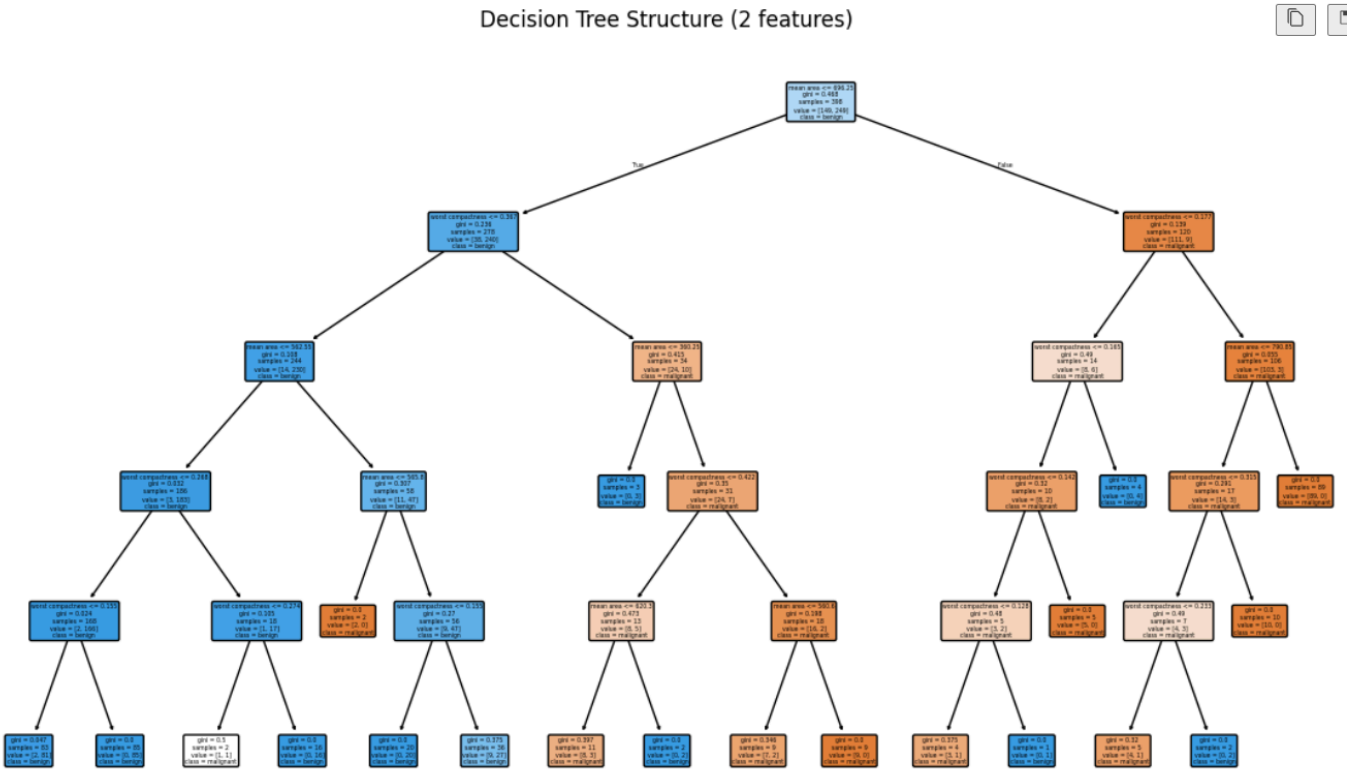
plt.figure(figsize=(10, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.RdYlBu)
scatter = plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.RdYlBu, edgecolor='k',
s=50)

plt.xlabel(data.feature_names[3])
plt.ylabel(data.feature_names[25])
plt.title("Decision Tree Classifier (Depth=3)")
# plt.legend(handles=scatter.legend_elements()[0], labels=data.target_names)
plt.show()

# Будуємо дерево прийняття рішень
plt.figure(figsize=(14, 8))
plot_tree(clf, feature_names=[data.feature_names[3], data.feature_names[25]],
          class_names=data.target_names, filled=True, rounded=True)
plt.title("Decision Tree Structure (2 features)")
plt.show()
```

```
Accuracy on test set: 0.9298245614035088
```





10. Ансамблеве навчання

Якщо ми використовуємо для прогнозування лише одну модель, вона може мати доволі таки високу варіативність - будь-яка зміна вхідних даних буде призводити до іншого результату. Саме тому в нагоді стає ідея використання набору моделей.

Ансамблеве навчання — техніка машинного навчання, що використовує кілька навчених алгоритмів з метою отримати кращу передбачальну ефективність, ніж можна отримати від кожного алгоритму окремо. На відміну від статистичного ансамблю в статистичній механіці, який зазвичай нескінченний, ансамбль моделей у машинному навчанні складається з конкретної скінченної множини альтернативних моделей, але зазвичай дозволяє існувати істотно гнучкішим структурам.

Ідея ансамблів полягає в тому, що замість того, щоб покладатися на одну "думку", ми беремо багато моделей і об'єднуємо їхні прогнози. Тут можна навести аналогію з людьми - уявіть, що потрібно оцінити кількість монет в прозорій банці (нам відомо, що цих монет там 100). Одна людина може дуже сильно помилитися і назвати, наприклад, число `400`. Але якщо запитати 100 людей: `400, 50, 150, 70, 200, 40, ...`, то середнє значення часто виявляється набагато ближчим до істини.

Випадковий ліс

Випадковий ліс — це ансамблевий метод класифікації або регресії, який об’єднує велику кількість дерев рішень, кожне з яких навчається на випадковій підмножині даних і ознак. Ідея полягає в тому, щоб зменшити перенавчання (overfitting), властиве окремим деревам, і отримати більш стабільну та точну модель.

Нехай маємо навчальну вибірку:

$$D = (x_i, y_i)_{i=1}^{N}, \quad x_i \in \mathbb{R}^m,; y_i \in 1, 2, \ldots, K$$

Алгоритм випадкового лісу створює T незалежних дерев рішень:

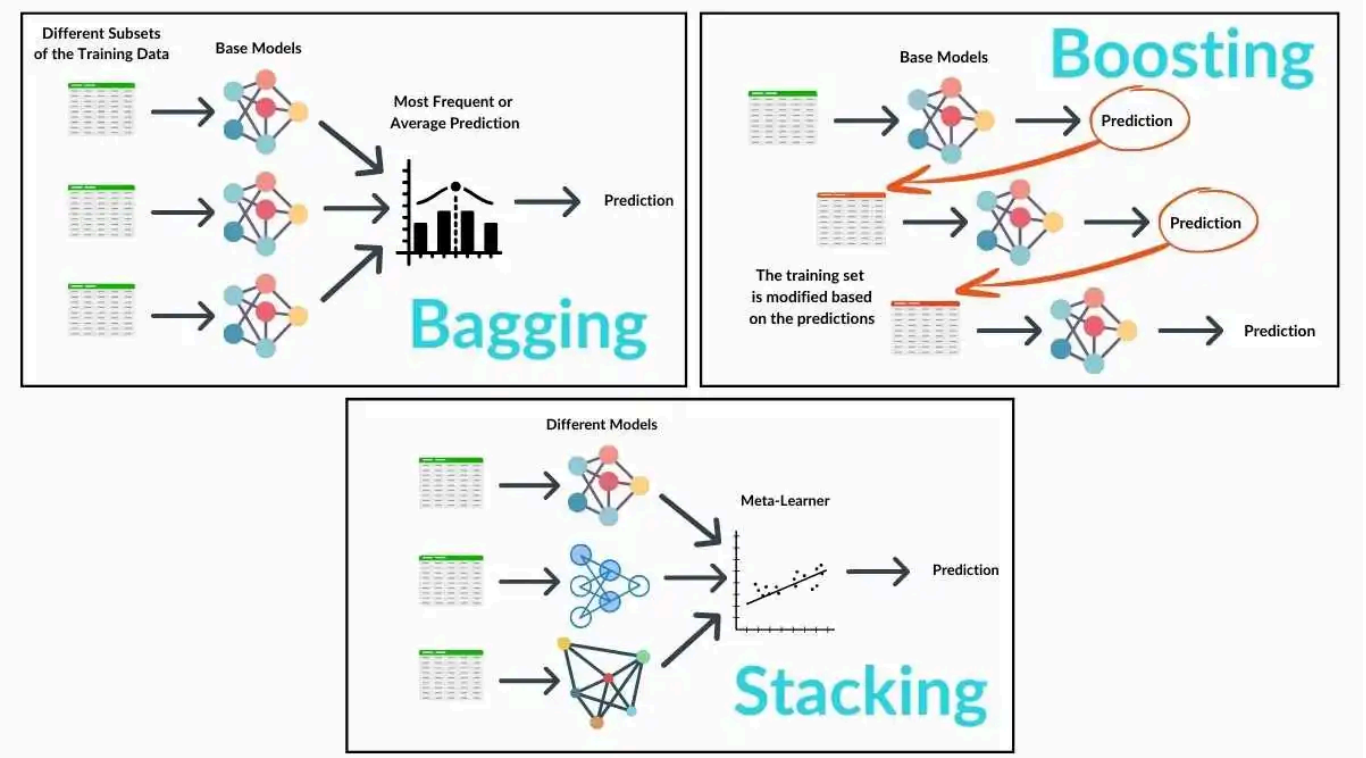
$$\mathrm{RandomForest} = h_1(x), h_2(x), \ldots, h_T(x)$$

Після побудови всіх дерев $h_t(x)$, прогноз робиться голосуванням (для класифікації):

$$\hat{y} = \mathrm{mode} h_1(x), h_2(x), \ldots, h_T(x)$$

або усередненням (для регресії):

$$\hat{y} = \frac{1}{T} \sum_{t=1}^{T} h_t(x)$$




```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score

# Завантажуємо датасет
data = load_breast_cancer()
X = data.data[:, [3, 25]]
y = data.target

# Розподіл на тренувальні та тестові дані
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Скейлимо ознаки
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Тренуємо Random Forest
clf = RandomForestClassifier(
    n_estimators=100, max_depth=5, random_state=42
)
clf.fit(X_train_scaled, y_train)

# Оцінка моделі
y_pred = clf.predict(X_test_scaled)
```

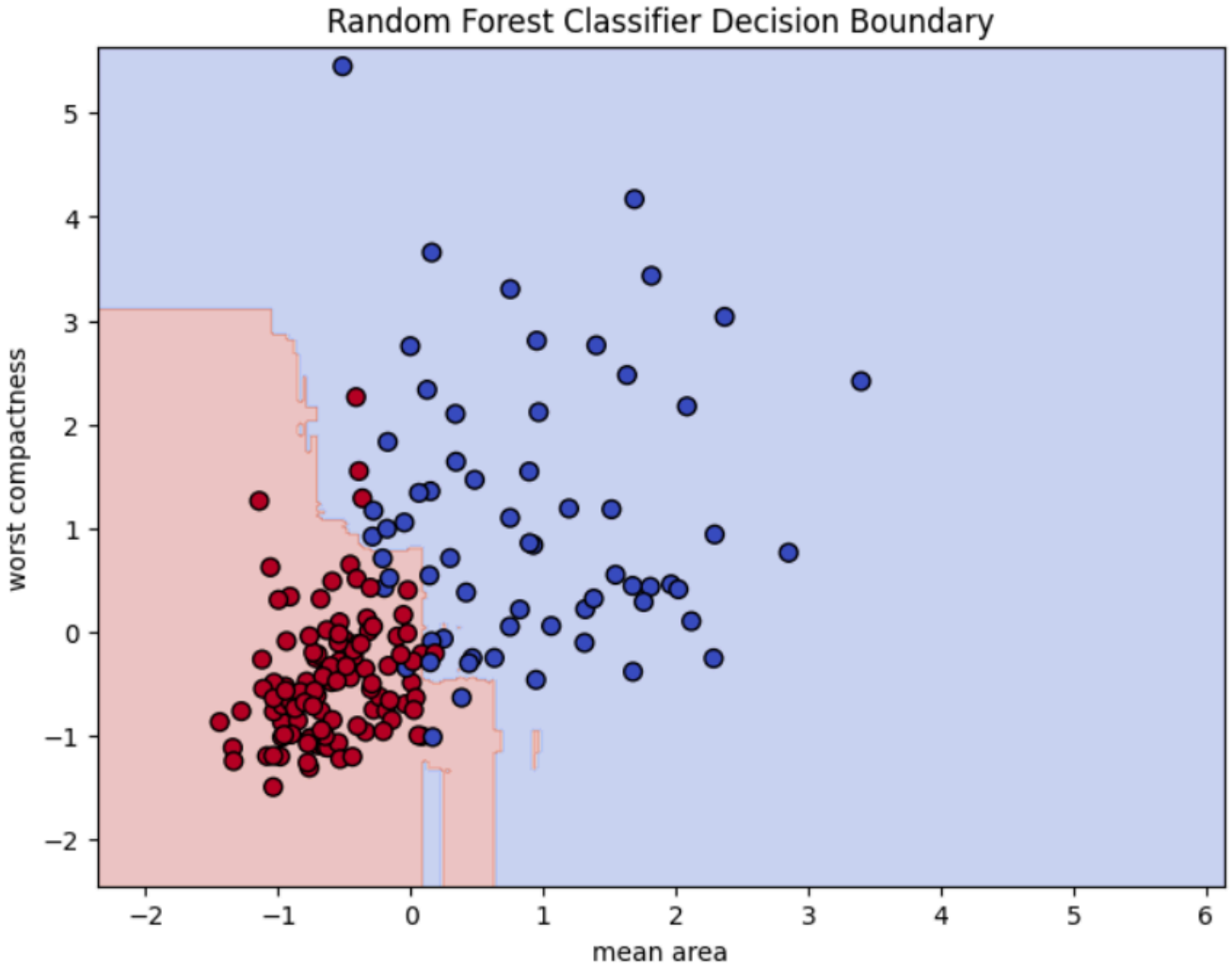
```
print("Accuracy:", accuracy_score(y_test, y_pred))

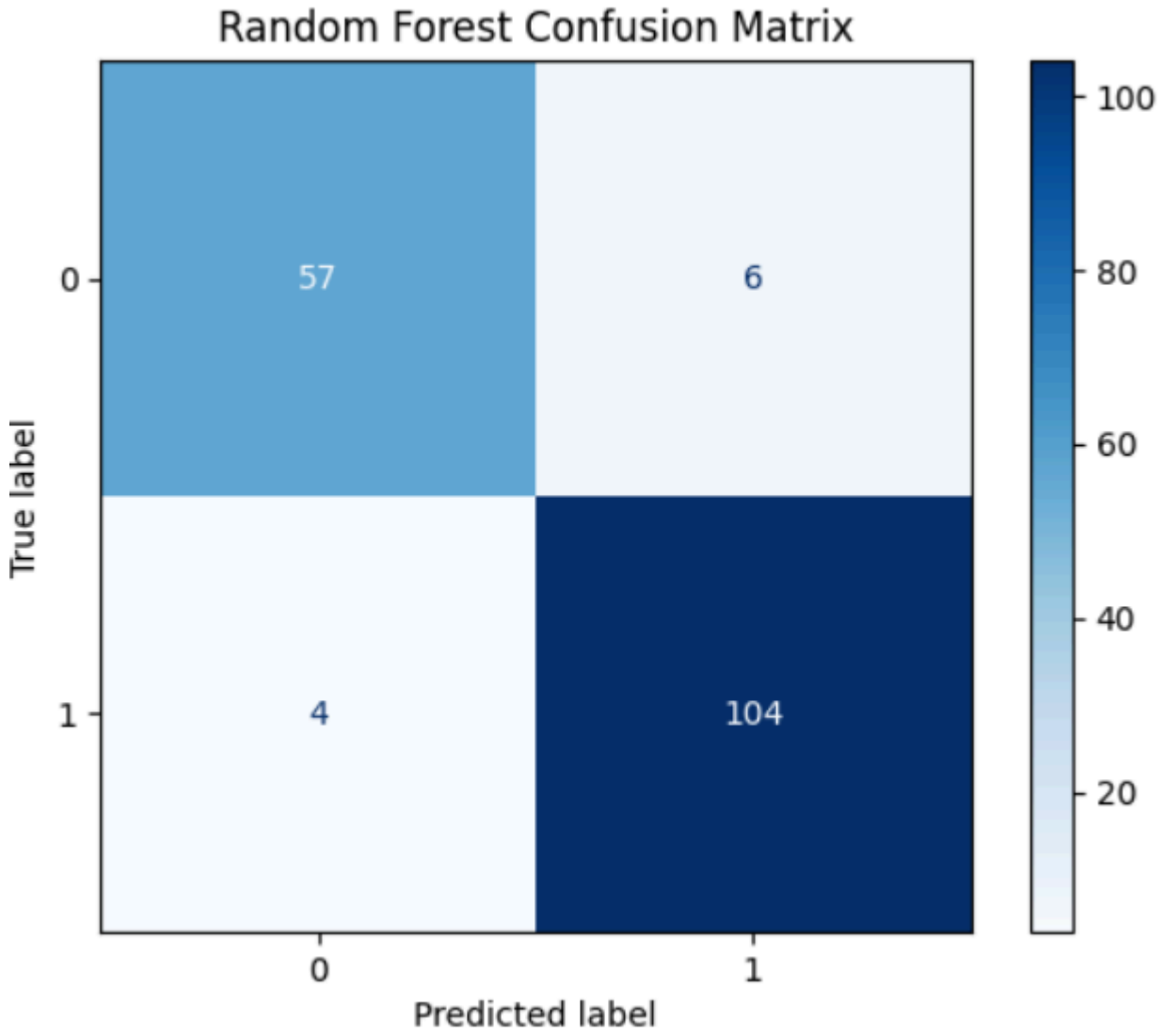
# Візуалізація
x_min, x_max = X_train_scaled[:, 0].min() - 1, X_train_scaled[:, 0].max() + 1
y_min, y_max = X_train_scaled[:, 1].min() - 1, X_train_scaled[:, 1].max() + 1
xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 300),
    np.linspace(y_min, y_max, 300)
)

Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
plt.scatter(X_test_scaled[:, 0], X_test_scaled[:, 1],
            c=y_test, cmap='coolwarm', edgecolors='k', s=50)
plt.xlabel(data.feature_names[3])
plt.ylabel(data.feature_names[25])
plt.title("Random Forest Classifier Decision Boundary")
plt.show()

# Матриця похибок
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, cmap='Blues')
plt.title("Random Forest Confusion Matrix")
plt.show()
```


Random Forest Classifier Decision Boundary
worst compactness
mean area
5
4
3
2
1
0
−1
−2
−2
−1
0
1
2
3
4
5
6



XGBoost (Extreme Gradient Boosting)

XGBoost — це ансамблевий метод машинного навчання, заснований на градієнтному бустингу дерев рішень. На відміну від випадкового лісу, де дерева будуються незалежно одне від одного, в XGBoost кожне нове дерево навчається виправляти помилки попередніх дерев.

Нехай маємо навчальну вибірку:

$$D = (x_i, y_i)_{i=1}^{N}, \quad x_i \in \mathbb{R}^m$$

Модель будується послідовно як сума дерев рішень:

$$F_T(x) = \sum_{t=1}^{T} f_t(x)$$

де $f_t(x)$

— t-те дерево рішень, а T — загальна кількість дерев.

На першій ітерації модель може складатися лише з одного дерева:

$$F_1(x) = f_1(x)$$

На кожній наступній ітерації додається нове дерево, яке апроксимує залишки (помилки) поточної моделі:

$$F_t(x) = F_{t-1}(x) + f_t(x)$$

Для задач регресії залишки визначаються як:

$$r_i^{(t)} = y_i - F_{t-1}(x_i)$$

У загальному випадку XGBoost мінімізує функцію втрат:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{N} l(y_i, \hat{y}_i^{(t)}) + \sum_{k=1}^{t} \Omega(f_k)$$

де $l(y_i, \hat{y}_i)$ — функція втрат (наприклад MSE або LogLoss), а $\Omega(f)$ — регуляризаційний доданок для боротьби з перенавчанням.

Для побудови нового дерева використовується градієнтний розклад функції втрат:

$$g_i = \frac{\partial l(y_i, \hat{y}_i)}{\partial \hat{y}_i}$$

$$h_i = \frac{\partial^2 l(y_i, \hat{y}_i)}{\partial \hat{y}_i^2}$$

де g_i та h_i є відповідно першим та другим похідними функції втрат.

Після побудови всіх дерев прогноз моделі має вигляд:

$$\hat{y} = \sum_{t=1}^{T} f_t(x)$$

Для задач класифікації отримане значення додатково пропускається через логістичну функцію:

$$P(y = 1|x) = \frac{1}{1 + e^{-\hat{y}}}$$

та клас визначається за максимальним значенням ймовірності.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score
from xgboost import XGBClassifier

# Завантажуємо датасет
data = load_breast_cancer()
X = data.data[:, [3, 25]]  # mean smoothness, worst concave points
y = data.target

# Розподіл на тренувальні та тестові дані
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Скейлимо дані
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Тренування XGBoost classifier
clf = XGBClassifier(
    n_estimators=200,
    max_depth=4,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    use_label_encoder=False,
    eval_metric='logloss',
    random_state=42
)
clf.fit(X_train_scaled, y_train)

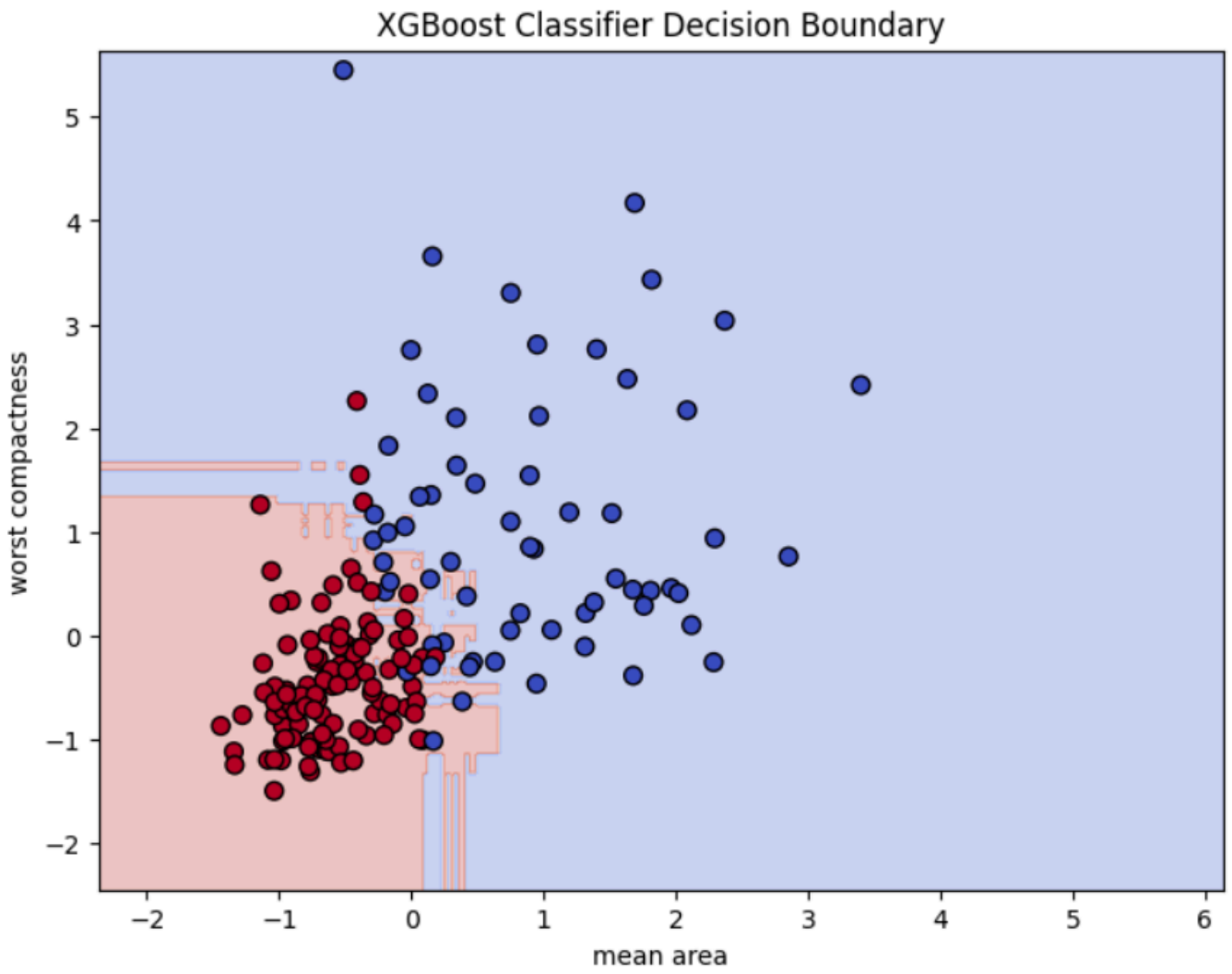
# Передбачення та оцінка моделі
y_pred = clf.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))

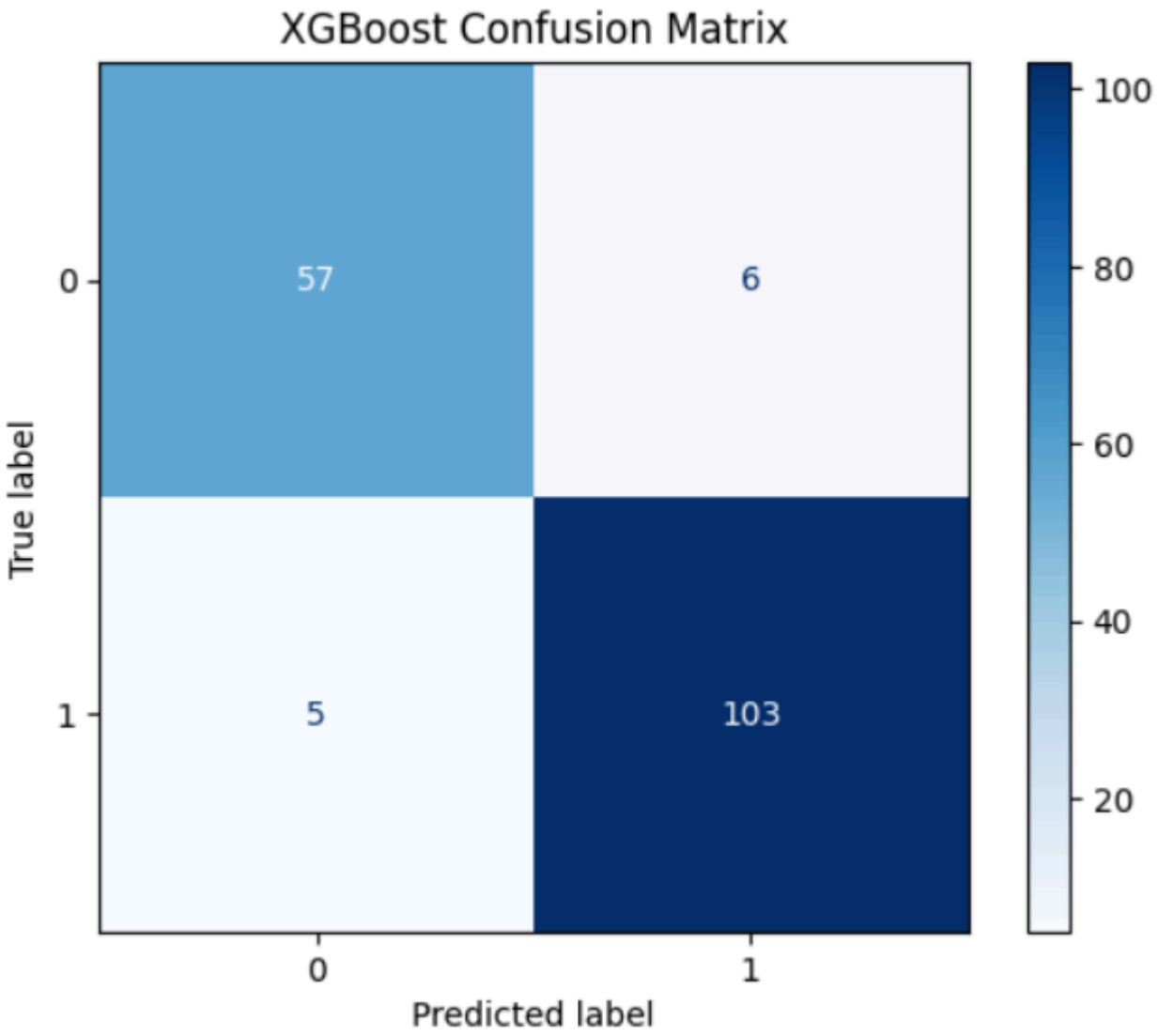
# Візуалізація
x_min, x_max = X_train_scaled[:, 0].min() - 1, X_train_scaled[:, 0].max() + 1
y_min, y_max = X_train_scaled[:, 1].min() - 1, X_train_scaled[:, 1].max() + 1
xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 300),
    np.linspace(y_min, y_max, 300)
)

Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
plt.scatter(X_test_scaled[:, 0], X_test_scaled[:, 1],
            c=y_test, cmap='coolwarm', edgecolors='k', s=50)
plt.xlabel(data.feature_names[3])
plt.ylabel(data.feature_names[25])
plt.title("XGBoost Classifier Decision Boundary")
plt.show()

# Матриця похибок
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, cmap='Blues')
plt.title("XGBoost Confusion Matrix")
plt.show()
```


XGBoost Classifier Decision Boundary
worst compactness
mean area
5
4
3
2
1
0
−1
−2
−2
−1
0
1
2
3
4
5
6



Компроміс зсуву та дисперсії (Bias–variance tradeoff)

Мета будь-якої моделі машинного навчання — знайти закономірність у даних, щоб робити точні передбачення. Однак модель може:

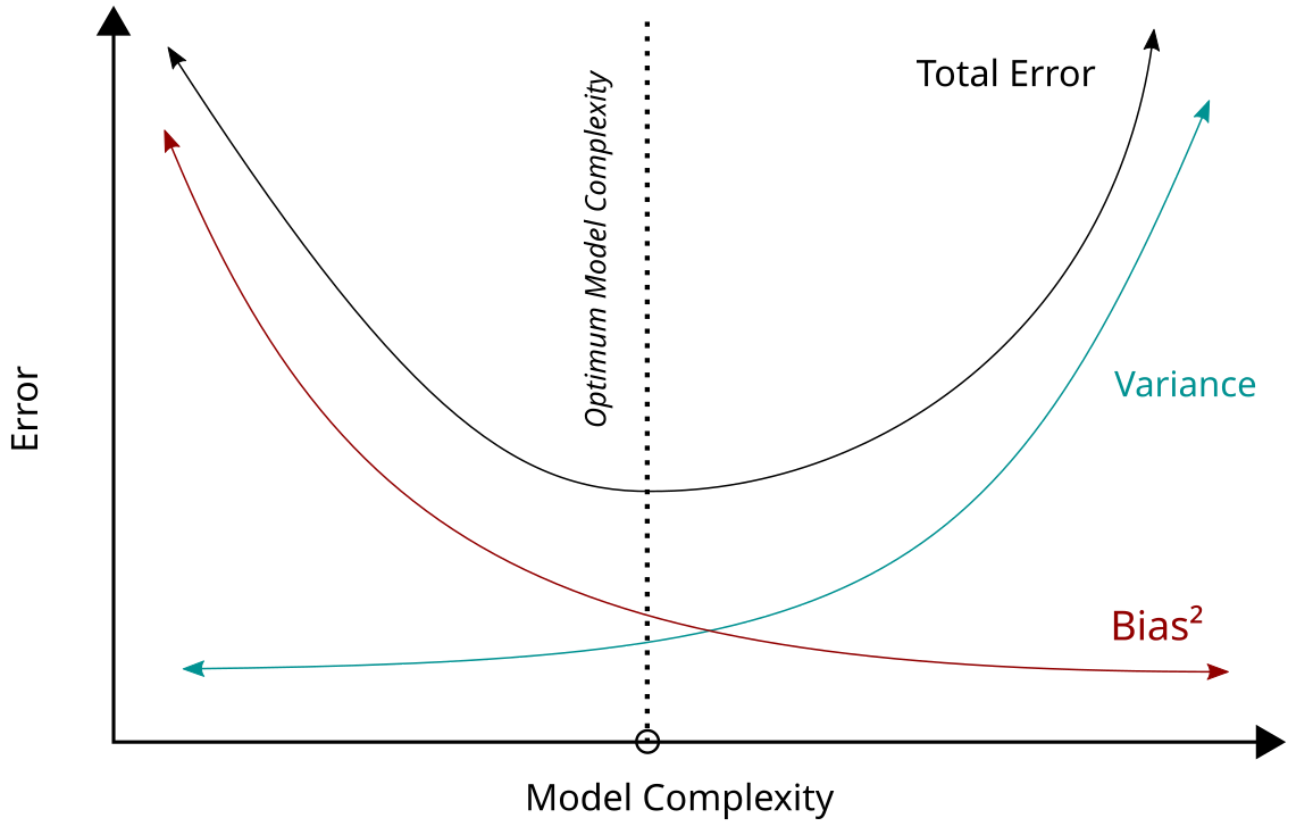
- бути занадто простою (недонавченою, underfitting)
- або занадто складною (перенавченою, overfitting)

Цей баланс описує компроміс між зсувом і дисперсією.

Компонента	Формула	Пояснення
Зсув (Bias)	$\mathrm{Bias}[\hat{f}(x)] = \mathbb{E}[\hat{f}(x)] - f(x)$	Наскільки “в середньому” модель відхиляється від істинної залежності. Великий зсув → модель занадто проста.
Дисперсія (Variance)	$\mathrm{Var}[\hat{f}(x)] = \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$	Наскільки модель “чутлива” до зміни навчальних даних. Велика дисперсія → модель перенавчається.
Невідворотний шум	$\sigma^2 = \mathrm{Var}(\varepsilon)$	Частина похибки, яку неможливо усунути, бо вона спричинена випадковими факторами в даних.

Мета навчання — знайти модель, у якій Bias² + Variance мінімальні (шум ми не контролюємо).

Варто запам'ятати, що в переважній більшості випадків, bagging ансамблеві моделі(наприклад, Випадковий Ліс) - зменшують дисперсію (variance), а boosting (XGBoost) - зменшують зсув (bias).



11. Навчання без учителя (Unsupervised Learning)

Кластеризація методом к–середніх (k-means clustering)

Метод k–середніх — це один із найпопулярніших алгоритмів некерованого навчання (unsupervised learning), який використовується для розбиття даних на k кластерів так, щоб об’єкти всередині одного кластера були схожими між собою, а між різними кластерами — максимально відрізнялися.

Нехай маємо вибірку з n об’єктів:

$$X = x_1, x_2, \ldots, x_n, \quad x_i \in \mathbb{R}^d$$

де d — кількість ознак (вимірів).

Ми хочемо розбити множину X на k кластерів:

$$C_1, C_2, \ldots, C_k$$

так, щоб мінімізувати суму квадратів відстаней точок до центрів своїх кластерів.

Алгоритм мінімізує наступну функцію вартості (відомо як *within-cluster sum of squares*, WCSS):

$$J = \sum_{j=1}^{k} \sum_{x_i \in C_j} |x_i - \mu_j|^2$$

де:

- μ_j — центр (середнє) кластера C_j:

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

- $|x_i - \mu_j|^2$ — квадрат евклідової відстані між точкою та центром кластера.

Алгоритм ітеративний і складається з таких кроків:

1. Ініціалізація: Вибрати випадковим чином k початкових центрів кластерів $\mu_1, \mu_2, \ldots, \mu_k$.
2. Крок призначення (Assignment step): Для кожної точки (x_i) визначити найближчий центр:

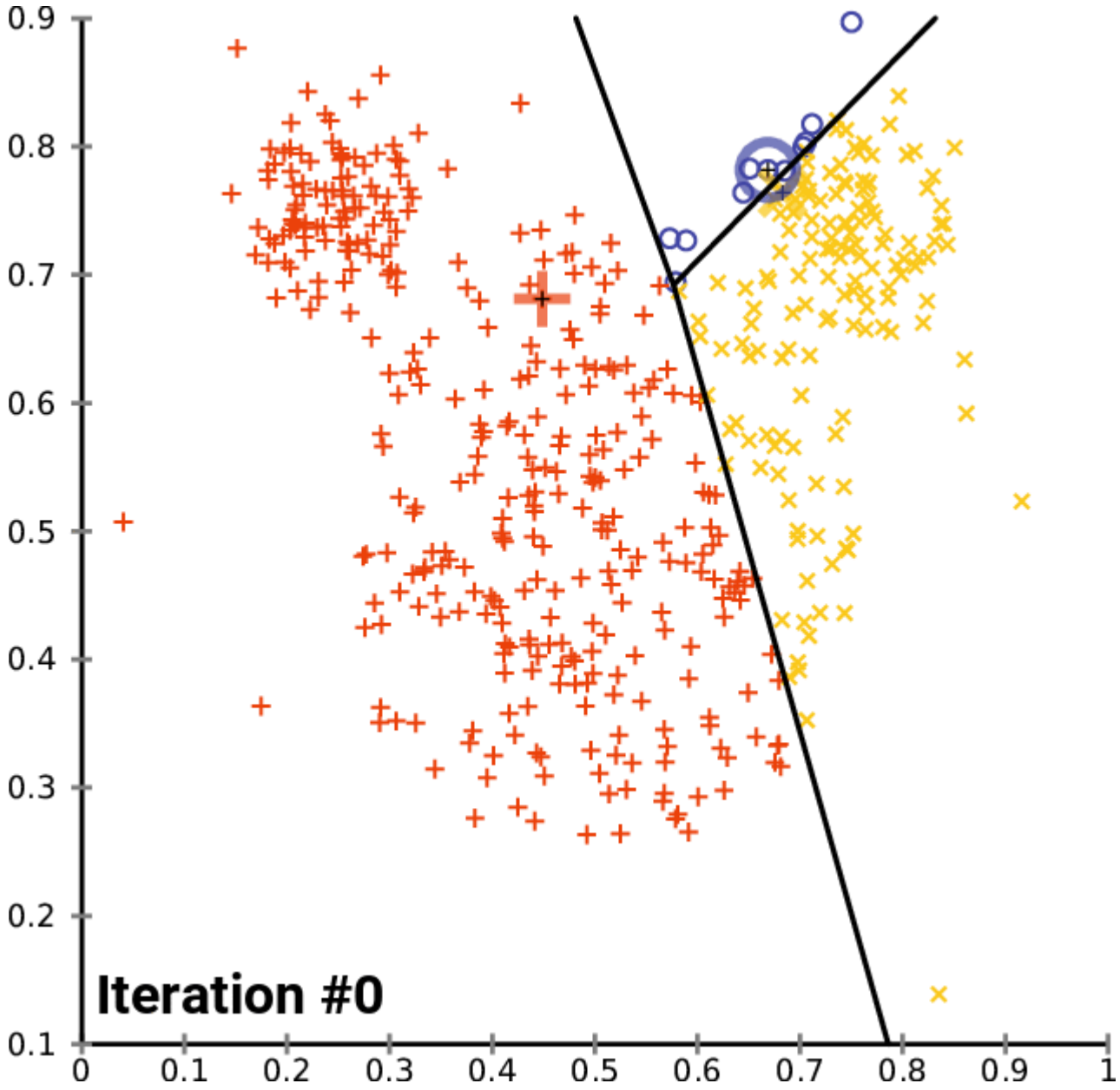
$$c_i = \arg\min_{j \in 1,\ldots,k} |x_i - \mu_j|^2$$

   Тобто точка належить до того кластера, чий центр ближчий.

3. Крок оновлення центрів (Update step) Для кожного кластера C_j оновити центр як середнє всіх точок цього кластера:

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

4. Перевірка зупинки: Алгоритм зупиняється, коли центри кластерів перестають змінюватися або зміни стають незначними.

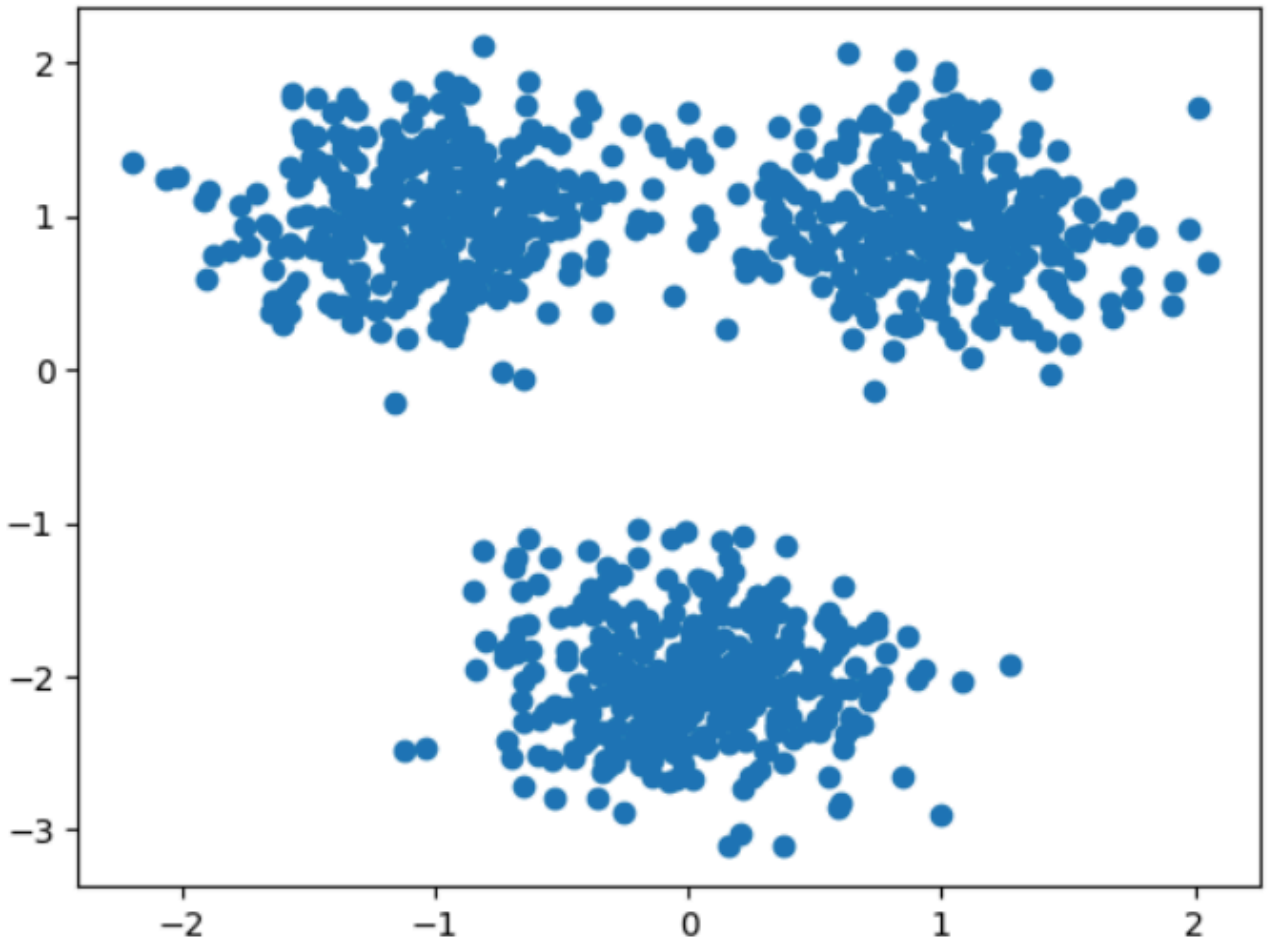


Згенеруємо в python набір даних із трьох нормальних розподілів.

```
import numpy as np
import matplotlib.pyplot as plt

X = m,n = (1000,2)
np.random.seed(0)
X = 0.4*np.random.randn(m,n)
Mu0 = np.array([[0,-2], [-1,1], [1,1]])
X += Mu0[np.random.choice(np.arange(3), m),:]
plt.scatter(X[:,0], X[:,1])
```

```
<matplotlib.collections.PathCollection at 0x1598e4ea110>
```



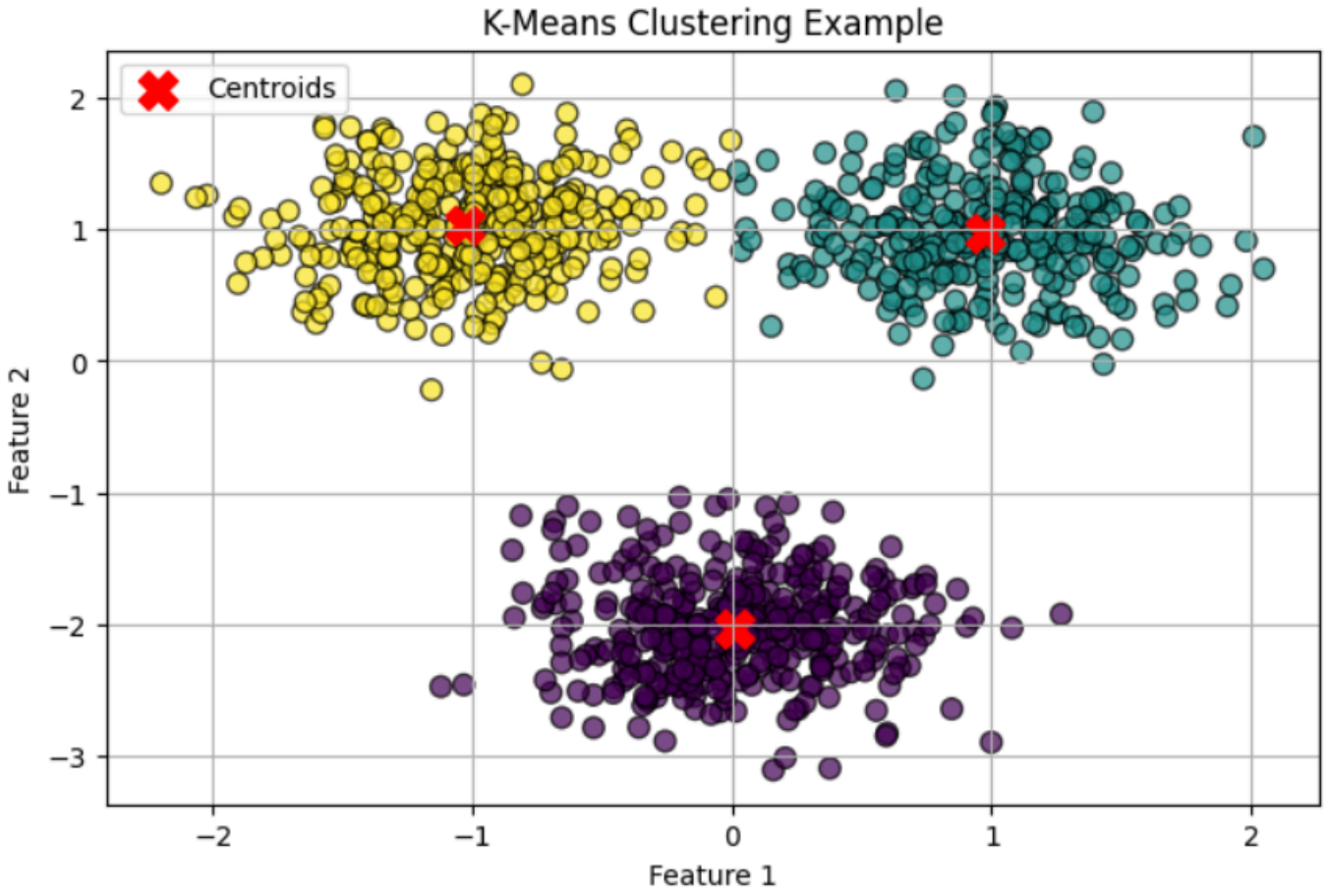
Приклад виконання алгоритму k-середніх на даному датасеті:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# застосуємо k-середніх
kmeans = KMeans(n_clusters=3, random_state=42)
labels = kmeans.fit_predict(X)
centers = kmeans.cluster_centers_

# візуалізація
plt.figure(figsize=(8, 5))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=60, edgecolors='k',
alpha=0.7)
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, marker='X',
label='Centroids')
plt.title('K-Means Clustering Example')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
```

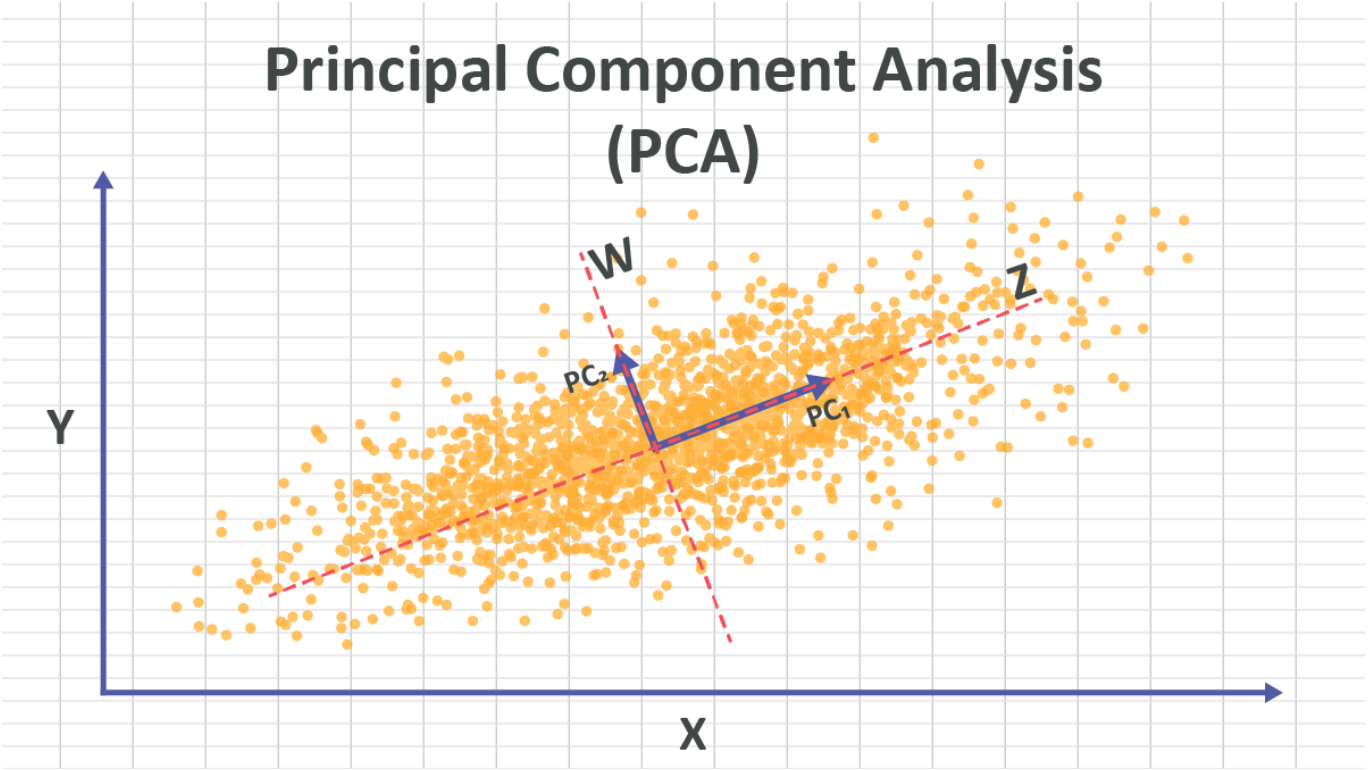
```
plt.grid(True)
plt.show()
```



Метод головних компонент (PCA)

Метод головних компонент (PCA) — це статистичний метод зменшення розмірності даних. Його ідея полягає в тому, щоб знайти нову систему координат, де:

- перша вісь (перша головна компонента) пояснює максимальну дисперсію даних,
- друга — наступну за величиною дисперсію, і т. д. Так при збереженні головних компонент ми зберігаємо максимум інформації при мінімумі вимірів.



Застосуємо алгоритм головних компонент на відомий нам датасет раку молочної залози. Завантажимо дані та виведемо розмірність.

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer

cancer = load_breast_cancer(as_frame=True)
# створюємо датафрейм
df = cancer.frame
print('Original Dataframe shape :',df.shape) # вхідні ознаки
X = df[cancer['feature_names']]
print('Inputs Dataframe shape :', X.shape)
```

```
Original Dataframe shape : (569, 31)
Inputs Dataframe shape : (569, 30)
```

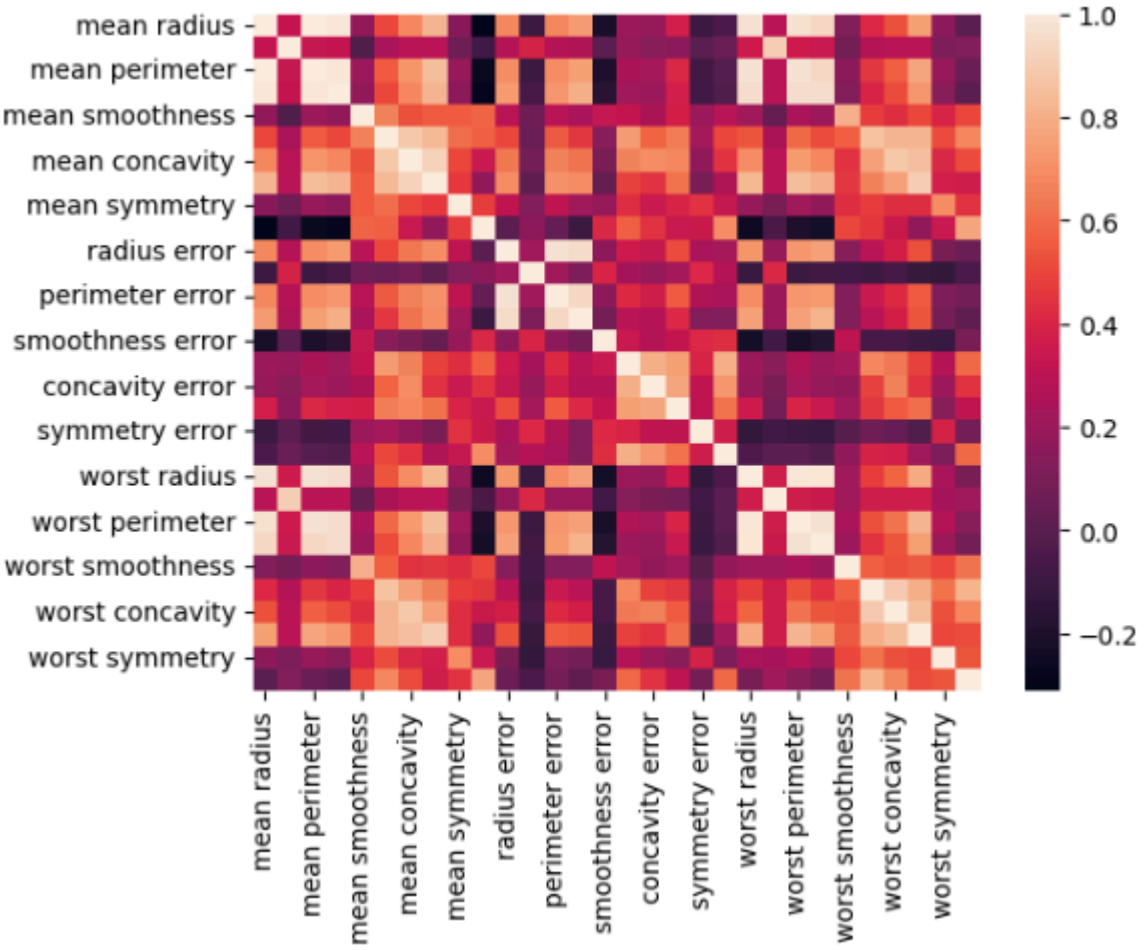
30 вхідних ознак потрібно спроектувати на 2 головні ознаки.

```
X_mean = X.mean()
X_std = X.std()
X_normalized = (X - X_mean) / X_std # нормалізуємо дані
```

Побудуємо матрицю коваріацій

```
c = X_normalized.cov()

import matplotlib.pyplot as plt
import seaborn as sns
sns.heatmap(c)
plt.show()
```



Знайдемо на основі даних про коваріацію власні значення - власні значення матриці коваріації.

```
eigenvalues, eigenvectors = np.linalg.eig(c)
print('Eigen values:\n', eigenvalues)
print('Eigen values Shape:', eigenvalues.shape)
print('Eigen Vector Shape:', eigenvectors.shape)
```

```
Eigen values:
 [1.32816077e+01 5.69135461e+00 2.81794898e+00 1.98064047e+00
 1.64873055e+00 1.20735661e+00 6.75220114e-01 4.76617140e-01
 4.16894812e-01 3.50693457e-01 2.93915696e-01 2.61161370e-01
 2.41357496e-01 1.57009724e-01 9.41349650e-02 7.98628010e-02
 5.93990378e-02 5.26187835e-02 4.94775918e-02 1.33044823e-04
 7.48803097e-04 1.58933787e-03 6.90046388e-03 8.17763986e-03
```

```
 1.54812714e-02 1.80550070e-02 2.43408378e-02 2.74394025e-02
 3.11594025e-02 2.99728939e-02]
Eigen values Shape: (30,)
Eigen Vector Shape: (30, 30)
```

```
# Проіндексуємо власні значення в порядку спадання
idx = eigenvalues.argsort()[::-1]
eigenvalues = eigenvalues[idx] # відповідно відсортуємо і власні вектори
eigenvectors = eigenvectors[:,idx]
```

```
explained_var = np.cumsum(eigenvalues) / np.sum(eigenvalues)
print(explained_var)
```

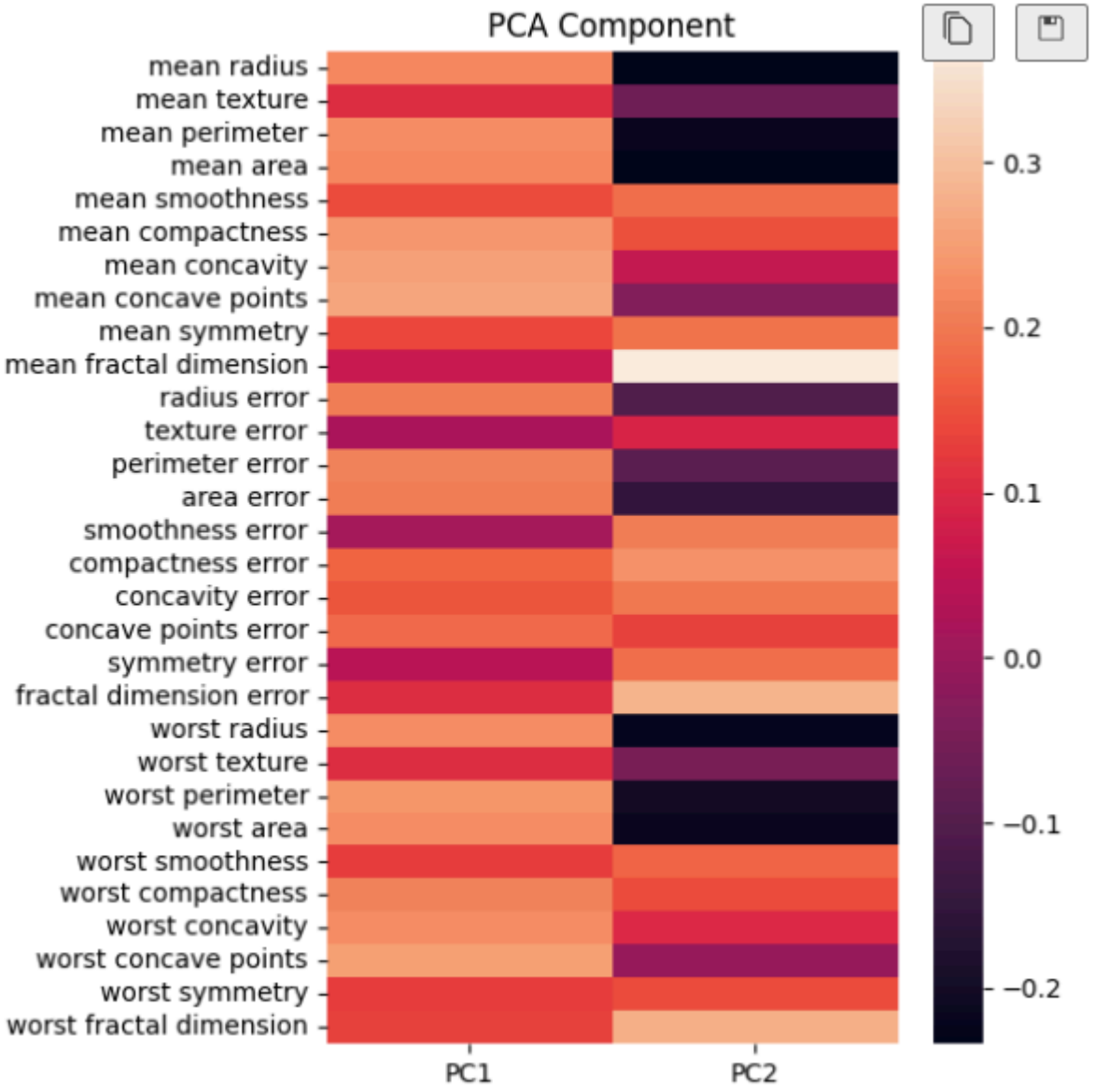
```
[0.44272026 0.63243208 0.72636371 0.79238506 0.84734274 0.88758796
 0.9100953  0.92598254 0.93987903 0.95156881 0.961366   0.97007138
 0.97811663 0.98335029 0.98648812 0.98915022 0.99113018 0.99288414
 0.9945334  0.99557204 0.99657114 0.99748579 0.99829715 0.99889898
 0.99941502 0.99968761 0.99991763 0.99997061 0.99999557 1.        ]
```

```
n_components = np.argmax(explained_var >= 0.50) + 1
print(n_components)
```

```
2
```

```
# PCA компоненти
u = eigenvectors[:,:n_components]
pca_component = pd.DataFrame(u, index = cancer['feature_names'], columns =
['PC1','PC2'] )

# будуємо heatmap
plt.figure(figsize =(5, 7))
sns.heatmap(pca_component)
plt.title('PCA Component')
plt.show()
```




```
# Множимо матриці нормалізованих даних на PCA компоненти
Z_pca = X_normalized @ pca_component
Z_pca.rename({'PC1': 'PCA1', 'PC2': 'PCA2'}, axis=1, inplace=True)
print(Z_pca)
```

```
          PCA1       PCA2
0     9.184755   1.946870
1     2.385703  -3.764859
2     5.728855  -1.074229
3     7.116691  10.266556
4     3.931842  -1.946359
..         ...        ...
564   6.433655  -3.573673
565   3.790048  -3.580897
566   1.255075  -1.900624
567  10.365673   1.670540
568  -5.470430  -0.670047
```

```
[569 rows x 2 columns]
```

Тепер той же самий алгоритм, але використовуємо модуль sklearn.

```
from sklearn.decomposition import PCA

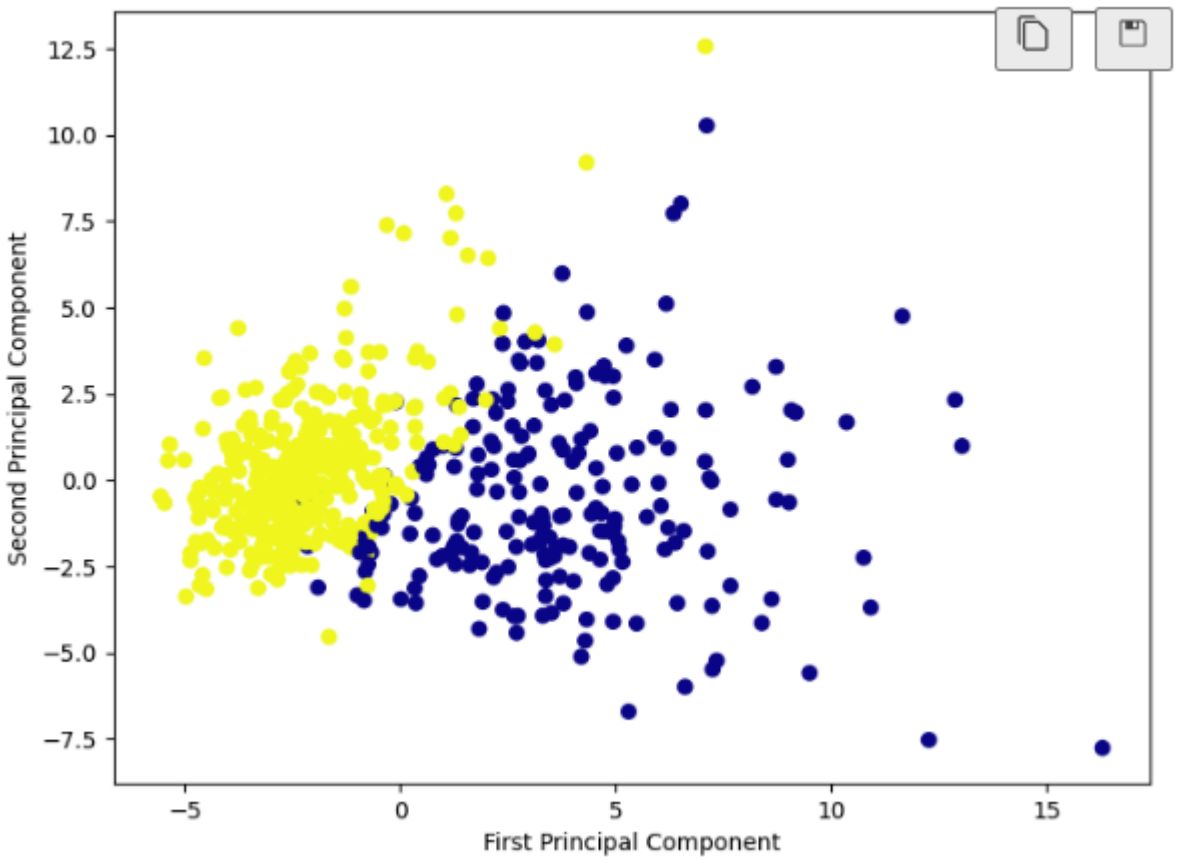
pca = PCA(n_components=2)
pca.fit(X_normalized)
x_pca = pca.transform(X_normalized)

df_pca1 = pd.DataFrame(x_pca, columns=['PC{}'. format(i+1) for i in
range(n_components)])
print(df_pca1)

# Будуємо графік
plt.figure(figsize=(8, 6))
plt.scatter(x_pca[:, 0], x_pca[:, 1], c=cancer['target'], cmap='plasma')
plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
plt.show()
```

```
           PC1        PC2
0     9.184755   1.946870
1     2.385703  -3.764859
2     5.728855  -1.074229
3     7.116691  10.266556
4     3.931842  -1.946359
..         ...        ...
564   6.433655  -3.573673
565   3.790048  -3.580897
566   1.255075  -1.900624
567  10.365673   1.670540
568  -5.470430  -0.670047

[569 rows x 2 columns]
```



12. Перевірка статистичних гіпотез (Hypothesis testing)

Перевірка гіпотез — це фундаментальний статистичний метод, який використовується в науці про дані для формування висновків або узагальнень про популяцію на основі вибіркових даних. Він допомагає нам визначити, чи є спостережуваний ефект у наборі даних статистично значущим, чи він міг виникнути випадково.

У науці про дані перевірка гіпотез використовується в різних сферах, зокрема в A/B-тестуванні, регресійному аналізі та оцінці моделей машинного навчання.

Ключові поняття в тестуванні гіпотез

Нульова гіпотеза (H_0): Нульова гіпотеза — це твердження про відсутність ефекту або різниці. Вона слугує базовим або вихідним припущенням. Наприклад, в A/B-тесті нульова гіпотеза може стверджувати, що дві групи мають однакову середню продуктивність.

Альтернативна гіпотеза (H_1): Альтернативна гіпотеза є протилежною нульовій гіпотезі. Вона передбачає, що існує статистично значущий ефект або різниця. Наприклад, альтернативна гіпотеза може стверджувати, що група A має вищий середній показник, ніж група B.

Тестова статистика: Тестова статистика — це стандартизоване значення, обчислене на основі вибіркових даних, яке використовується для оцінки сили доказів проти нульової гіпотези. До поширених тестових статистик належать t-оцінки та z-оцінки.

P-значення: P-значення представляє ймовірність отримання тестової статистики, що є принаймні такою ж екстремальною, як і спостережувана, припускаючи, що нульова гіпотеза є правдивою. Мале P-значення (зазвичай менше 0,05) вказує на сильні докази проти нульової гіпотези.

Рівень значущості (α): Рівень значущості (часто позначається як α) — це поріг, обраний аналітиком для прийняття рішення про відхилення нульової гіпотези. Зазвичай α встановлюється на рівні 0,05, що означає 5% ймовірність відхилення нульової гіпотези, коли вона є справжньою (помилка I типу).

Помилки типу I та типу II:

Помилка типу I: Неправильне відхилення нульової гіпотези, коли вона є істинною (хибнопозитивний результат). Помилка типу II: Невідхилення нульової гіпотези, коли вона є хибною (хибнонегативний результат). Потужність тесту: Потужність тесту — це ймовірність правильного відхилення нульової гіпотези, коли вона є хибною. Висока потужність означає менший ризик помилки типу II.

Приклад: У нас є ігровий кубик (гральна кістка). Ми підозрюємо, що він "підкручений" — тобто шістка випадає частіше, ніж мала б.

Ми кидаємо кубик 60 разів, і отримуємо:

кількість шісток: 15

- Нульова гіпотеза H_0: кубик чесний

$$p = \frac{1}{6} \approx 0.1667$$

- Альтернативна гіпотеза H_1: кубик "підкручений на шістку"

$$p > \frac{1}{6}$$

(Тобто — односторонній тест.)

Випадкова величина - Позначимо:

$$X = \text{кількість шісток у 60 кидках}$$

Тоді:

$$X \sim \text{Binomial}(n = 60, p = 1/6)$$

Очікуване значення та дисперсія тоді будуть наступні:

$$\mathbb{E}[X] = np = 60 \times \frac{1}{6} = 10$$

$$\text{Var}(X) = np(1-p) = 60 \times \frac{1}{6} \times \frac{5}{6} \approx 8.33$$

$$\sigma = \sqrt{8.33} \approx 2.89$$

Наближено нормальний тест - для (n=60) можна використати нормальне наближення:

$$Z = \frac{X - \mathbb{E}[X]}{\sigma} = \frac{15 - 10}{2.89} \approx 1.73$$

Рівень значущості - візьмемо рівень $\alpha = 0.05$. Критичне значення для одностороннього тесту:

$$Z_{\text{кр}} = 1.645$$

Прийняття рішення - оскільки $Z = 1.73 > 1.645$, то ми відхиляємо (H_0) на рівні значущості 0.05.

Отже, є статистичні підстави вважати, що кубик дійсно “підкручений” на шістку. Це одновибірковий біноміальний тест.

Перевірка гіпотез у машинному навчанні

У машинному навчанні перевірка гіпотез відіграє важливу роль в оцінці моделей. Наприклад, при порівнянні двох моделей (наприклад, дерева рішень і випадкового лісу) можна використовувати перевірку гіпотез, щоб оцінити, чи є різниця в продуктивності (наприклад, точність, прецизійність) статистично значущою.

Типовим сценарієм є використання перехресного валідації для навчання моделей на різних підмножинах даних, а потім виконання статистичного тесту (наприклад, парного t-тесту) для визначення, чи одна модель постійно перевершує іншу.

Практичний приклад: A/B-тестування з перевіркою гіпотез

Уявіть ситуацію, коли компанія, що займається електронною комерцією, хоче перевірити, чи новий дизайн веб-сайту (B) приносить більше покупок, ніж поточний дизайн (A). Компанія проводить A/B-тестування і збирає дані про кількість покупок, зроблених користувачами, які відвідують кожну версію сайту.

Етапи:

- Нульова гіпотеза (H_0): новий дизайн не впливає на рівень покупок (тобто немає різниці між A і B).
- Альтернативна гіпотеза (H_1): новий дизайн збільшує рівень покупок.
- Рівень значущості: Встановіть α = 0,05.
- Збір даних: Припустимо, ви зібрали такі дані:
- Версія A: 500 відвідувачів, 50 покупок.
- Версія B: 600 відвідувачів, 80 покупок.
- Проведення тесту: Використовуйте z-тест для порівняння частоти покупок між двома групами.
- Висновок: На основі p-значення відхиліть або не відхиліть нульову гіпотезу.

Наведемо деякі приклади статистичних перевірок.

Одновибірковий t-критерій (One-sample t-test)

Нехай у нас є дані про ріст студентів. Нашим припущенням є те, що є певний середній ріст студентів (у прикладі нижче - 183 сантиметри). Проведемо одновибірковий t-критерій, щоб перевірити цю гіпотезу

```
# імпортуємо пакети
import scipy.stats as stats
import numpy as np

data = [182, 180, 190, 184]

# проводимо one sample t-test
t_statistic, p_value = stats.ttest_1samp(a=data, popmean=183)
print(t_statistic , p_value)
```

```
0.46291004988627577 0.6749411569291093
```

Оскільки $p = 0.675 > 0.05$, ми не можемо відхилити нульову гіпотезу. Отримані дані не дають статистично значущих підстав стверджувати, що середній зріст відрізняється від 183 см.

Інакше кажучи, вибірка $182, 180, 190, 184$ узгоджується з припущенням про середнє значення 183.

Однофакторний дисперсійний аналіз (One-way ANOVA)

```python
# Візьмемо дані з трьох незалежних груп
group1 = [2.9, 3.0, 2.5, 2.6, 3.2, 2.8]
group2 = [3.8, 2.7, 4.0, 3.9, 3.2, 3.1]
group3 = [3.1, 3.4, 3.7, 3.0, 3.1, 3.6]

# Проводимо one-way ANOVA
f_statistic, p_value = stats.f_oneway(group1, group2, group3)

print(f"F-statistic: {f_statistic}")
print(f"P-value: {p_value}")
```

```
F-statistic: 4.434477379095165
P-value: 0.03068555150519455
```

Оскільки $p = 0.0307 < 0.05$ нульова гіпотеза відхиляється. Існують статистично значущі відмінності між середніми значеннями хоча б двох груп.

Одновибірковий критерій хі-квадрат (англ. One-sample Chi-Squared test)

Застосуємо одновибірковий критерій хі-квадрат до нашого прикладу із гральним кубиком.

```python
# Розподіл результатів кидків грального кубика - від 1 до 6
observed = np.array([30, 10, 5, 5, 5, 5])

avg = observed.sum()

expected = np.array([avg/6, avg/6, avg/6, avg/6, avg/6, avg/6])

# Проводимо one-sample Chi-Square goodness of fit test
chi2_stat, p_value = stats.chisquare(f_obs=observed, f_exp=expected)

print(f"Chi-Square Statistic: {chi2_stat}")
print(f"P-value: {p_value}")
```

```
Chi-Square Statistic: 50.0
P-value: 1.3857973367009573e-09
```

Оскільки $p << 0.05$ нульова гіпотеза впевнено відхиляється. Отриманий розподіл практично неможливо пояснити випадковими коливаннями - ігровий кубик є зваженим.

Критерій незалежності хі-квадрат (Chi-Square Test of Independence)

```python
# Приклад даних: таблиця співвідношень зі спостережуваними частотами
# Наприклад, це можуть бути дані про вподобання споживачів щодо двох брендів у різних вікових групах.
observed = np.array([[1,2,2,5],
                     [1, 3, 2, 1]])

# Проводимо Chi-Square Test of Independence
chi2_stat, p_value, dof, expected = stats.chi2_contingency(observed)

print(f"Chi-Square Statistic: {chi2_stat}")
print(f"P-value: {p_value}")
print(f"Degrees of Freedom: {dof}")
print(f"Expected Frequencies:\n{expected}")
```

```
Chi-Square Statistic: 2.412380952380953
P-value: 0.49133407367391724
Degrees of Freedom: 3
Expected Frequencies:
[[1.17647059 2.94117647 2.35294118 3.52941176]
 [0.82352941 2.05882353 1.64705882 2.47058824]]
```

Оскільки $p = 0.491 > 0.05$, ми не можемо відхилити нульову гіпотезу. Статистично значущого зв'язку між віковою групою та вибором бренду не виявлено.

Спостережувані відмінності можуть бути пояснені випадковими коливаннями вибірки.

Тип даних / Кількість груп	**1 група**	**2 групи**	**2 і більше груп**
Категоріальні дані	Тест для частки (наближення Z-критерієм) Критерій хі-квадрат	Тест для частки (наближення Z-критерієм) Критерій хі-квадрат	Критерій хі-квадрат
Неперервні дані	Z-критерій та його варіанти T-критерій та його варіанти	Z-критерій та його варіанти T-критерій та його варіанти	Дисперсійний аналіз ANOVA (F-критерій, однофакторний або двофакторний)

Тип даних / Кількість груп	1 група	2 групи	2 і більше груп
Порушення класичних припущень	Знаковий критерій (Sign Test) Критерій знакових рангів (Signed Rank Test)	Критерій Вілкоксона–Манна–Уїтні Парний t-критерій Критерій Мак-Немара	Критерій Краскела–Уолліса

13. Рекомендаційні системи (Recommender Systems)

Колаборативна фільтрація (collaborative filtering)

Колаборативна фільтрація (CF) — це підхід у системах рекомендацій, який прогнозує, які об’єкти (товари, фільми, книги тощо) можуть сподобатися користувачу, виходячи з подібності між користувачами або між самими об’єктами.

Підходи «користувач – користувач»: знайти користувачів, які найбільш схожі на мене (на основі тільки тих елементів, які оцінені для нас обох), і передбачити оцінки для інших елементів на основі середнього значення

Підходи «елемент – елемент»: знайти елементи, найбільш схожі на даний елемент (на основі всіх користувачів, які оцінили обидва елементи), і передбачити оцінки для інших користувачів на основі середнього значення

Підходи матричної факторизації: знайти деяку низькорангову декомпозицію матриці X, яка узгоджується з спостережуваними значеннями

Маємо:

- множину користувачів $U = u_1, u_2, \ldots, u_m$,
- множину об’єктів $I = i_1, i_2, \ldots, i_n$.

Створюється матриця оцінок:

$$R = [r_{ui}]_{m \times n}$$

де:

- r_{ui} — рейтинг, який користувач u поставив об’єкту i,
- якщо користувач не оцінив об’єкт — r_{ui} відсутній (невідомий).

Передбачити невідомий рейтинг:

$$\hat{r_{ui}} \approx r_{ui}$$

для пари *(користувач, об’єкт)*, якої ще немає у матриці.

User-based Collaborative Filtering (на основі подібності користувачів)

Ідея: Шукаємо користувачів, схожих на користувача u, і прогнозуємо його рейтинг на основі оцінок цих схожих користувачів.

Кроки:

1. Обчислити схожість між користувачами, наприклад, за допомогою косинусної подібності:

$$\text{sim}(u, v) = \frac{\sum_{i \in I_{uv}} r_{ui} r_{vi}}{\sqrt{\sum_{i \in I_{uv}} r_{ui}^2} \sqrt{\sum_{i \in I_{uv}} r_{vi}^2}}$$

   де I_{uv} — множина об'єктів, які оцінили обидва користувачі.

2. Прогноз рейтингу:

$$\hat{r_{ui}} = \bar{r_u} + \frac{\sum_{v \in N(u)} \text{sim}(u, v)(r_{vi} - \bar{r}_v)}{\sum_{v \in N(u)} |\text{sim}(u, v)|}$$

   де:

   - $N(u)$ — набір найбільш схожих користувачів (сусідів),
   - $\bar{r}_u$ — середній рейтинг користувача u.

Окремо можна винести: *Кореляцію Пірсона*:

$$\text{sim}(x, y) = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}$$

Користувач / Фільм 🎬	Фільм A	Фільм B	Фільм C	Фільм D
👤 **U1**	5	4	?	2
👤 **U2**	3	?	4	1
👤 **U3**	?	2	5	4
👤 **U4**	4	3	?	?

```
import numpy as np
import pandas as pd

# Приклад user-item матриці з рейтингами
ratings_matrix = np.array([
    [5, 3, 0, 1],
    [4, 0, 0, 1],
    [1, 1, 0, 5],
    [1, 0, 0, 4],
    [0, 1, 5, 4],
])

# Перетворюємо матрицю в датафрейм
ratings_df = pd.DataFrame(ratings_matrix, columns=["Item1", "Item2", "Item3",
"Item4"])
```

```
user_similarity = ratings_df.T.corr(method='pearson')

print("User-User Pearson Correlation Matrix:\n", user_similarity)
```

```
User-User Pearson Correlation Matrix:
          0         1         2         3         4
0  1.000000  0.774291 -0.186441 -0.178683 -0.978839
1  0.774291  1.000000  0.019854  0.162791 -0.628768
2 -0.186441  0.019854  1.000000  0.972828  0.221028
3 -0.178683  0.162791  0.972828  1.000000  0.258904
4 -0.978839 -0.628768  0.221028  0.258904  1.000000
```

Передбачимо рейтинг користувача.

```
# Функція для передбачення рейтингу користувача за айтем
def predict_rating(user_index, item_index, ratings_df, similarity_matrix):
    user_means = ratings_df.replace(0, np.nan).mean(axis=1).values

    # Отримуємо рейтинг за айтем від усіх інших користувачів
    item_ratings = ratings_df.iloc[:, item_index]

    # Отримуємо подібність користувача з іншими користувачами
    user_similarities = similarity_matrix[user_index]

    # Фільтруємо користувачів за тими, хто надав рейтинг цьому айтему і вилучаємо
самоподібність
    non_zero_indices = item_ratings[item_ratings > 0].index
    similarities = user_similarities[non_zero_indices]
    ratings = item_ratings[non_zero_indices]
    means = user_means[non_zero_indices]

    # Якщо жоден інший користувач надав рейтинг цьому айтему, ставимо 0
    if len(similarities) == 0:
        return 0

    # Зважена сума рейтингів, нормалізована за подібностями користувачів.

    ratings_weighted = ratings - means
    weighted_sum = np.dot(similarities, ratings_weighted)

    sum_of_similarities = np.sum(np.abs(similarities))

    predicted = weighted_sum / sum_of_similarities if sum_of_similarities != 0
else 0

    return ratings.mean() + predicted

print(predict_rating(4, 0, ratings_df, user_similarity))
```

```
1.0331985858244794
```

14. Часові Ряди

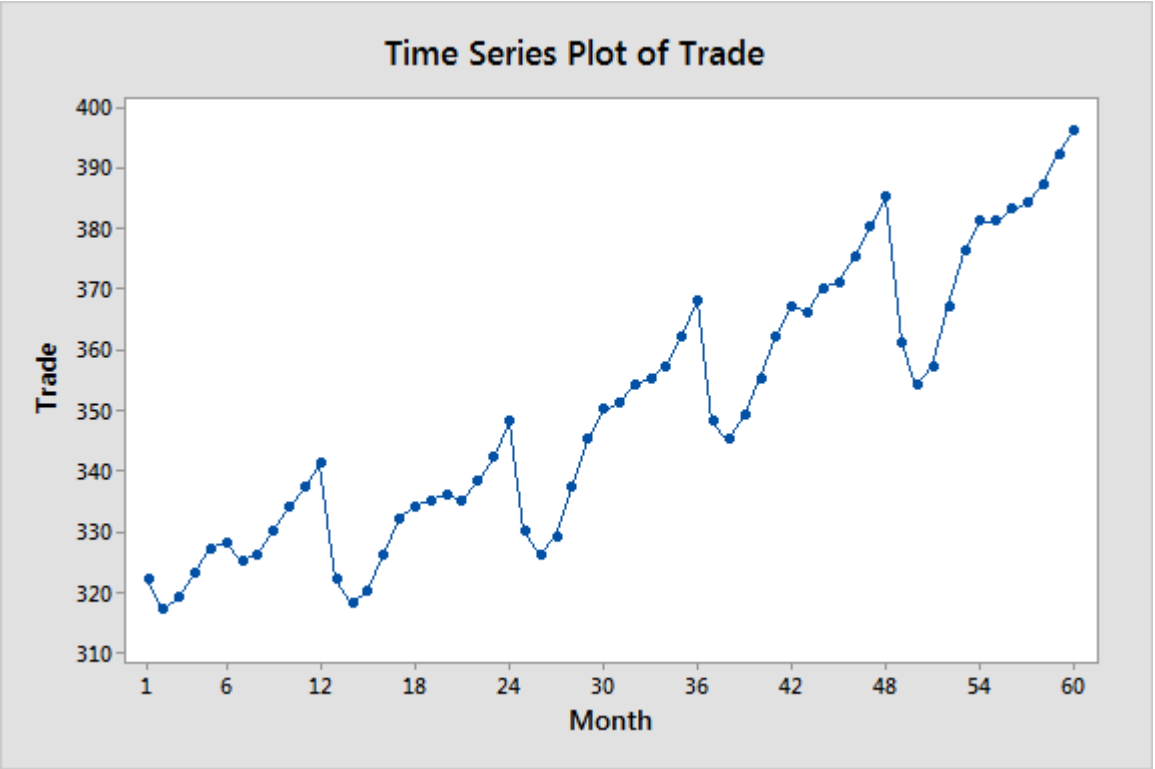
Часовий ряд дає можливість передбачити значення наступної (майбутньої) точки (змінної), наприклад, передбачити ціну акції через годину, день, тиждень, базуючись на розмірності даних.

Ряд складається з двох частин - сигналу та шуму. Сигнал - це математично залежні змінні, які можна проаналізувати та прибрати з часового ряду. Шум навпаки складається з рандомних значень, частіше за все нормального розподілу, та візуально схожий на звуковий шум.

Патерни сигналу:

- Сезонність (seasonality) - циклічні коливання змінної в залежності від часу. Наприклад, зниження температури взимку.
- Тренд (trend/level) - це наклон ряду до горизонтальної осі. Тренд може бути зростаючим або спадаючим, якщо тренд не змінюється, то часовий ряд розташований горизонтально.
- Автокореляція - це “пам’ять” ряду, значення змінної в конкретний час напряму залежить від попередніх значень змінної.

Якщо ряд не містить патерни, його називають стаціонарним. Стаціонарним (Stationary) є ряд без тренду та сезонності, середнє та стандартне відхилення якого є константами. Окремим патерном є часові ряди, які не можна зробити стаціонарними, прибравши тренд та/або сезонність. Такі ряди називають difference - stationary.

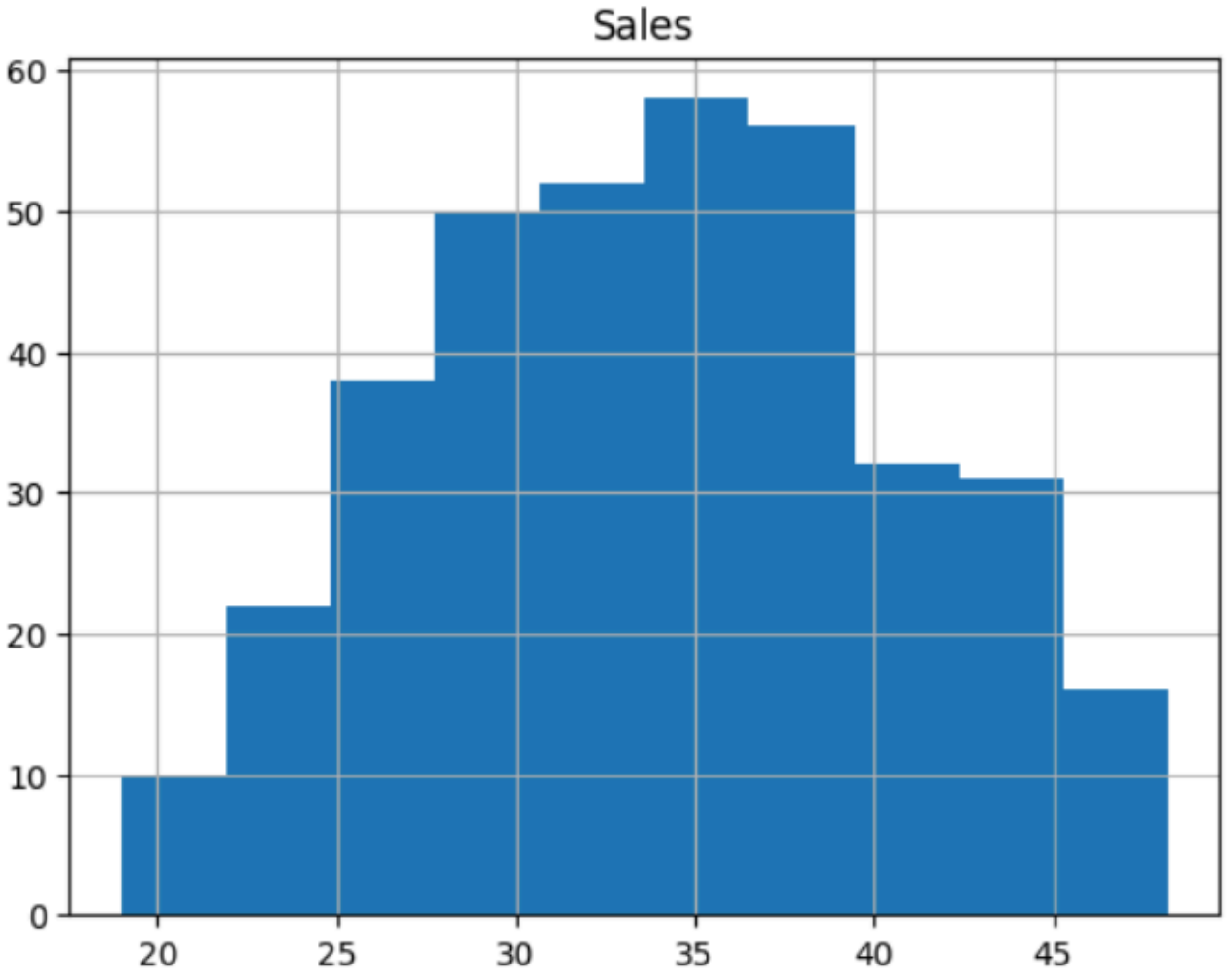



```
import pandas as pd
```

```
data = pd.read_csv('../resources/sales.csv')
display(data.head(10))

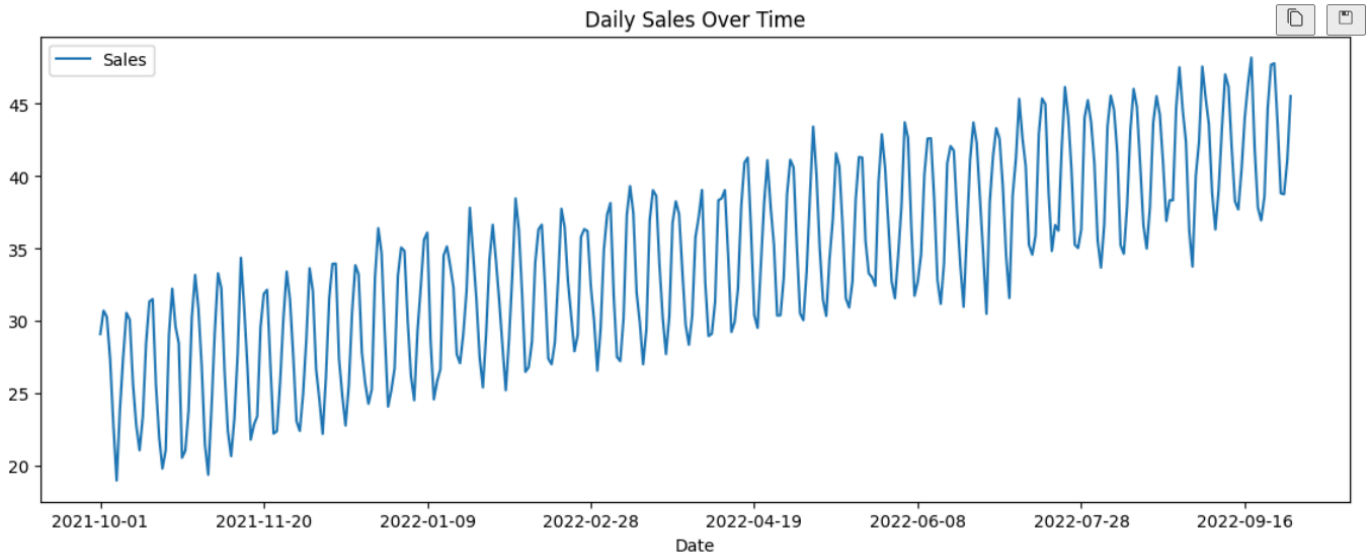
# data = data.drop('Unnamed: 0', axis=1)
data.hist()
```

```
Unnamed: 0  Date    Sales
0   0   2021-10-01  29.109547
1   1   2021-10-02  30.720435
2   2   2021-10-03  30.305173
3   3   2021-10-04  27.390893
4   4   2021-10-05  22.841123
5   5   2021-10-06  19.002444
6   6   2021-10-07  23.904594
7   7   2021-10-08  27.544137
8   8   2021-10-09  30.567059
9   9   2021-10-10  30.087033
```



Зобразимо даний часовий ряд:

```
data.plot(x='Date', y='Sales', kind='line', title='Daily Sales Over Time',
figsize=(14, 5))
```




```
# data has no missing values
print(data.describe())
print("NAN values:",data.isna().sum())
```

```
            Sales
count  365.000000
mean    34.093942
std      6.572683
min     19.002444
25%     29.290376
50%     33.971567
75%     38.647294
max     48.190967
NAN values: Date     0
Sales    0
dtype: int64
```

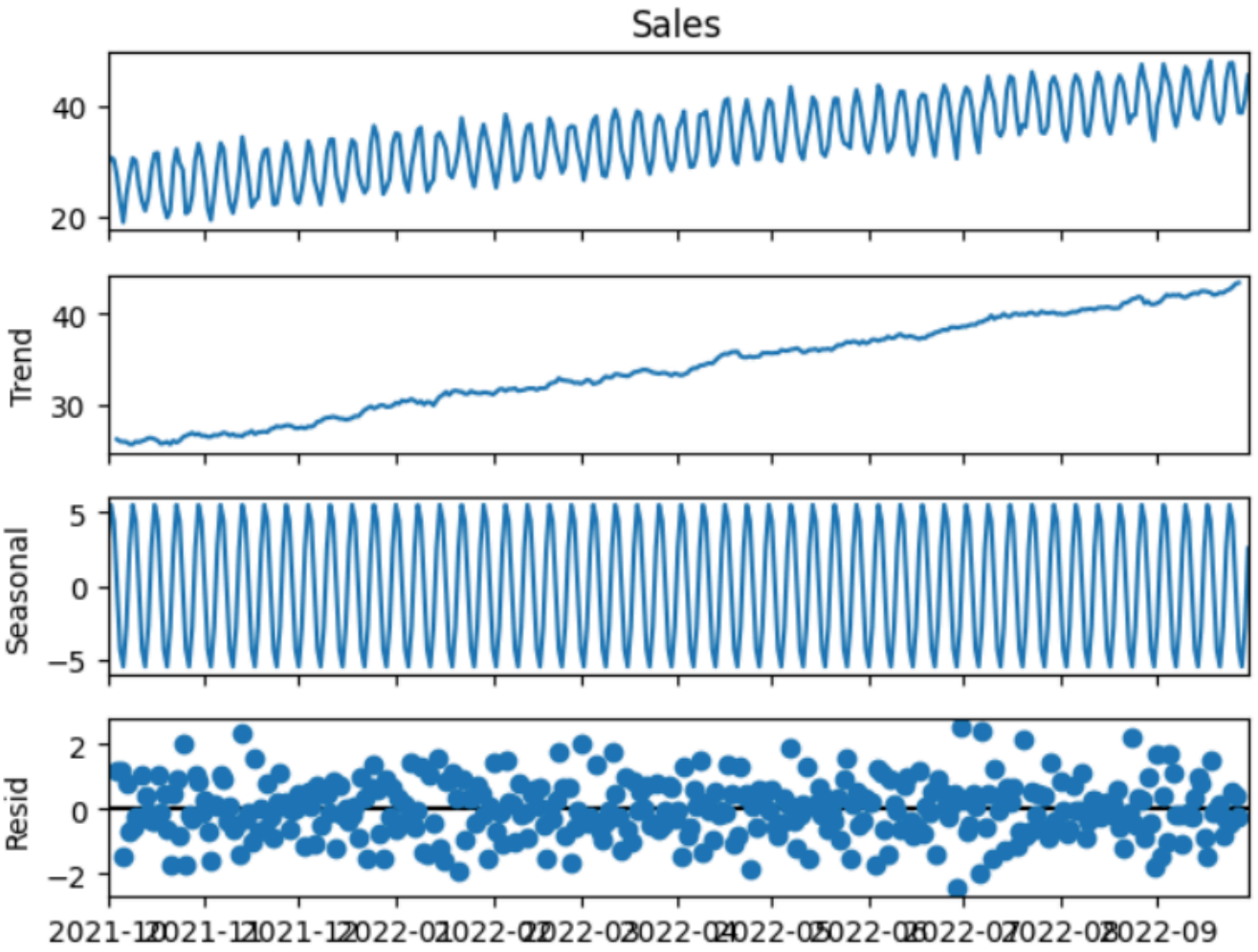
На графіку вище чітко видно сезонну закономірність, яка повторюється тиждня, та висхідний тренд. Давайте детальніше розглянемо тренд та сезонну закономірність:

Шум (resid) - це часовий ряд після прибирання з нього тренду та сезонності.

```
data['Date'] = pd.to_datetime(data['Date'])
data = data.set_index('Date')
data = data.asfreq('D')
```

```
from statsmodels.tsa.seasonal import seasonal_decompose

decomposed = seasonal_decompose(data['Sales'])
decomposed.plot()
```



Доповнений тест Дікі - Фуллера (ADF)

Тест ADF - це тест для перевірки стаціонарності часового ряду на основі unit root - це процес, який через диференціювання визначає чи є часовий ряд стаціонарним. Тест працює за принципом приймання або відкидання нульової гіпотези, яка полягає у тому, що ряд не є стаціонарним. Приймати чи відкидати нульову гіпотезу вирішується за значенням p (вірогідність неправильності нульової гіпотези): якщо значення є більшим за визначену константу, то нульова гіпотеза приймається (ряд не є стаціонарним), якщо меншим - гіпотеза відкидається (ряд є стаціонарним). Традиційно, константа = 0.05 (5%), але може бути іншою, в залежності від даних та моделі. Також важливо дивитися і на значення статистики, якщо воно більше за будь-яке критичне значення, то часовий ряд не є стаціонарним.

```
from statsmodels.tsa.stattools import adfuller
import matplotlib.pyplot as plt

# data = data.drop('Unnamed: 0', axis=1)

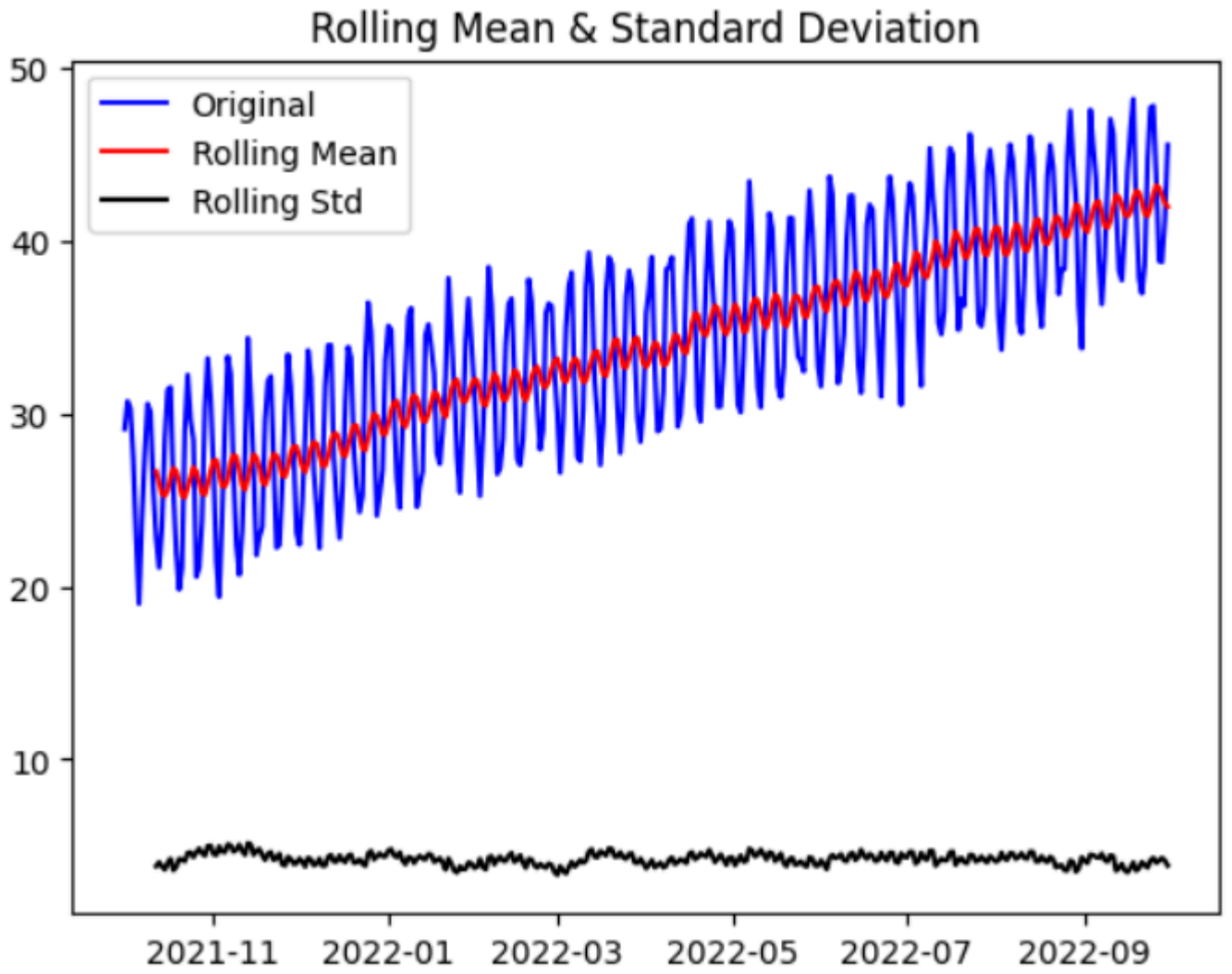
def adfuller_test(df):
```

```
    adf=adfuller(df)
    statistic= adf[0]
    p_value=adf[1]
    critical_values=adf[4]
    return p_value,statistic,critical_values

def stat_check(df,p_value_check=0.05):
    movingAverage = df.rolling(window=12).mean()
    movingSTD = df.rolling(window=12).std()
    orig = plt.plot(df, color='blue', label='Original')
    mean = plt.plot(movingAverage, color='red', label='Rolling Mean')
    std = plt.plot(movingSTD, color='black', label='Rolling Std')
    plt.legend(loc='best')
    plt.title('Rolling Mean & Standard Deviation')
    plt.show(block=False)

    p_value,statistic,critical_values=adfuller_test(df)
    if (p_value<p_value_check):
        print('Time series is stationary, p value:{}'.format(p_value))
    else:
        print('Time series is NOT stationary, p value:{}'.format(p_value))
    print('Statistic:{}'.format(statistic))
    for key, value in critical_values.items():
        print('Critial Values:')
        print(f'   {key}, {value}')

stat_check(data)
```



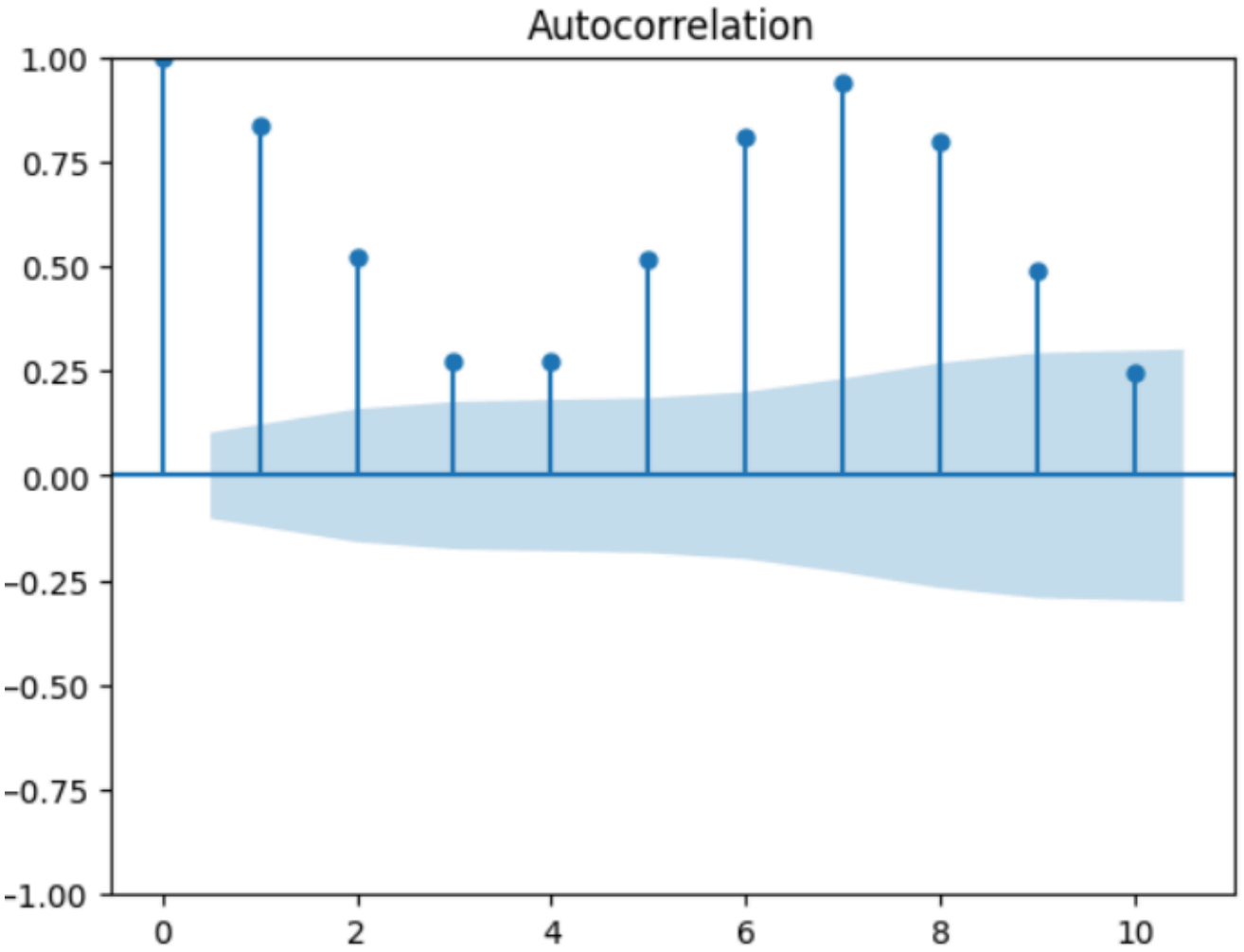

```
Time series is NOT stationary, p value:0.979126294138459
Statistic:0.34045531718527483
Critial Values:
   1%, -3.4492815848836296
Critial Values:
   5%, -2.8698813715275406
Critial Values:
   10%, -2.5712138845950587
```

Автокореляція

Кореляція - це зв’язок змінних між собою, автокореляція - зв’язок змінної з власними попередніми значенням. Оскільки ряд має тільки одну змінну, аналізувати потрібно тільки автокореляцію. Перед тим, як графік автокореляції розкриється у всій красі, розкажу про поняття lags (лаги) - це значення змінної n - кількість часу назад, оскільки розмірністю нашого часового ряду є день, то lag -1 означає значення 1 днів тому для поточної змінної, -2 - два дні тому, тощо. На наведеному нижче графіку кількість лагів дорівнює 10.

```
from statsmodels.graphics.tsaplots import plot_acf
```

```
plot_acf(data,lags=10)
plt.show()
```



Декомпозиція - це процес перетворення нестаціонарного часового ряду у стаціонарний шляхом прибирання з нього сезонності та тренду.

Реконструкція часового ряду - це процес зворотнього додавання до ряду тренду та сезонності, якщо цього не зробити, ряд буде паралельним осі x та буде мало корисним як прогнозування.

Функції втрати

Показниками якості моделей та прогнозування для часових рядів є функції втрати та метрики похибки. Найчастіше використовують такі функції втрати як MAE (L1 norm) - “Mean Square Error”, MSE(L2 norm) - “Mean Absolute Error”.

MAE надає кожній помилці однакову “вагу”, тобто є робастною до викидів та шуму функцією, але одночасно з цим занижує “вплив” великих помилок.

MSE в свою чергу навпаки, дає більшим помилкам більшу вагу, що робить функцію більш чутливою до викидів. Якщо модель повинна бути чутлива до помилок, обирають саме цю функцію.

```
import numpy as np

def calculate_mape(actual, predicted):

    if([isinstance(actual, np.ndarray) or isinstance(predicted, np.ndarray)]):
        actual, predicted = np.array(actual),  np.array(predicted)
    else:
        actual, predicted = np.Int(actual),  np.Int(predicted)

    return round(np.mean(np.abs((
      actual - predicted) / actual)) * 100, 2)
```

ARIMA (AutoRegressive Integrated Moving Average) — один із найпопулярніших методів класичного прогнозування часових рядів. Його використовують, коли дані мають автокореляцію, тренд, але не обов'язково мають сезонність.

ARIMA дозволяє моделювати складні часові залежності й робити прогноз на майбутні значення.

Модель ARIMA описується трійкою параметрів:

$$ARIMA(p, d, q)$$

де

- p – кількість авторегресивних лагів (AR)
- d – порядок диференціювання (I, Integrated), щоб зробити ряд стаціонарним
- q – порядок ковзного середнього (MA)

Поточне значення залежить від попередніх:

$$y_t = \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \varepsilon_t$$

Авторегресійна частина (AR(p)) - поточне значення залежить від попередніх.

Інтегрована частина (I(d)) - диференціювання робить ряд стаціонарним:

$$y_t^d = y_t - y_{t-1}$$

або для d>1 – повторюється кілька разів. Стаціонарність важлива, бо AR і MA працюють саме з нею.

Ковзне середнє (MA(q))

$$y_t = \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \cdots + \theta_q \varepsilon_{t-q} + \varepsilon_t$$

Це модель залежності від попередніх випадкових шоків (помилок).

Повна модель ARIMA - поєднання всіх частин:

$$ARIMA(p, d, q) = AR(p) + I(d) + MA(q)$$

Після диференціювання:

$$y'_t = \phi_1 y'_{t-1} + ... + \phi_p y'_{t-p} + \theta_1 \varepsilon_{t-1} + ... + \theta_q \varepsilon_{t-q} + \varepsilon_t$$

Починаємо із $y'_0 = y_0$

```
from statsmodels.tsa.arima.model import ARIMA

model = ARIMA(data['Sales'], order=(7,1,7))
model_fit = model.fit()

print(model_fit.summary())
forecast = model_fit.forecast(steps=30)

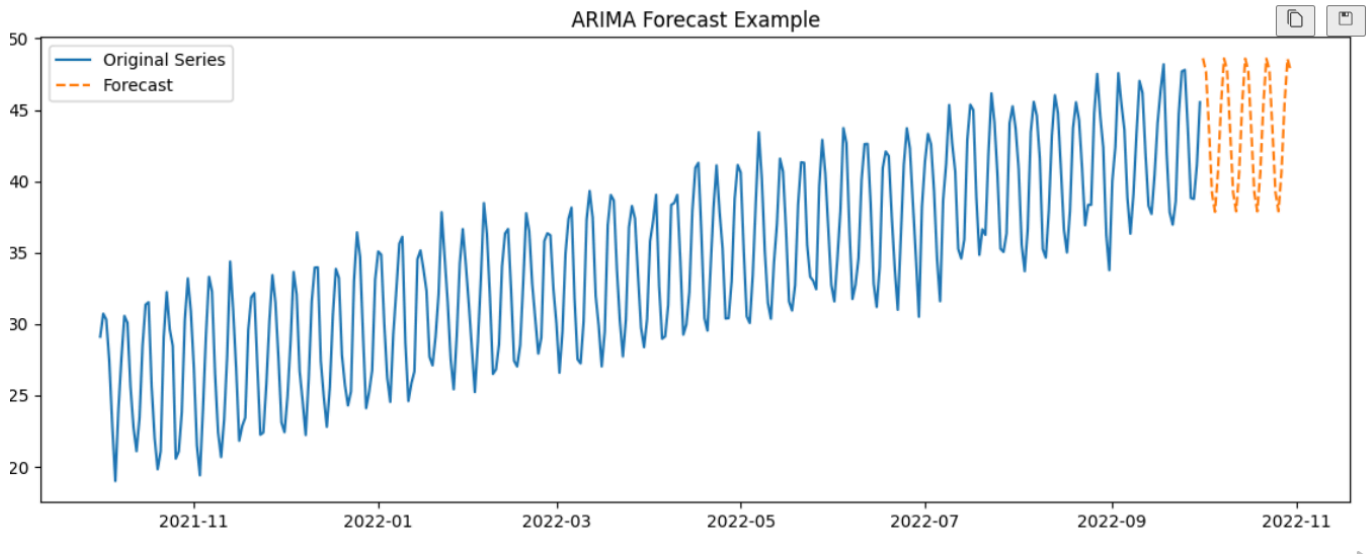
print("Forecasted values:")
print(forecast)

plt.figure(figsize=(14, 5))
plt.plot(data['Sales'], label="Original Series")
plt.plot(forecast.index, forecast, label="Forecast", linestyle='--')
plt.legend()
plt.title("ARIMA Forecast Example")
plt.show()
```

```
                               SARIMAX Results
==============================================================================
Dep. Variable:                  Sales   No. Observations:                  365
Model:                 ARIMA(7, 1, 7)   Log Likelihood                -536.758
Date:                Fri, 21 Nov 2025   AIC                           1103.517
Time:                        12:02:09   BIC                           1161.974
Sample:                    10-01-2021   HQIC                          1126.751
                         - 09-30-2022
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -1.5086      1.264     -1.193      0.233      -3.986       0.969
ar.L2         -1.0640      0.866     -1.228      0.219      -2.762       0.634
ar.L3         -0.8506      0.554     -1.536      0.124      -1.936       0.235
ar.L4         -1.3599      0.572     -2.376      0.018      -2.482      -0.238
ar.L5         -1.4469      1.224     -1.182      0.237      -3.846       0.952
ar.L6         -0.7478      0.760     -0.984      0.325      -2.238       0.742
ar.L7         -0.1451      0.231     -0.629      0.529      -0.597       0.307
ma.L1          0.5698      1.260      0.452      0.651      -1.900       3.039
ma.L2         -0.2415      0.583     -0.414      0.679      -1.385       0.902
ma.L3          0.0016      0.369      0.004      0.997      -0.721       0.724
ma.L4          0.6490      0.221      2.934      0.003       0.216       1.083
ma.L5          0.3343      0.746      0.448      0.654      -1.129       1.797
ma.L6         -0.4259      0.335     -1.270      0.204      -1.083       0.232
ma.L7         -0.3330      0.435     -0.765      0.444      -1.186       0.520
sigma2         1.1157      0.091     12.310      0.000       0.938       1.293
===================================================================================
=
Ljung-Box (L1) (Q):                   0.00   Jarque-Bera (JB):
```

```
0.39
Prob(Q):                              0.95   Prob(JB):
0.82
Heteroskedasticity (H):               0.92   Skew:
0.08
Prob(H) (two-sided):                  0.67   Kurtosis:
3.02
================================================================================
=

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-
step).
Forecasted values:
2022-10-01    48.616060
2022-10-02    47.693508
2022-10-03    43.710640
2022-10-04    39.170891
2022-10-05    37.856026
2022-10-06    40.696951
2022-10-07    45.373877
2022-10-08    48.592830
2022-10-09    47.813319
2022-10-10    43.598087
2022-10-11    39.219554
2022-10-12    37.894455
2022-10-13    40.630354
2022-10-14    45.411763
2022-10-15    48.584311
2022-10-16    47.786612
2022-10-17    43.630119
2022-10-18    39.215936
2022-10-19    37.889536
2022-10-20    40.644359
2022-10-21    45.394903
2022-10-22    48.578806
2022-10-23    47.792791
2022-10-24    43.627127
2022-10-25    39.225426
2022-10-26    37.895271
2022-10-27    40.638293
2022-10-28    45.391572
2022-10-29    48.573010
2022-10-30    47.790048
Freq: D, Name: predicted_mean, dtype: float64
```



Передбачення із використанням методів машинного навчання.

$y_n(y_{n-1}, y_{n-2}, \dots, y_{n-p})$

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor

def create_lags(series, n_lags=10):
    df = pd.DataFrame({"y": series})
    for i in range(1, n_lags + 1):
        df[f"lag_{i}"] = df["y"].shift(i)
    return df.dropna()

N_LAGS = 10
df_lags = create_lags(data['Sales'], N_LAGS)

X = df_lags.drop("y", axis=1)
y = df_lags["y"]

model = RandomForestRegressor(n_estimators=500, random_state=42)
model.fit(X, y)

steps = 30
last_values = list(data['Sales'].values[-N_LAGS:])
forecast_values = []

for _ in range(steps):
    x_input = np.array(last_values[-N_LAGS:]).reshape(1, -1)
    y_pred = model.predict(x_input)[0]

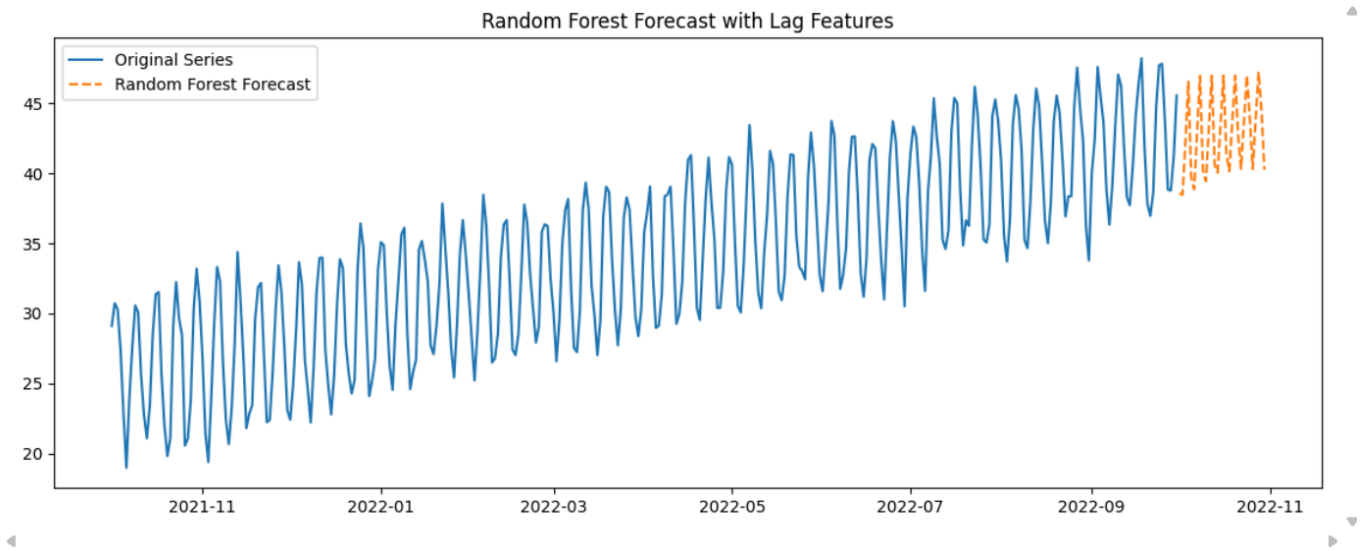
    forecast_values.append(y_pred)
    last_values.append(y_pred)

last_date = data.index[-1]
future_index = pd.date_range(start=last_date, periods=steps+1, freq='D')[1:]
```

```
forecast = pd.Series(forecast_values, index=future_index)

print("Forecast:")
plt.figure(figsize=(14, 5))
plt.plot(data['Sales'], label="Original Series")
plt.plot(forecast.index, forecast, label="Random Forest Forecast", linestyle='--')
plt.legend()
plt.title("Random Forest Forecast with Lag Features")
plt.show()
```

```
Forecast:
```



Експоненційне зглажування

Експоненційне згладжування — це клас методів прогнозування часових рядів, у яких останні спостереження отримують більшу вагу, ніж старі. Ваги зменшуються експоненційно, звідки й назва.

Нехай y_t — фактичне значення часового ряду, а $\hat{y}_t$ — прогноз.

Тоді *просте експоненційне згладжування (SES)* визначається як:

$$\hat{y}_t = \alpha y_{t-1} + (1 - \alpha)\hat{y}_{t-1}$$

де

- $0 < \alpha < 1$ — коефіцієнт згладжування
- велике α - модель сильніше реагує на нові дані
- мале α - прогноз згладжений, повільно змінюється

Модель Холта (Holt's Linear Trend): якщо в ряді є тренд, ми додаємо окрему модель для тренду:

$$\hat{y}_t = \ell_{t-1} + b_{t-1}\ \ell_t \quad = \alpha y_t + (1 - \alpha)(\ell_{t-1} + b_{t-1})\ b_t = \beta(\ell_t - \ell_{t-1}) + (1 - \beta)b_{t-1}$$

де

- ℓ_t — рівень

- b_t — тренд
- $\alpha, \beta \in (0, 1)$

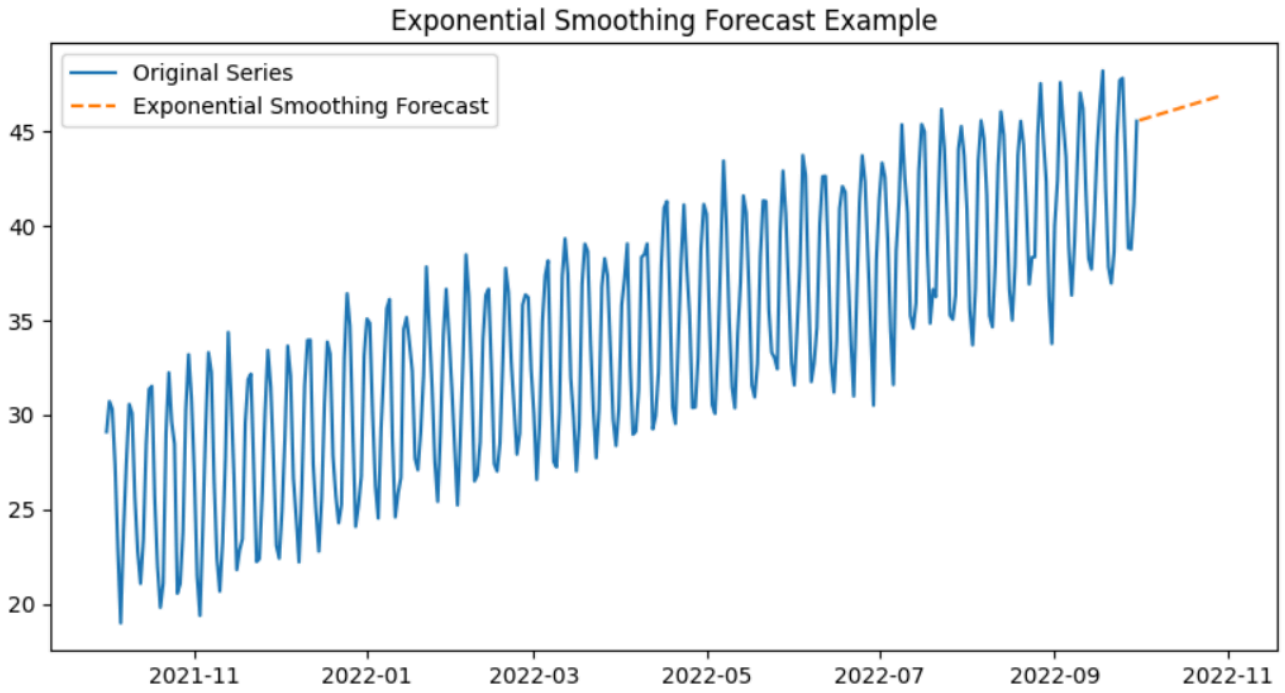
```
import matplotlib.pyplot as plt
from statsmodels.tsa.holtwinters import ExponentialSmoothing

model = ExponentialSmoothing(
    data['Sales'],
    trend="add",
    seasonal=None
)

model_fit = model.fit()
forecast = model_fit.forecast(30)

print("Forecast:")
plt.figure(figsize=(10, 5))
plt.plot(data['Sales'], label="Original Series")
plt.plot(forecast.index, forecast, label="Exponential Smoothing Forecast",
linestyle="--")
plt.legend()
plt.title("Exponential Smoothing Forecast Example")
plt.show()
```

```
Forecast:
```



15. Моделювання Кризових Явищ

Передбачення фінансових криз.

```
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

# Завантажуємо дані з ринку S&P 500 цінних паперів
sp500_data = yf.download('^GSPC', start='2000-01-01', end='2023-01-01')

print(sp500_data.head(10))
sp500_data['Return'] = sp500_data['Close'].pct_change()  # Calculate daily returns

# Завантажуємо дані про дохідність облігацій (як економічний індикатор)
bond_yield_data = yf.download('^TNX', start='2000-01-01', end='2023-01-01')
bond_yield_data['Yield'] = bond_yield_data['Close']

# Об'єднуємо датасети за датами
data = sp500_data[['Return']].merge(bond_yield_data[['Yield']], left_index=True,
right_index=True)
data.dropna(inplace=True)  # Видаляємо відсутні значення

sp500_data[:10]
```

```
Price             Close         High          Low         Open      Volume
Ticker            ^GSPC        ^GSPC        ^GSPC        ^GSPC       ^GSPC
Date
2000-01-03  1455.219971  1478.000000  1438.359985  1469.250000   931800000
2000-01-04  1399.420044  1455.219971  1397.430054  1455.219971  1009000000
2000-01-05  1402.109985  1413.270020  1377.680054  1399.420044  1085500000
2000-01-06  1403.449951  1411.900024  1392.099976  1402.109985  1092300000
2000-01-07  1441.469971  1441.469971  1400.729980  1403.449951  1225200000
2000-01-10  1457.599976  1464.359985  1441.469971  1441.469971  1064800000
2000-01-11  1438.560059  1458.660034  1434.420044  1457.599976  1014000000
2000-01-12  1432.250000  1442.599976  1427.079956  1438.560059   974600000
2000-01-13  1449.680054  1454.199951  1432.250000  1432.250000  1030400000
2000-01-14  1465.150024  1473.000000  1449.680054  1449.680054  1085900000
[*********************100%***********************]  1 of 1 completed
```

Нам потрібно визначити, що таке фінансова криза. Загальноприйнятий підхід полягає у виділенні періодів екстремального спаду на ринку, коли прибутковість фондового ринку значно падає за короткий проміжок часу.

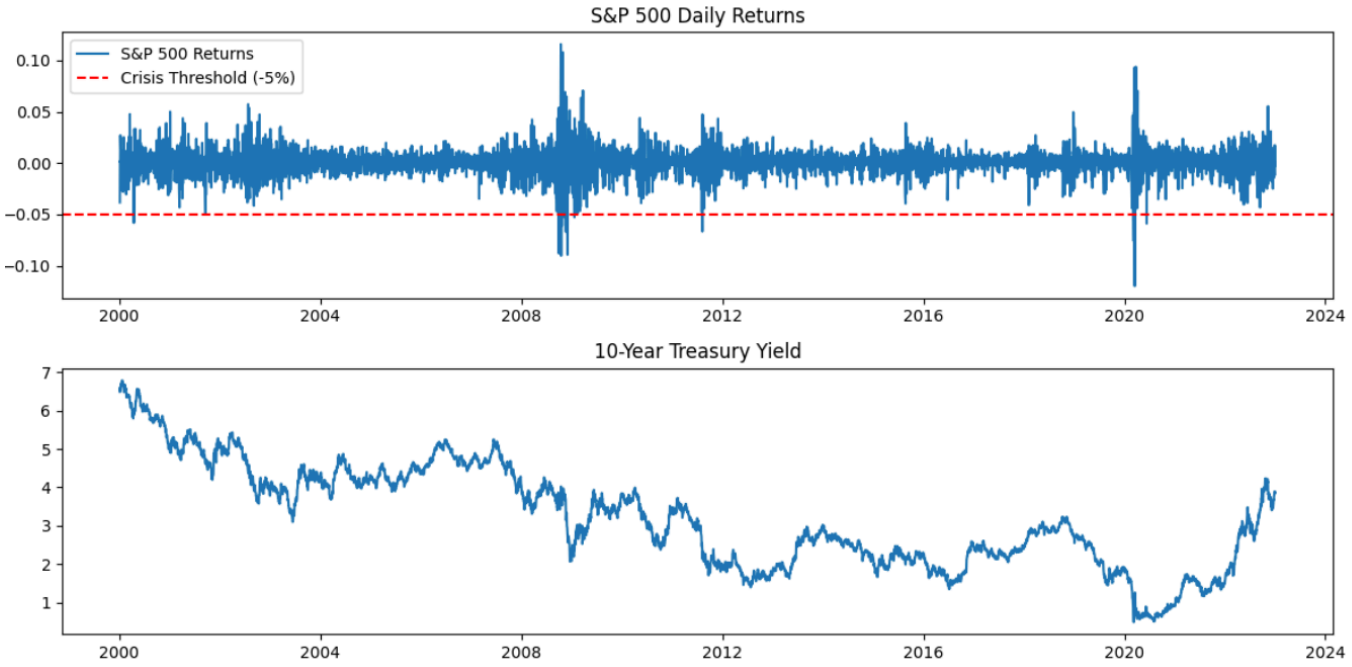
```
# Визначити кризову ситуацію, якщо прибутковість опуститься нижче певного
порогового значення (наприклад, -5 % за один день)
data['Crisis'] = (data['Return'] < -0.05).astype(int)
```

```
# Перевіряємо розподіл кризових моментів
print(data['Crisis'].value_counts())

plt.figure(figsize=(12, 6))
plt.subplot(2, 1, 1)
plt.plot(data.index, data['Return'], label='S&P 500 Returns')
plt.axhline(y=-0.05, color='r', linestyle='--', label='Crisis Threshold (-5%)')
plt.legend()
plt.title('S&P 500 Daily Returns')

plt.subplot(2, 1, 2)
plt.plot(data.index, data['Yield'], label='10-Year Treasury Yield')
plt.title('10-Year Treasury Yield')
plt.tight_layout()
plt.show()
```

```
Crisis
0    5761
1      19
Name: count, dtype: int64
```



Ми розділимо дані на навчальні та тестові набори, щоб оцінити ефективність нашої моделі.

```
# Визначаємо характеристики (економічні показники) та ціль (мітку «Криза»)
X = data[['Return', 'Yield']]
y = data['Crisis']

# Розділяємо дані на тестові та тренувальні
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
shuffle=False)
```

Ми будемо використовувати RandomForestClassifier для моделювання взаємозв'язку між економічними показниками та кризовими подіями.

```
# Будуємо модель класифікатор Випадковий Ліс
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```

RandomForestClassifier

▼ Parameters

n_estimators	100
criterion	'gini'
max_depth	None
min_samples_split	2
min_samples_leaf	1
min_weight_fraction_leaf	0.0
max_features	'sqrt'
max_leaf_nodes	None
min_impurity_decrease	0.0
bootstrap	True
oob_score	False
n_jobs	None
random_state	42
verbose	0
warm_start	False
class_weight	None
ccp_alpha	0.0
max_samples	None
monotonic_cst	None

Після навчання моделі ми оцінюємо її ефективність за допомогою матриці плутанини та звіту про класифікацію.

```
# Робимо передбачення на тестових даних
y_pred = model.predict(X_test)

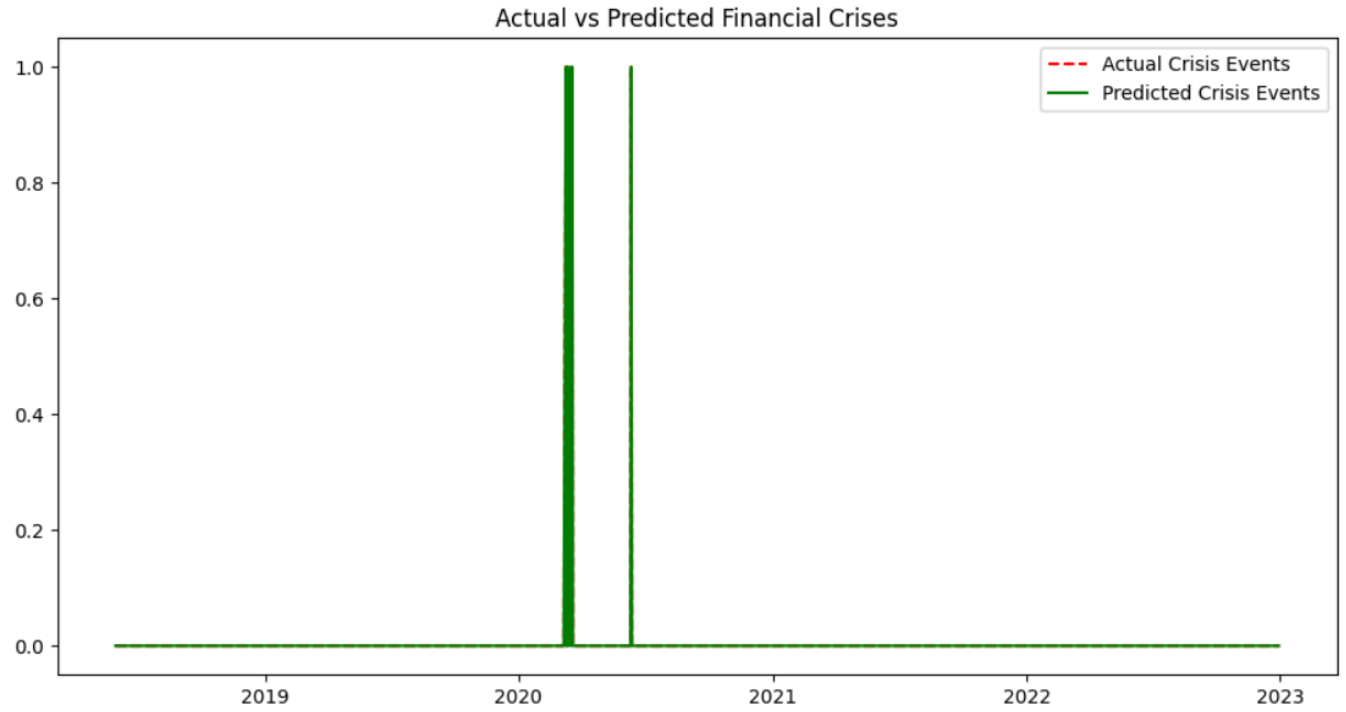
# Матриця похибок
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

# Будуємо графік порівняння прогнозованих і фактичних криз
plt.figure(figsize=(12, 6))
plt.plot(y_test.index, y_test.values, label='Actual Crisis Events', color='r',
linestyle='--')
plt.plot(y_test.index, y_pred, label='Predicted Crisis Events', color='g')
plt.title('Actual vs Predicted Financial Crises')
plt.legend()
plt.show()
```

```
[[1151    0]
 [   0    5]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      1151
           1       1.00      1.00      1.00         5

    accuracy                           1.00      1156
   macro avg       1.00      1.00      1.00      1156
weighted avg       1.00      1.00      1.00      1156
```



Моделювання розповсюдження епідемій

SIR-модель — це одна з найвідоміших математичних моделей для опису поширення інфекційних захворювань у популяції. Вона дозволяє зрозуміти, як хвороба поширюється, коли досягає піку та за яких умов епідемія згасає.

Модель була запропонована Кермаком та МакКендріком у 1927 році і досі є основою епідеміологічного моделювання.

SIR — це абревіатура від трьох станів, у яких може знаходитися кожна людина в популяції:

- S (Susceptible) — сприйнятливі: Люди, які ще не хворіють, але можуть заразитися.
- I (Infectious) — інфіковані: Люди, які хворіють і здатні передавати інфекцію іншим.
- R (Recovered/Removed) — одужалі (або видалені): Люди, які або одужали й отримали імунітет, або були ізольовані, або померли (тобто не взаємодіють із іншими).

Популяція вважається закритою, тобто:

$$N = S(t) + I(t) + R(t) = const$$

Система диференціальних рівнянь: SIR-модель описується трьома рівняннями:

$$\frac{dS(t)}{dt} = -\beta \frac{S(t) * I(t)}{N}$$

$$\frac{dI}{dt} = \beta \frac{SI}{N} - \gamma I$$

$$\frac{dR}{dt} = \gamma I$$

- β (бета) - коефіцієнт інфікування - показує, як часто відбуваються контакти та з якою ймовірністю вони ведуть до зараження.
- γ (гама) - коефіцієнт одужання - це швидкість переходу з I → R. Якщо середня тривалість хвороби — D днів, то:

$$\gamma \approx \frac{1}{D}$$

Базове репродуктивне число R_0 - найважливіший параметр епідемії:

$$R_0 = \frac{\beta}{\gamma}$$

- Якщо $R_0 > 1$ — хвороба поширюється → епідемія.
- Якщо $R_0 < 1$ — хвороба згасає.

Інтуїтивно це означає, скільки людей у середньому заразить один хворий у незахищеній популяції.

Динаміка епідемії: типовий перебіг для SIR:

- На початку багато S та мало I → інфекція швидко росте.
- Максимальне значення I, потім спадає

- З часом S зменшується, тому інфікування сповільнюється.
- Коли кількість сприйнятливих падає нижче певного порогу, хвиля епідемії природно згасає.
- Збільшується група R — тих, хто набув імунітету.

Інтерпретація кривих: на графіку зазвичай видно:

- S(t) плавно зменшується.
- I(t) різко зростає, досягає піку, потім спадає.
- R(t) зростає та стабілізується.

Пік I(t) залежить від параметрів β та γ — і саме його намагаються "сплющити" карантинними заходами.

```
import numpy as np
from scipy.integrate import odeint
import matplotlib.pyplot as plt

# Параметри SIR моделі
N = 1000         # Чисельність населення
I0 = 1           # Початкове число інфікованих
R0 = 0           # Початкове число виздоровівших/вилучених
S0 = N - I0 - R0 # Початкове число тих, хто може бути зараженим
beta = 0.5     # Рівень/коефіцієнт інфікування
gamma = 0.1      # Рівень/коефіцієнт визоровлення

# Дати
t = np.linspace(0, 160, 160)

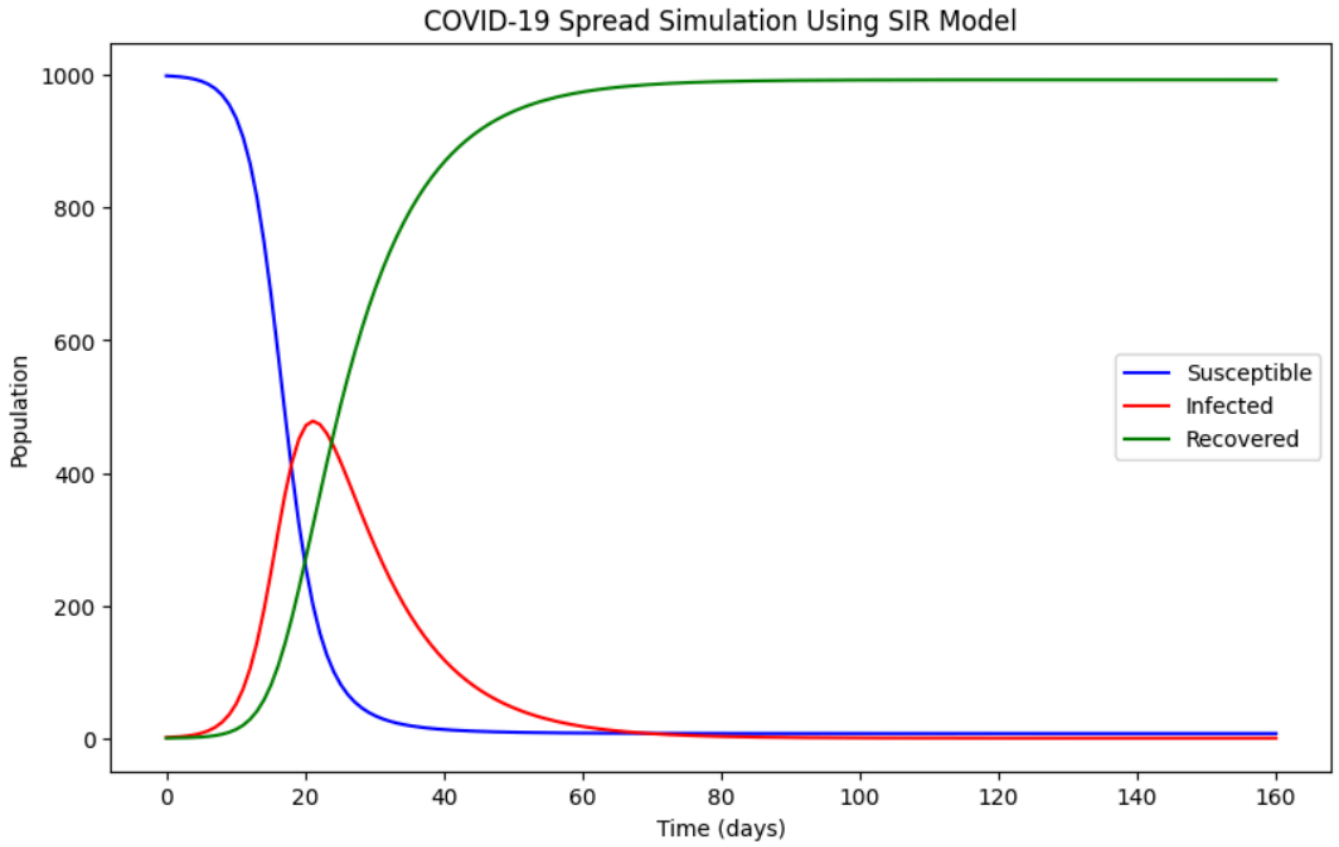
# Диференціальні рівняння SIR моделі
def sir_model(y, t, N, beta, gamma):
    S, I, R = y
    dSdt = -beta * S * I / N
    dIdt = beta * S * I / N - gamma * I
    dRdt = gamma * I
    return dSdt, dIdt, dRdt

# Вектор початкових станыв
y0 = S0, I0, R0

# SIR рівняння за часом, t
ret = odeint(sir_model, y0, t, args=(N, beta, gamma))
S, I, R = ret.T

# Будуємо графік
plt.figure(figsize=(10, 6))
plt.plot(t, S, 'b', label='Susceptible')
plt.plot(t, I, 'r', label='Infected')
plt.plot(t, R, 'g', label='Recovered')
plt.xlabel('Time (days)')
plt.ylabel('Population')
plt.title('COVID-19 Spread Simulation Using SIR Model')
```

```
plt.legend()
plt.show()
```



16. Використання нейронних мереж для аналізу даних